







中国研究生创新实践系列大赛

“华为杯”第二十三届中国研究生数学建模竞赛

学　　校	
参赛队号	
队员姓名	1.
	2.
	3.

中国研究生创新实践系列大赛
“华为杯”第二十三届中国研究生
数学建模竞赛

题　目：　基于 QIF 精确平均场模型的视觉认知脑电机理研究

摘　要：

额区脑电中的视觉响应受眨眼、心电与漂移干扰。本文以两名被试、两个项目共 400 次试验为对象，依次研究保留响应特征的降噪、视觉刺激到头皮脑电的机理，以及视觉认知过程中的持续输入。

针对问题一，按伪影的时间结构建立**分源模板去噪模型**，用事件锁时模板相减去除心电与眨眼，以高频小波截断抑制尖峰；再建立**滤波一致的响应曲线拟合模型**，采用**信赖域反射算法**拟合高斯和，按有效样本数的 BIC 选定成分数。注入检验的幅度保留率为 **0.86**，均方根误差为 **1.22** μV。项目一 Fz 正峰为 **5.12** μV，三个电极拟合 R^2 为 0.951 到 0.981；项目二呈宽负波，具体曲线见表 4 及式 (6)、式 (7)，眼电残留不能排除。

针对问题二，用高斯差分、朝向能量与 **COSFIRE 顶点编码**表示视觉刺激，以 QIF 神经元群的精确平均场和偶极正演建立 **LGN—皮层—头皮三段式脑电计算模型**，采用**多起点信赖域算法**求解，并要求选出的参数同时通过扰动稳定性与有限网络检验。项目一诱发段 R^2 为 **0.673，0.623，0.883**，峰潜伏期为 0.328s，与 16000 个脉冲神经元相比源项误差为 **1.98%**。偏侧化特征取 300 到 500ms 的 F3 减 F4 均值，半球耦合 $J = 0.5$ 时左右差为 **0.624** μV，低于本数据 **3.68** μV 的检出限，单试次解码未显示可靠区分能力。

针对问题三，截取提示到应答前 0.1s 的信号，建立**带失误的证据累加模型**与**双源输入宏观认知模型**，用行为、频带能量及跨被试预测验证。项目二平均多出 0.225s 的比对时长；维持期 delta 与 theta 绝对能量为项目一的 **3.85 倍**与 **2.84 倍**。只变海马模型 χ^2 为 **115.1**，纯慢输入为 139.8，只变丘脑为 411.9，相对支持增加带 theta 节律的持续输入。跨被试数值预测仍有偏差，行为推出的 4.444Hz 未得到频谱峰支持，结论限于当前数据与模型。

关键词：　模板去噪、精确平均场、顶点编码、双源输入、证据累加

目录

一、 问题重述 3
1.1 问题背景 3
1.2 问题重述 3
1.3 建模总体思路 4
二、 模型的假设 4
三、 符号说明 5
四、 问题一模型的建立、分析与求解 6
4.1 问题分析 6
4.2 数据读取与事件编码 6
4.3 伪影的机理分析 6
4.4 分源模板去噪模型 7
4.5 滤波一致的响应曲线拟合模型 9
4.6 模型的求解与检验 9
4.7 本章小结 13
五、 问题二模型的建立、分析与求解 13
5.1 问题分析 13
5.2 视觉通路的形状编码模型 14
5.3 前额叶群体的精确平均场模型 15
5.4 头皮正演模型 17
5.5 模型的求解与检验 18
5.6 左右三角差异的形成机制与特征表示 22
5.7 本章小结 24
六、 问题三模型的建立、分析与求解 24
6.1 问题分析 24
6.2 信号截取与应答编码 25
6.3 认知过程的机理分析 25
6.4 双源输入宏观认知模型 26
6.5 模型的求解与检验 28
6.6 本章小结 32
七、 各问结果汇总 32
八、 模型的灵敏度分析 33
8.1 高通截止频率 33
8.2 平均场参数 33
8.3 海马输入的形状 34
九、 模型的评价 35
9.1 模型的优点 35
9.2 模型的缺点 35
9.3 模型的改进 36
9.4 模型的推广 36

一、 问题重述

1.1 问题背景

脑电图记录的是大量皮层神经元同步活动在头皮上叠加出的电位，采集无创、时间分辨率高，是脑机接口与精神性疾病诊断最常用的信号。视觉刺激出现后约 250 到 500 ms，头皮上会出现一个正向的事件相关电位，即 P300[1]，它反映大脑对刺激的评估与更新，是视觉认知研究的主要观测量。困难在于单次试验的脑电里，眨眼、心电与肌电的幅度是视觉响应的十倍以上，原始波形看不出 P300；而常用的去伪影方法在去掉干扰的同时往往也削弱了视觉响应本身。

题目给出两名被试在视觉提示实验中的额区脑电，电极为 Fz、F3、F4，采样率 256 Hz。每次试验先出现一个指向左或右的三角形提示，持续约 0.2 s，约 2 s 后两侧出现目标，被试点击其一。一次试验的时序如图 1 所示，两个项目只在末帧分岔：项目一的提示给出目标位置，被试只需等待目标出现；项目二的提示只给出目标形状，目标随机出现在一侧，被试要把记住的形状与两个候选比对。每名被试每个项目各记录约 15 分钟、100 次试验，共四组数据。

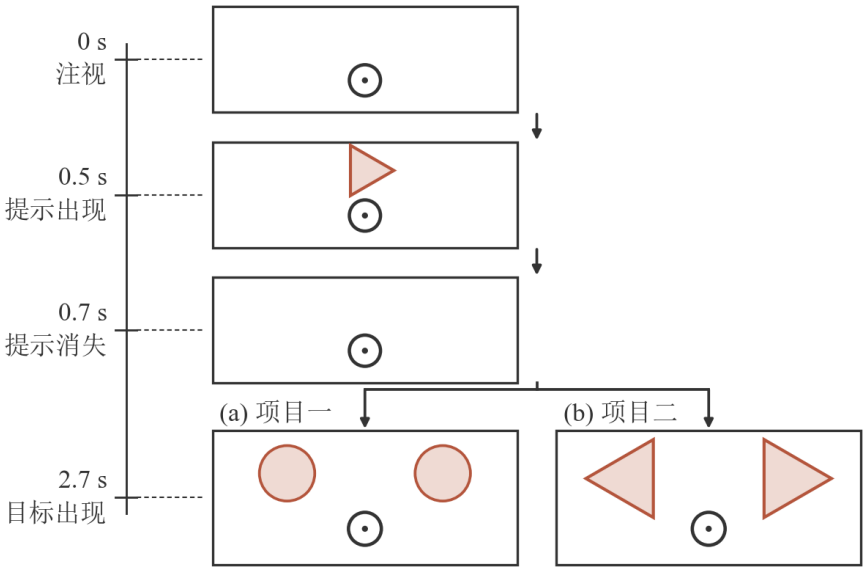


图 1　视觉提示实验的试验流程

每组数据有十个通道：通道 1 到 3 为原始脑电，通道 4 到 6 为设备自带滤波后的数据，题目要求不直接使用；通道 7 为心电；通道 8 为视觉提示，−1 为左、+1 为右；通道 9 为目标应答，±2 表示点击左侧或右侧；通道 10 为时间戳。

1.2 问题重述

在上述数据的基础上建立数学模型，完成三个递进的问题。

1. 分别对项目一与项目二的数据做预处理，设计能在保留视物形状脑电特征的前提下去除运动、眨眼等伪影的降噪方法，提取各观测点位的有效视觉响应，并拟合相应曲线。
2. 建立脑电图计算模型，说明从外侧膝状体核 LGN 到皮层、再到头皮观测脑电图的机理；用模型解释左、右三角形刺激产生的脑电信号差异的形成机制，并给出能区分左、右三角形视觉刺激响应的特征表示。
3. 截取与视觉认知相关联的信号，其终点可取应答前约 100 ms；借助问题二的脑电形成机制，并考虑信号可能同时来源于海马体与 LGN，建立认知的宏观模型，用所给数据验证，并注意应答可能错误或未及时应答。

1.3　建模总体思路

三问的关系如图 2 所示。问题一从原始脑电中得到各电极的视觉响应曲线，它既是问题二拟合的对象，也为问题三提供干净的连续信号；问题二建立从视网膜经 LGN、视皮层到前额叶，再经导体到头皮的计算模型，给出前额叶群体的动力学参数；问题三沿用这群神经元，在输入端加上海马与丘脑两路来源，把行为与脑电放进同一个宏观认知模型。

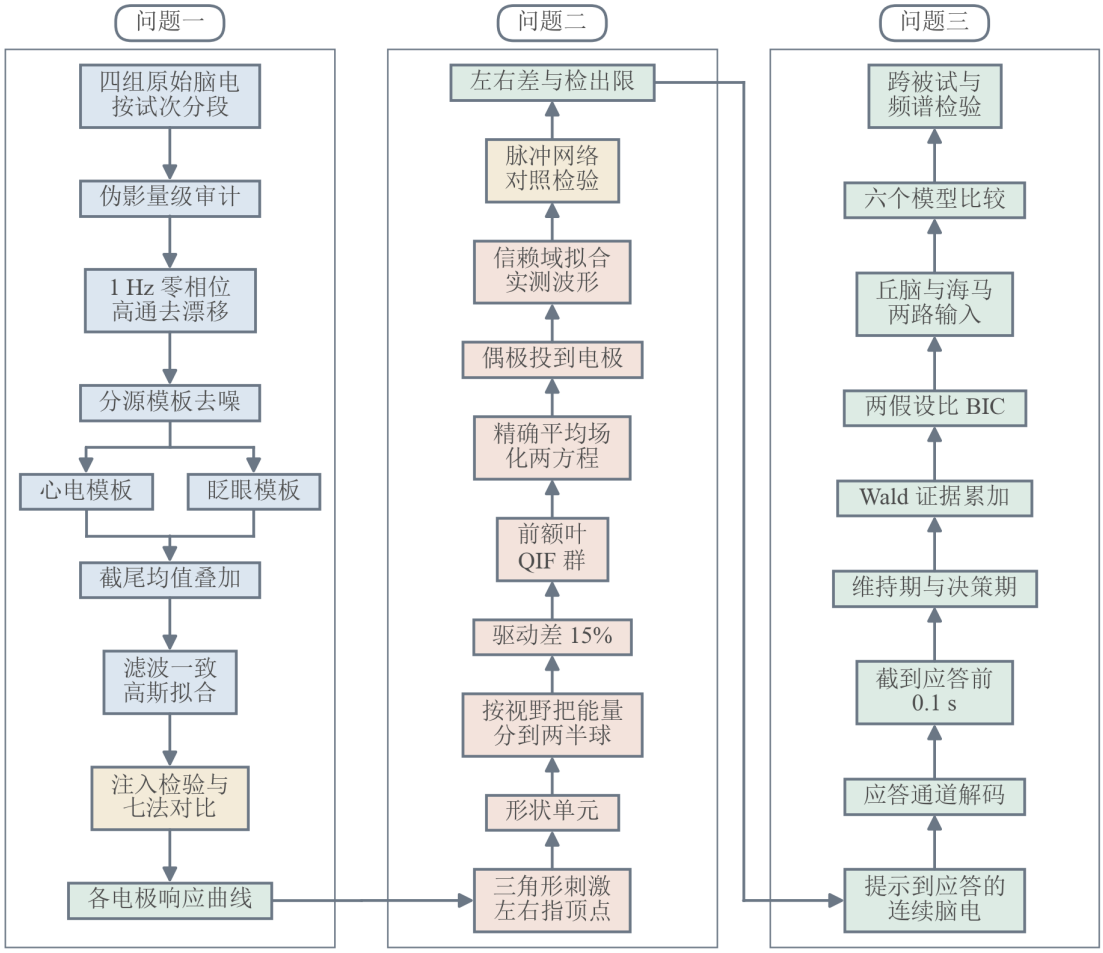


图 2　全文求解思路框架图

二、　模型的假设

1. 伪影与脑电线性叠加。同一份记录内心电与眨眼的波形形状稳定，只有幅度逐次变化。
2. 视觉响应锁时于提示出现时刻，各次试验的响应相同，试验间的背景脑电相互独立。
3. 额区三个电极记到的视觉响应主要来自两侧前额叶的锥体细胞群，每侧用一个径向等效偶极表示，偶极到电极的导联系数取无限大均匀介质的解析式。P300 的额区成分 P3a 以额叶为主要来源[1]，额区电极对它最敏感，而顶区为主的 P3b 在额区电极上较弱。
4. 前额叶神经元群由兴奋性服从洛伦兹分布的二次积分发放神经元组成，群内全连接，突触后电流按 alpha 函数变化。
5. 认知模型中两个项目的前额叶神经元群内在参数相同，任务的差异只来自输入，即海马与丘脑两路输入的强度不同。
6. 应答时刻取目标显示段的终点。第 4.2 节用项目二的点击标记验证了这一点，项目一没有点击标记，按同一规则处理。

三、 符号说明

全文用到的主要符号列于表 1，按在正文中首次出现的先后排列，其余符号在首次出现处说明。

表 1　符号说明

序号	符号	单位	含义
1	$M(t)$	μV	心电与眨眼的锁时中位数模板
2	β_k	—	第 k 个事件的模板增益
3	c_j	μV	第 j 层小波细节系数，$j=1,2,3$
4	$\bar{y}(t)$	μV	全部保留试次的稳健平均波形
5	K	—	高斯成分个数
6	B_i, μ_i, ω_i	μV、s、s	第 i 个高斯成分的幅度、潜伏期与宽度
7	$\mathbf{\Phi}$	—	形状单元对当前刺激的响应
8	S_R, S_L	—	右指与左指形状单元的响应
9	σ	—	带符号的形状指数
10	P_L, P_R	—	左、右半球的驱动强度
11	κ	—	形状指数与两半球驱动差的耦合系数
12	T_m	s	膜时间常数
13	η_0, σ_η	—	神经元兴奋性分布的中心与半宽
14	ρ	Hz	群体发放率
15	V	—	群体平均膜电位
16	R	—	Kuramoto 序参数
17	Q	—	共形映射的复变量，由 ρ 与 V 合成
18	y_e, y_i	—	兴奋、抑制突触变量
19	T_e, T_i	s	兴奋、抑制突触的时间常数
20	G_e, G_i	—	兴奋、抑制突触的耦合强度
21	J	—	两半球之间的兴奋性耦合强度
22	$u(t)$	—	前额叶群的外部输入电流
23	F, T_d, t_L	—、s、s	驱动的幅度、平滑时间尺度与延迟
24	$U_e(t)$	μV	电极 e 上记到的头皮电位
25	g_{ek}	—	偶极 k 到电极 e 的导联系数
26	c	—	源电流到头皮电位的整体增益
27	$d_{\min}$	μV	头皮左右差的可检出限
28	k	s^{-1}	证据累加的漂移率
29	θ_b	—	应答边界
30	t_d	s	非决策时间
31	ϵ	—	随机失误率
32	T_c	s	项目二多出的模板比对时长
33	σ_0, c_T, c_H	—	背景、丘脑、海马三路输入的耦合强度
34	ζ_B, ζ_α	—	背景噪声与丘脑 alpha 随机振子
35	$h(t)$	—	海马输入
36	ω_θ, ω_γ	Hz	theta 与 gamma 的频率
37	η	—	海马输入中不随 theta 起伏的慢调制份额
38	μ	—	丘脑门控系数
39	Δc_H	—	海马耦合在两项目之间的增量

四、 问题一模型的建立、分析与求解

4.1 问题分析

问题一要求在去掉伪影的同时保住视觉响应的形状，再给出各电极的响应曲线。

首先，要弄清原始脑电里有哪些干扰、各有多大。额区电极离眼睛近，眨眼在三个电极上同向出现，幅度可达上百微伏；心电随每次心搏锁时出现；肌电与动作尖峰集中在高频；部分采样还被放大器量程截在 $\pm 1000\ \mu V$。视觉响应只有几微伏，频带与眨眼重叠，所以不能靠一刀切的带通把伪影滤掉，否则响应会一起受损。

然后，按每类伪影自己的时间结构去除它。心电和眨眼都是形状稳定、时刻可知的事件，用中位数模板逐事件拟合增益后相减，只减去与事件锁时的那部分；肌电只落在高频，用只作用于最高三层细节的小波截断压掉；这些步骤合成**分源模板去噪模型**。叠加得到平均波形后，把高斯和经过与数据相同的滤波器再与之比较，建立**滤波一致的响应曲线拟合模型**，成分个数按有效样本数计算的 BIC 选定。

最后，用注入已知响应的方法检验去噪是否保住了响应：把一段已知的合成响应加进真实数据，看各方法处理后能找回多少，以此选高通截止频率，并与消融版本和三种常用做法比较；再用 R 波锁时的残余幅度检验心电模板的作用。

问题一的求解路线如图 3 所示，上两行是去噪的各步，末行是检验与曲线拟合。

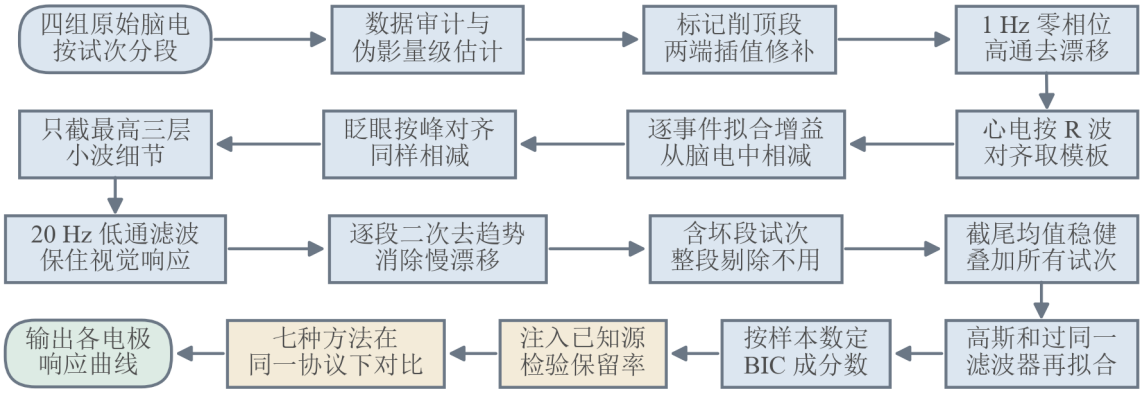


图 3 问题一思路框架图

4.2 数据读取与事件编码

四组数据各约 15 分钟、100 次试验，记为 A1、A2、B1、B2，字母为被试，数字为项目。视觉提示通道 8 的非零段起点即提示时刻，符号给出指向。应答通道 9 的编码需要先弄清：每次试验在提示后约 2.215 s 出现一段 ± 1，持续 0.82 到 1.97 s 不等；项目二在这一段结束的同一个采样点另记一段 ± 2，即题目所说的点击标记，两份数据 200 次全部如此。因此 ± 1 段在点击那一刻结束，它就是目标显示区间，两个项目的应答时刻都取它的终点，项目一没有 ± 2 标记也就有了反应时。

± 1 的符号要谨慎解读。项目二中 ± 1 与 ± 2 逐次同号，数据区分不了 ± 1 记的是目标侧还是点击侧；项目二的目标随机出现在一侧，正确的点击侧本就与提示方向无关，± 1 与提示方向只有 47% 与 48% 一致，与此相符，它的对错无法从应答通道判定。项目一的目标就在提示那一侧，不一致只能是点错，A1 全部一致，B1 有 2 次。

原始通道有三处特性要先处理。一是量程饱和，$|x| \geq 999.9\ \mu V$ 的采样 F3 上占 1.0% 到 2.0%，F4 上占 0.6% 到 3.0%，Fz 只在被试 B 的记录里出现，最多 1.7%。二是心电，通道 7 上每份记录检出 1023 到 1076 个 R 波，约每分钟 70 次。三是眨眼，每分钟 13.5 到 30.2 次，项目二明显更多，A2 为 30.2、B2 为 26.4，A1 与 B1 为 13.5 与 17.6 次。

4.3 伪影的机理分析

观测信号是四个来源的叠加，如图 4 所示。眨眼时眼睑滑过角膜，眼球这个前正后负的电偶极在额区留下上百微伏、持续约 0.3 s 的同向偏转；心脏的电活动经体表传导，每次

R 波后留下约 10 μV 的锁时波形；颞肌与额肌收缩产生 20 Hz 以上的肌电；皮层的视觉响应只有几微伏，压在最下面。四条分量画在同一比例尺下，视觉响应成了一条细波，这就是原始波形看不出 P300 的原因。

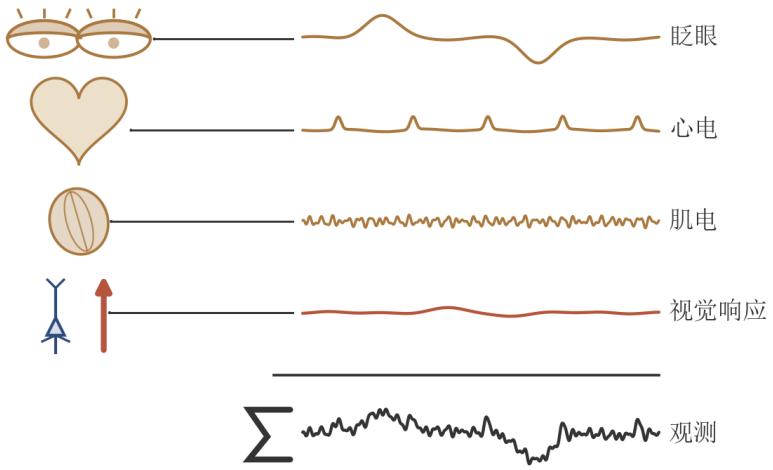


图 4　观测信号的四个来源与量级

四类干扰在时间结构上各有特点，这正是分源去除的依据：视觉响应锁时于提示，心电锁时于 R 波，眨眼锁时于眨眼峰，肌电不与任何事件锁时但只占高频。如图 5 所示，按提示对齐叠加时与提示无关的成分互相抵消，只留下锁时的那一段；按心搏对齐取中位数得心电模板，逐搏减去后只剩背景。同一个锁时性，一处提取响应，一处去除伪影。

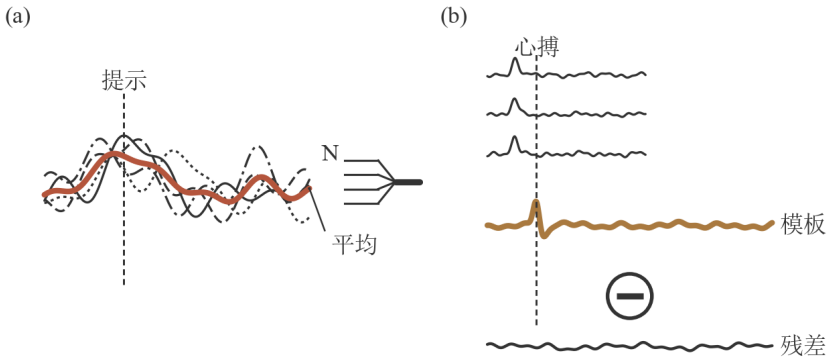


图 5　锁时性的两处用法

4.4　分源模板去噪模型

每一类伪影按自己的时间结构单独去掉，每一步做完之后波形变成什么样，如图 6 所示。

4.4.1　饱和修补与高通

饱和采样及其前后各 50 ms 标为坏段，用两端的有效采样线性插值补上，坏段标记保留下来，叠加时含坏段的试次整段剔除。随后用二阶 Butterworth 零相位高通去掉电极漂移，截止频率 f_c 由第 4.6.2 节的注入实验确定为 1 Hz。

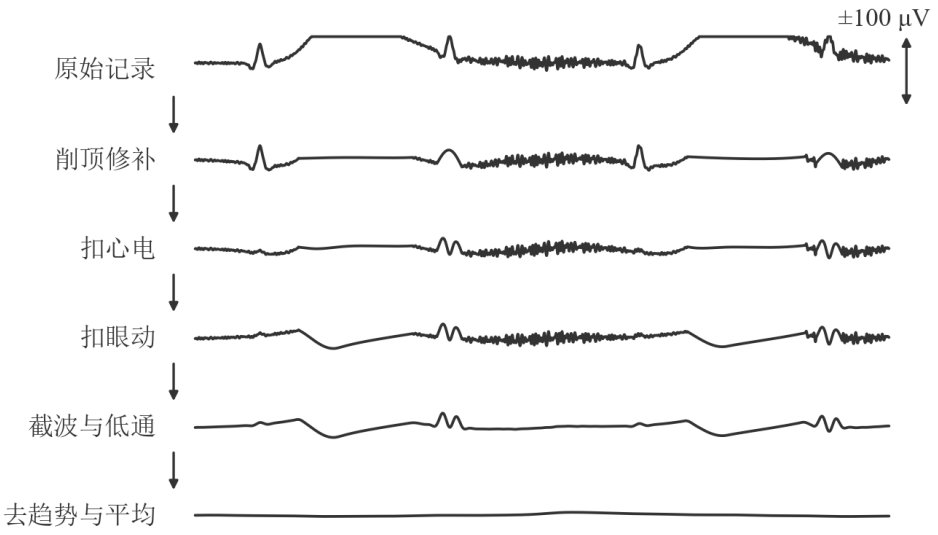


图 6　分源去噪各步之后的波形

4.4.2　心电与眨眼的模板相减

以事件时刻 t_k 对齐截段，取中位数得到模板 $M(t)$，两端各 60 ms 用线性窗压到零，避免相减时引入台阶。每个事件单独拟合增益 β_k，再减去：

$$x(t) \leftarrow x(t) - \sum_k \beta_k\, M(t - t_k), \qquad \beta_k = \operatorname{clip}\Big(\frac{\langle x_k,\, M\rangle}{\langle M,\, M\rangle},\ \beta_{\min},\ \beta_{\max}\Big), \tag{1}$$

其中 x_k 是以 t_k 为中心的一段信号。心电取 R 波前 0.25 s 到后 0.55 s，增益限在 0 到 3；眨眼取峰前 0.4 s 到后 0.6 s，增益限在 −6 到 6，并允许 ±60 ms 的对齐微调，因为眨眼的形状比心电更不规则。上述过程迭代两次，第二次在第一次减完的信号上重估模板。

眨眼峰在三个电极带通 0.5 到 8 Hz 后的逐点中位数上检测，阈值取 $\max(35\ \mu\text{V},\ 5\hat{s})$，$\hat{s}$ 为该信号的稳健标准差，并要求峰宽在 40 ms 到 1 s。取中位数而不是均值，是为了只认三个电极同向的大偏转，单个电极上的接触噪声不会被当成眨眼。

与独立成分分析相比，模板相减不需要把信号分解成若干个统计独立的源。三通道的独立成分分析只有三个成分，眨眼与视觉响应都在三个电极上同向出现，很难被分到不同成分里；模板相减只减去与眨眼时刻锁时的部分，与提示锁时的视觉响应不受影响。第 4.6.3 节的对比证实了这一点。

4.4.3　高频小波截断

肌电与运动带来的动作尖峰不与任何事件锁时，但集中在 16 Hz 以上。对信号做 6 层 sym8 小波分解，只对第 1 到 3 层细节，即 64 到 128、32 到 64、16 到 32 Hz 三个频段做幅度截断，近似层与低频细节不动[2]：

$$c_j(n) \leftarrow \operatorname{sign}\big(c_j(n)\big)\,\min\big(|c_j(n)|,\ 5\hat{s}_j\big), \qquad \hat{s}_j = \frac{\operatorname{median}|c_j|}{0.6745}, \qquad j = 1, 2, 3. \tag{2}$$

阈值取 5 倍稳健标准差，正常脑电的系数几乎都在阈值以下，只有尖峰被削平。之后用四阶 20 Hz 零相位低通去掉电极噪声。

4.4.4　逐段去趋势、剔除与稳健叠加

以提示时刻为零点截取 −0.5 到 1.2 s 的段，逐段减去二次多项式趋势，以 −0.2 到 0 s 为基线。−0.2 到 0.6 s 内峰峰值超过 $\min(120\ \mu\text{V},\ \text{med} + 3\,\text{MAD})$ 或含饱和坏段的试次剔除，余下的试次按采样点取 10% 截尾均值，削弱个别残留伪影的影响。

综上，分源模板去噪模型为

$$\begin{cases} x_1 = h_{\mathrm{HP}} * \mathcal{S}(x), & \mathcal{S}\ \text{为饱和段插值修补} \\ x_2 = x_1 - \sum_k \beta_k^{E}\, M^{E}(t - t_k^{E}), & t_k^{E}\ \text{为 R 波时刻} \\ x_3 = x_2 - \sum_k \beta_k^{B}\, M^{B}(t - t_k^{B}), & t_k^{B}\ \text{为眨眼峰时刻} \\ x_4 = h_{\mathrm{LP}} * \mathcal{W}^{-1}\big(\,\mathrm{clip}_{5\hat{s}_j}\,\mathcal{W} x_3\big), & j = 1,2,3 \\ \bar{y}(t) = \mathrm{TrimMean}_{10\%}\{\mathcal{D}\, x_4(t_m + t) :\ m \in \mathcal{H}\}, & \mathcal{H}\ \text{为保留的试次} \end{cases} \tag{3}$$

其中 $\mathcal{W}$ 为小波变换，$\mathcal{D}$ 为逐段去趋势与基线校正，t_m 为第 m 次提示时刻。

4.5 滤波一致的响应曲线拟合模型

平均波形 $\bar{y}(t)$ 由若干个正负成分组成，用高斯和刻画。关键在于 $\bar{y}$ 已经过 1 Hz 高通与 20 Hz 低通，而宽的成分经过高通后会出现反向的旁瓣。若直接用高斯和去拟合，旁瓣会被误认成独立成分，所以先让模型经过同一个滤波器 q，再与数据比较：

$$\hat{f}(t) = q * \sum_{i=1}^{K} B_i \exp\!\Big(-\frac{(t-\mu_i)^2}{2\omega_i^2}\Big). \tag{4}$$

B_i 是滤波前的幅度，μ_i 为潜伏期，ω_i 为宽度。成分之间按潜伏期排序，相邻至少相隔 60 ms，避免两个成分重合在同一处互相抵消。

拟合窗取 −0.1 到 0.65 s，共 $n = 192$ 个采样点。数据经 20 Hz 低通后以 256 Hz 采样，相邻点强相关，独立样本远少于 n，按奈奎斯特间隔估计的有效样本数为 $n_e = n \cdot 2 \cdot 20/256 = 30$。若直接用 n 计算 BIC，会高估数据量而偏向多成分，所以成分个数由有效样本数的 BIC 决定。综上，滤波一致的响应曲线拟合模型为

$$\min_{K,\ \{B_i,\mu_i,\omega_i\}} \ \mathrm{BIC}_e = n_e \ln\frac{\mathrm{RSS}}{n} + 3K\ln n_e, \qquad \mathrm{RSS} = \sum_{t} \big(\bar{y}(t) - \hat{f}(t)\big)^2$$

$$\text{s.t.}\ \begin{cases} \hat{f}(t)\ \text{由式 (4) 给出}, \\ |B_i| \le 80\ \mu\mathrm{V}, \quad 0.02 \le \omega_i \le 0.30\ \mathrm{s}, \\ 0.05 \le \tau_1 \le 0.6\ \mathrm{s}, \quad 0.06 \le \tau_{i+1} - \mu_i \le 0.06 + \delta_K, \\ K \in \{1,2,3,4\}, \end{cases} \tag{5}$$

其中 $\delta_K = (0.49 - 0.06(K-1))/(K-1)$，使各成分合起来的跨度不超过 0.55 s。对每个 K 用信赖域反射算法[3] 从 16 个随机起点最小化 RSS，取 RSS 最小者，再在 K 之间比较 BIC_e。滤波用一次 FFT 卷积完成，不对每个候选参数重复做零相位滤波。幅度与宽度的上界起初取 25 μV 与 0.15 s，项目二的宽负波在这一上界下两项成分都顶到边界，拟合曲线在提示前偏离数据，R^2 只有 0.56 到 0.69，因此放宽到 80 μV 与 0.30 s。

4.6 模型的求解与检验

4.6.1 注入检验的设计

去噪效果无法直接看，因为真实的视觉响应本身未知。本文在每份原始数据的每次试验中，于目标出现后 3.5 s 处加入一段已知的合成响应，这个时刻离真实的提示响应与目标响应都远。合成响应由一个 0.33 s、+6 μV、宽 45 ms 的正峰与一个 0.45 s、−4 μV、宽 60 ms 的负峰组成，三个电极共有；F3 与 F4 上另加一个 0.40 s、$\pm$3 μV 的反号成分，方向随机。处理后按注入时刻叠加，与已知的真值比较三个指标：幅度保留率 $\langle \hat{z}, z\rangle / \langle z, z\rangle$，越接近 1 越好；与真值的相关；均方根误差。协议如图 7 所示。

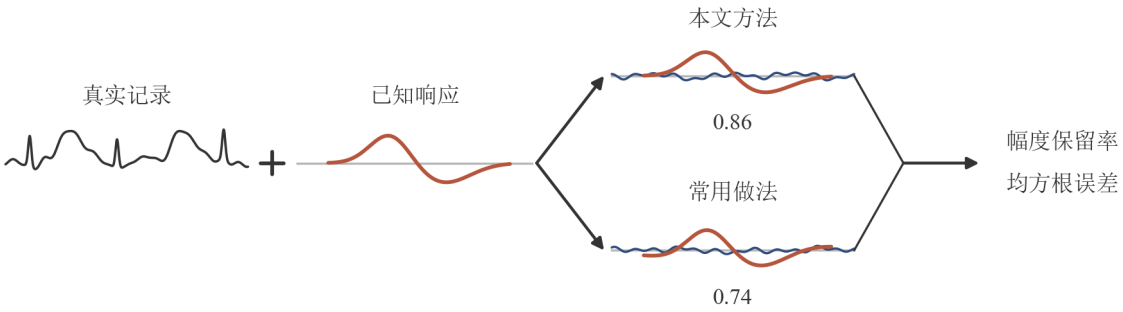


图 7　注入检验的协议

表 2　高通截止频率对注入响应的影响

截止频率/Hz	幅度保留率	与真值相关	均方根误差/μV
0.3	0.96	0.794	2.20
0.5	0.92	0.848	1.85
0.75	0.86	0.881	1.45
1	0.86	0.901	1.22
1.5	0.71	0.888	1.17
2	0.59	0.838	1.35
3	0.39	0.716	1.72

4.6.2　高通截止频率的选择

高通截止越高，漂移去得越干净，但视觉响应的低频部分损失越多。在 0.3 到 3 Hz 之间取七档，完整流水线只换高通截止，结果见表 2，数值为四组数据三个电极共 12 个组合的平均。

截止频率从 0.3 Hz 升到 1 Hz，相关从 0.794 升到 0.901，误差从 2.20 降到 1.22 μV，这是漂移被去掉的收益；再往上，幅度保留率从 0.86 跌到 3 Hz 时的 0.39，响应本身开始被削掉。1 Hz 处相关最高、误差接近最低、幅度保留 0.86，因此取 1 Hz。若把流水线的高通换成 2 Hz，幅度只剩 0.59，说明较高的高通截止会显著削弱视觉响应，这是题目所说现有滤波破坏视觉特征的一个原因。

4.6.3　多方法对比

在同一套注入数据上比较七种方法：本文方法，它的三个消融版本，即分别去掉小波截断、眨眼模板与心电模板，以及三种常用做法，即 1 到 20 Hz 带通后做同样的去趋势与剔除、以 2 Hz 高通近似的设备滤波、三通道 FastICA 去眼电[4, 5]。ICA 去掉的是幅度包络与眨眼时刻相关性最强的那个成分。结果见表 3。截止扫描与方法对照使用不同的注入检验批次，因此同一方法的平均相关系数可有末位差异。

表 3　七种去噪方法在注入检验上的对比

方法	幅度保留率	与真值相关	均方根误差/μV	单份用时/s
本文方法	0.86，0.64 到 1.08	0.902	1.22	0.33
去掉小波截断	0.86，0.64 到 1.08	0.902	1.22	0.36
去掉眨眼模板	0.97，0.49 到 1.40	0.891	1.81	0.15

续表

方法	幅度保留率	与真值相关	均方根误差/μV	单份用时/s
去掉心电模板	0.88，0.63 到 1.05	0.898	1.22	0.27
带通加同样的剔除	1.00，0.50 到 1.40	0.892	1.72	0.03
2 Hz 高通，无剔除	0.74，0.58 到 0.87	0.853	1.50	0.03
三通道 FastICA	0.01，-0.27 到 0.17	0.115	2.44	0.15

注：前五行都含削顶修补、去趋势与试次剔除，只在高通与模板两步上不同；「2 Hz 高通，无剔除」一行只有高通，不含削顶与剔除，与其余各行不同口径。

表 3 给出三点结论。第一，本文方法的误差为 1.22 μV，与两个消融版本并列最低，比带通加剔除低 29%，比只做 2 Hz 高通低 19%，幅度保留率 0.86，还有 14% 的系统损失，主要来自 1 Hz 高通。带通的保留率虽然接近 1，但 12 个组合里从 0.50 到 1.40 分散很开，平均接近 1 是高估与低估相互抵消的结果。第二，起主要作用的是眨眼模板，去掉它误差升到 1.81 μV；去掉小波截断或心电模板误差不变，这两步要另作检验。第三，三通道 FastICA 几乎把响应全部去掉，幅度只剩 0.01，因为注入的响应与眨眼一样在三个电极上同向出现，落在同一个成分里被一起去掉，这正是现有方法破坏视觉特征的一个具体机制。

注入检验看不出心电模板与小波截断的作用，因为心电与提示不锁时，叠加后自然衰减，注入时刻也避开了强肌电段。心电模板按 R 波锁时直接检验：不减模板时四份数据三个电极上 R 波锁时平均的峰峰值为 8.4 到 11.9 μV，减去后降到 1.1 到 2.1 μV，去掉了约 85%，这部分心电在单试次上与视觉响应同量级。小波截断几乎不起作用：有无它，被剔除的试次数完全相同，13 到 20 Hz 的能量只降低 0 到 1.5%，说明这批记录中超过 5 倍稳健标准差的高频尖峰很少；保留这一步不影响结果，只在肌电强的记录上起防护作用。

4.6.4 各电极的有效视觉响应

去噪前后的一段 A2 记录如图 8 所示。为便于显示，(a) 只去掉了 0.3 Hz 以下的缓慢漂移，三道之间相隔 400 μV，(b) 相隔 60 μV。(a) 中约 0.6 s 与 3.6 s 处的两次眨眼在三个电极上造成同向偏转，F3 在 6.1 到 6.8 s 饱和，F3 与 F4 在饱和前后还有数百微伏的慢漂移；(b) 为去噪后的同一段，饱和段与漂移已去除，眨眼的大幅偏转被压到与背景相当的水平，约 0.6 s 处仍可见三个电极同向的残余下陷。

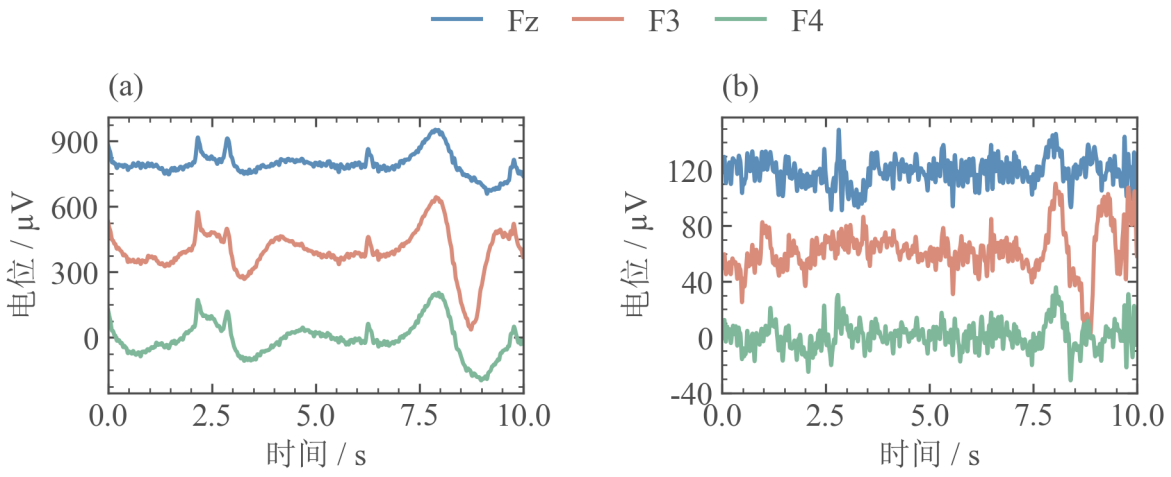


图 8　去噪前后的原始记录（通道按 Fz、F3、F4 排列，左图逐通道错开 400 μV，右图错开 60 μV）

两被试按项目合并后各电极的平均响应与拟合曲线如图 9 所示，峰值与拟合优度见

表 4。

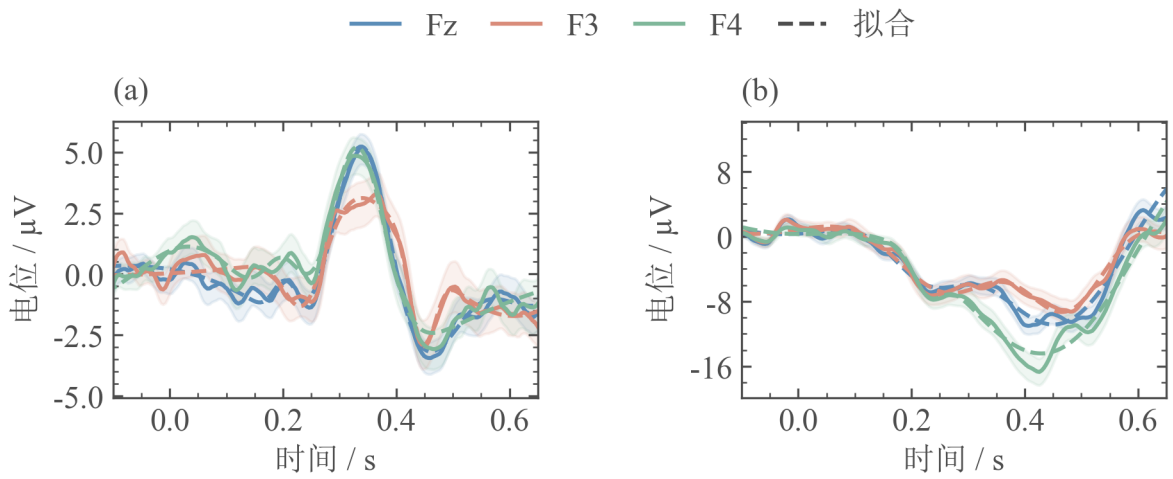


图 9　各电极的有效视觉响应与拟合曲线

表 4　两被试合并后各电极的有效视觉响应

项目	电极	试次数	曲线峰潜伏期/s	峰值/μV	t 值	成分数 K	拟合 R^2
一	Fz	193	0.340	+5.12	+8.2	4	0.981
一	F3	182	0.363	+3.28	+3.7	4	0.951
一	F4	179	0.324	+4.85	+6.4	3	0.964
二	Fz	198	0.406	−11.01	−10.5	3	0.948
二	F3	179	0.480	−9.26	−6.7	4	0.976
二	F4	179	0.426	−16.69	−10.2	3	0.974

注：曲线峰潜伏期是去噪后平均响应在 0.25 到 0.55 s 内绝对值最大的时刻，与正文所称主成分潜伏期不是同一个量，后者是主高斯分量的中心 μ_i，两者在项目一 F3 上相差约 29 ms。t 值由峰值除以逐试次幅度的标准误得到。

项目一三个电极在提示后 0.32 到 0.36 s 都出现正峰，Fz 最强，为 +5.12 μV，t = 8.2，落在 P300 的 250 到 500 ms 窗内。拟合曲线为

$$
\begin{aligned}
\hat{f}_{\mathrm{Fz}} &= q * [1.55G(0.201, 0.020) + 9.32G(0.337, 0.050) \\
&\quad - 8.55G(0.455, 0.222) + 2.30G(0.544, 0.057)], \\
\hat{f}_{\mathrm{F3}} &= q * [-2.55G(0.234, 0.033) + 2.34G(0.334, 0.064) \\
&\quad - 3.68G(0.442, 0.021) - 4.09G(0.604, 0.122)], \\
\hat{f}_{\mathrm{F4}} &= q * [5.18G(0.050, 0.113) + 2.87G(0.199, 0.035) + 9.09G(0.326, 0.049)],
\end{aligned}
\tag{6}
$$

其中 $G(\tau, \sigma) = \exp(-(t-\tau)^2/2\sigma^2)$，幅度单位为 μV，时间单位为 s。三个电极的正峰主成分潜伏期为 0.337、0.334、0.326 s，相差 11 ms，宽度 49 到 64 ms，R^2 为 0.951 到 0.981。F3 的正峰较弱，前后各有一个负成分，这与 F3 上饱和段较多、有效试次较少有关。Fz 首个成分的宽度与 F4 首个成分的潜伏期落在约束的下界上，这两个成分描述提示后 0.2 s 内的小起伏，对正峰的参数没有影响。

项目二同一段是一个宽负波，峰在 0.41 到 0.48 s，F4 最强，为 −16.69 μV。拟合曲线为

$$\begin{aligned}\hat{f}_{\mathrm{Fz}} &= q * [-64.12G(0.145,0.300) - 8.23G(0.223,0.065) - 36.58G(0.464,0.142)],\\ \hat{f}_{\mathrm{F3}} &= q * [-80.00G(0.114,0.300) - 34.68G(0.298,0.116)\\ &\quad - 26.30G(0.472,0.079) - 46.95G(0.656,0.157)],\\ \hat{f}_{\mathrm{F4}} &= q * [-80.00G(0.087,0.264) - 3.94G(0.224,0.025) - 80.00G(0.453,0.193)],\end{aligned} \tag{7}$$

R^2 为 0.948 到 0.976，峰附近的负成分潜伏期为 0.464、0.472、0.453 s，三个电极一致。需要说明的是，负波很宽，经 1 Hz 高通后它的低频部分被滤掉，滤波前的幅度与宽度可以互相替代：宽度加大、幅度同时加大，滤波后的曲线几乎不变。所以式 (7) 中有几项幅度顶到了 80 μV 或宽度顶到了 0.30 s 的上界，单个成分的幅度并不可辨识，可辨识的是滤波后的曲线本身与主成分的潜伏期，表 4 因此给出实测峰值。

两被试分开看，项目一的正峰两人都有。项目二的负波在两人的全部试次平均中都有，但项目二眨眼更频繁，需要核查它是否来自残留的眼电。按提示后 0.3 到 1.0 s 内有无眨眼把试次分开，眨眼少的试次每组只有 6 到 9 次：被试 B 在这些试次上仍有负波，Fz 为 −9.15 μV，F4 为 −17.81 μV；被试 A 在这些试次上没有负波。样本太少，这一核查不能排除项目二的负波部分来自眼电残留，把它定为视觉响应需要保留这一限定。

4.7 本章小结

本章建立了分源模板去噪模型与滤波一致的响应曲线拟合模型。心电与眨眼按事件锁时的模板逐事件拟合增益后相减，高通截止由注入实验选定为 1 Hz。注入检验表明，本文方法的误差为 1.22 μV，低于带通加同样剔除的 1.72 与只做 2 Hz 高通不加剔除的 1.50 μV，幅度保留 0.86；三通道 FastICA 会把与眨眼同向分布的视觉响应一起去掉，幅度只剩 0.01；心电模板使 R 波锁时残余从约 10 μV 降到约 1.6 μV，小波截断在这批数据上几乎不起作用。项目一三个电极在提示后 0.33 s 左右出现正峰，Fz 为 +5.12 μV，拟合 R^2 为 0.951 到 0.981；项目二同一段为宽负波，峰在 0.41 到 0.48 s，拟合 R^2 为 0.948 到 0.976，但它是否部分来自眼电残留不能排除。这两组曲线是问题二拟合的对象。

五、 问题二模型的建立、分析与求解

5.1 问题分析

问题二要求说明视觉刺激经 LGN、皮层到头皮的形成机理，解释左右三角形的脑电差异并给出区分特征。题目指出这条通路曾按反问题处理，因解不唯一、传输又复杂，实用价值有限，所以本文走正演：从刺激的形状出发逐段算到头皮电位，中间每一段都写成可积分的方程。

首先，把通路拆成三段。视觉刺激在视网膜与 LGN 被分解为明暗对比，在初级视皮层被分解为朝向，再在高级视皮层按特定空间方式组合成形状。题目提示的猴脸实验说明，颞下皮层的神经元识别的是按特定空间关系排列的局部特征。三角形的局部特征就是三个顶点，本文用 COSFIRE 顶点编码[6] 刻画形状表征，并把初级视皮层的朝向能量按视野分到两半球，得到左右三角形在两半球产生的驱动。

然后，头皮上记到的不是单个神经元，而是一大群锥体细胞同步活动的叠加[7]。本文用 Montbrió 等人的精确平均场[8] 描述前额叶的二次积分发放神经元群，下文简称 QIF 神经元群，它把上万个神经元的动力学严格化简为发放率与平均膜电位两个方程，并通过共形映射直接给出群体相位的集中程度，即 Kuramoto 序参数[9]。突触按 Byrne 等人的做法取 alpha 函数[10]，以净突触电流为偶极源，投到电极上，建立 **LGN—皮层—头皮三段式脑电计算模型**。

最后，用多起点信赖域算法把模型拟合到问题一的平均波形，与五种优化器对比；用上万个脉冲神经元的直接模拟检验平均场；再由模型推出左右差异在头皮上的大小，与本数据能测出的最小差比较，并用四类特征配四种分类器做单试次解码。

问题二的求解路线如图 10 所示，上一行是视觉通路的形状编码与两半球驱动，中间一行是前额叶群体的动力学与头皮正演，下一行是检验与左右差的判定。

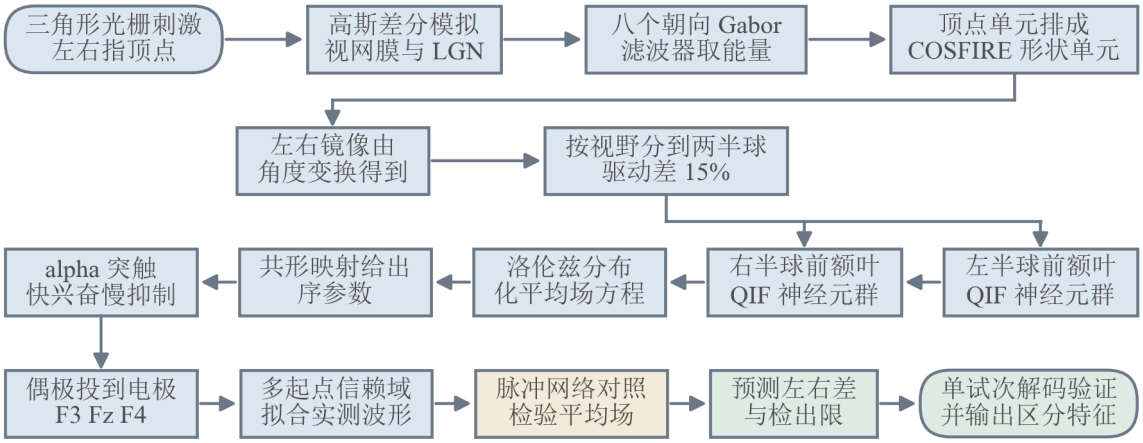


图 10　问题二思路框架图

5.2　视觉通路的形状编码模型

5.2.1　视网膜、LGN 与初级视皮层

刺激经视网膜与 LGN 中继后到达初级视皮层，再沿腹侧流经颞下皮层到达前额叶，电极记到的是前额叶皮层偶极在头皮上的投影。刺激图像 $I(x,y)$ 取 97×97 的网格，注视点在中心，三角形边长 22 个像素，位于注视圆上方。视网膜神经节细胞与 LGN 的中心环绕感受野用高斯差分描述，分为给光与撤光两路：

$$L_{\mathrm{on}} = \left[G_{\sigma_c} * u - G_{\sigma_s} * u\right]_+, \qquad L_{\mathrm{off}} = \left[G_{\sigma_s} * u - G_{\sigma_c} * u\right]_+, \qquad \sigma_c = 1,\ \sigma_s = 2.5. \tag{8}$$

初级视皮层的复杂细胞用 8 个朝向的 Gabor 正交对取能量：

$$A_\theta = \sqrt{\left(\psi_\theta^{\mathrm{e}} * (L_{\mathrm{on}} - L_{\mathrm{off}})\right)^2 + \left(\psi_\theta^{\mathrm{o}} * (L_{\mathrm{on}} - L_{\mathrm{off}})\right)^2}, \qquad \theta = \frac{m\pi}{8},\ m = 0,\dots,7, \tag{9}$$

其中 $\psi_\theta^{\mathrm{e}}, \psi_\theta^{\mathrm{o}}$ 为偶对称与奇对称的 Gabor 核，波长 7，包络宽 2.5，长宽比 0.6。

5.2.2　顶点单元与 COSFIRE 形状单元

顶点由两条边在一点相交构成。张角 60°、指向 β 的顶点单元取两条边方向上的朝向能量，沿各自边的方向外移 5 个像素后取几何平均，β 每隔 30° 取一档，共 12 档。形状单元把若干个顶点单元按极坐标 (r_i,θ_i) 排成原型形状，响应取加权几何平均：

$$\Phi(x,y) = \Big(\prod_i e_{beta_i}\big(x + r_i\cos\theta_i,\ y + r_i\sin\theta_i\big)^{\alpha_i}\Big)^{1/\sum_i \alpha_i}, \qquad \alpha_i = \exp\Big(-\frac{r_i^2}{2\sigma_\rho^2}\Big). \tag{10}$$

几何平均要求每个顶点都在，缺一个响应就接近零，这正是题目中颞下皮层神经元的性质：轮廓包裹双眼、双眼位于嘴部上方，任一特征缺失响应就可以忽略。

对右指三角形配置得到三个顶点：指向 30° 与 330° 的两个顶点位于 $\rho = 4.12$、$\theta = \mp 1.82$ rad，指向 270° 的顶点位于 $\rho = 9.49$、$\theta = -0.32$ rad。左指三角形是它的镜像，关于竖直轴反射只需把 $\beta \to 180^\circ - \beta$、$\theta \to \pi - \theta$，如图 11 所示。右指形状单元对右指三角的响应为 0.670，对左指三角只有 0.232，相差 2.9 倍，左指形状单元正好相反。

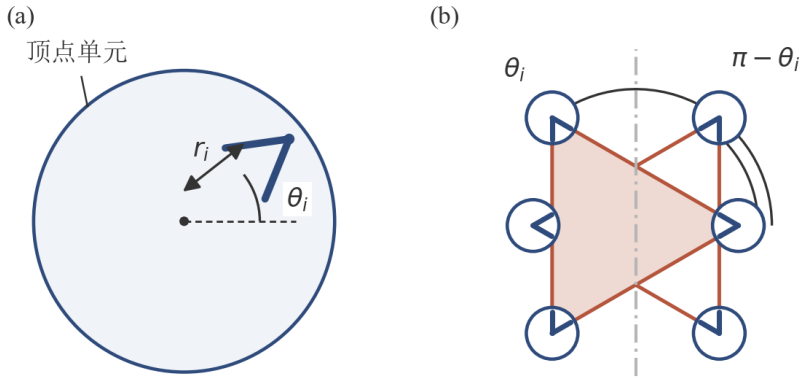


图 11　顶点单元的极坐标定义与镜像对照

5.2.3　两半球的驱动

左视野投射到右半球，右视野投射到左半球，如图 12 所示。把初级视皮层的朝向能量按视野的左右两半分别求和，得到两半球的驱动 P_L 与 P_R。右指三角形的竖直底边落在注视点左侧，边缘能量更多地进入左视野，右半球的驱动为 1252，左半球为 1086，相差 15%；左指三角形正好相反。

形状就是这样进入模型的。指向哪个方向、竖直底边落在注视点哪一侧，这本身就是形状的几何，它决定边缘能量落在视野两半上的多少，也就决定了两半球里哪些神经元被驱动、被驱动得多强。题目要求建立刺激形状与响应神经元分布之间的联系，本文用下面两步把它写实：先由几何得到两半球的能量，再由能量定两群神经元的驱动。

形状单元给出的响应是同一张能量图的另一种读法。设右指与左指形状单元对当前刺激的响应为 S_R 与 S_L，取

$$\sigma = \frac{S_R - S_L}{S_R + S_L} \tag{11}$$

作为带符号的形状指数。右指三角的 $S_R = 0.670$、$S_L = 0.232$，得 $\sigma = +0.49$；左指三角两者互换，得 $\sigma = -0.49$。把两半球的驱动也写成形状指数的函数

$$P_{R,L} = \bar{P}\,(1 \pm \kappa\sigma), \tag{12}$$

两半球驱动之和为 $2\bar{P}$、之差为 $2\bar{P}\kappa\sigma$，归一化的不对称就是 $\kappa|\sigma|$，取 $\kappa = 0.15$ 得 0.073，与按视野能量直接求和得到的 0.071 相当，两种写法给出同一个 Λ。也就是说，形状到神经元分布这一段既可以按视野能量算，也可以按形状指数算，二者等价；响应分布的强弱直接决定偶极强弱，再经正演得到头皮电位，这就是观测脑电与响应神经元分布之间的联系。形状单元的作用在这里是显式的：它把「是哪个三角」压缩成一个带符号的数，模型据此决定驱动偏向哪一半球。

5.3　前额叶群体的精确平均场模型

5.3.1　从单个神经元到群体

前额叶一群 N 个二次积分发放神经元，第 j 个的膜电位满足

$$T_m \dot{V}_j = V_j^2 + \eta_j + T_m I_{\mathrm{syn}}(t), \qquad V_j \geq V_p \;\Rightarrow\; V_j \leftarrow -V_p, \tag{13}$$

兴奋性 η_j 服从中心 η_0、半宽 σ_η 的洛伦兹分布。$N \to \infty$、$V_p \to \infty$ 时，膜电位的分布在任意时刻都保持洛伦兹形，其中心就是平均膜电位 V，半宽等于 $\pi T_m \rho$，ρ 为发放率。由此得到精确的两维平均场[8]：

$$T_m \dot{\rho} = \frac{\sigma_\eta}{\pi T_m} + 2r v, \qquad T_m \dot{V} = V^2 + \eta_0 + T_m I_{\mathrm{syn}}(t) - (\pi T_m \rho)^2. \tag{14}$$

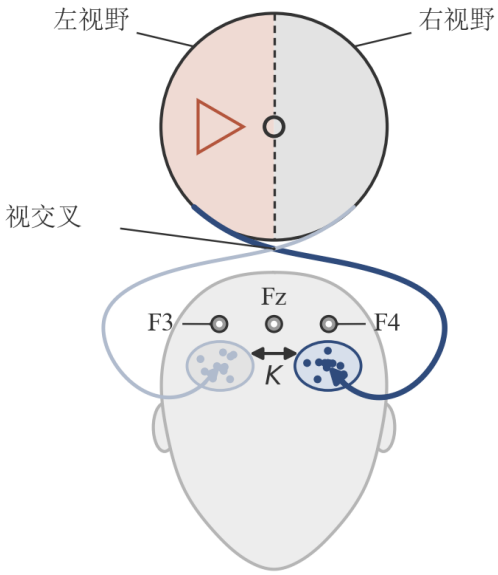


图 12　视野到半球的交叉投射

从单个神经元到群体这一步的化简如图 13 所示：单个神经元的行为由膜电位的二次项决定，兴奋性按洛伦兹分布散开的一群神经元被严格化成两个宏观量。

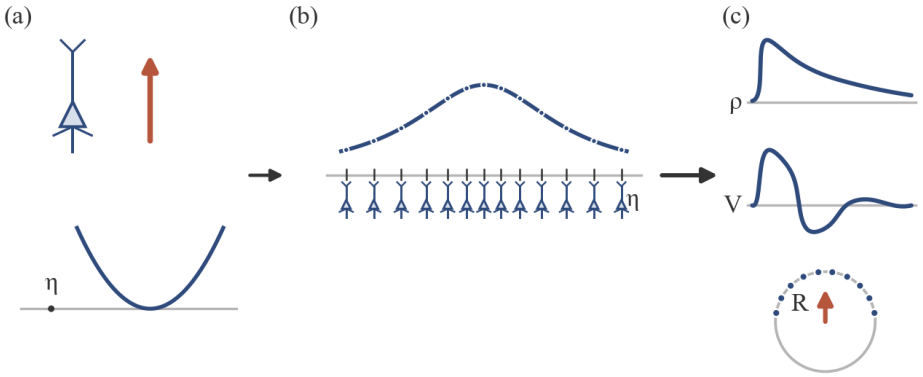


图 13　从单个神经元到群体的化简

它对无穷大网络精确成立，第 5.5.3 节用有限网络验证。

5.3.2　群体相位与头皮信号

把膜电位写成相位 $\theta_j = 2\arctan V_j$，群体相位的集中程度就是 Kuramoto 序参数 $R = \langle e^{\mathrm{i}\theta_j} \rangle$，它由 ρ 与 V 经共形映射直接给出：

$$Q = \pi T_m \rho + \mathrm{i} V, \qquad R = \frac{1 - \overline{Q}}{1 + \overline{Q}}. \tag{15}$$

$|R|$ 越接近 1，相位越集中。需要区分两种集中：静息时神经元都停在阈下的静息相位附近，$|R|$ 接近 1 但几乎不发放；受驱动时神经元先后越过阈值，相位被拉开，$|R|$ 下降，发放率升高。头皮电位来自突触电流，而突触电流由发放驱动，所以只有在神经元同时发放、偶极同向叠加时，电极上才有可观测的偏转，如图 14 所示：相位分散时各细胞的偶极相互抵消，相位集中时同样数目的细胞给出可观测的偏转。图 15 给出同一群神经元在两种同步程度下的相位分布与对应的宏观量。

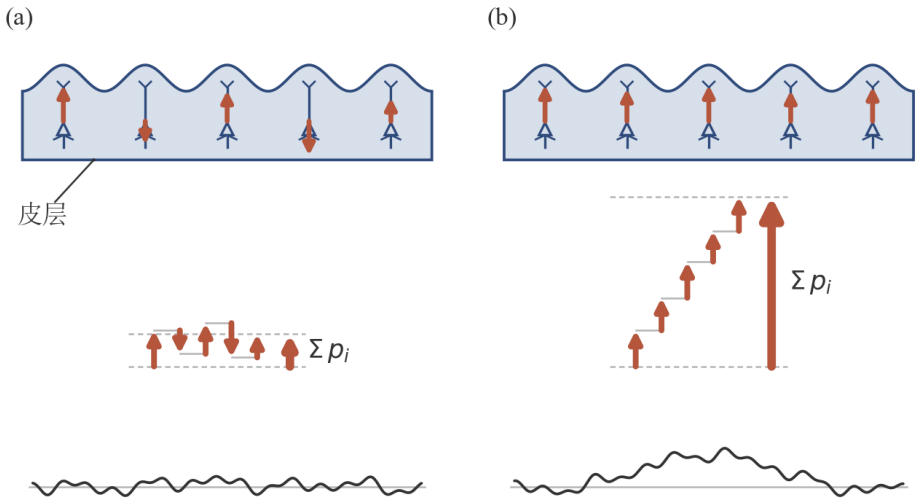


图 14　皮质偶极层的叠加

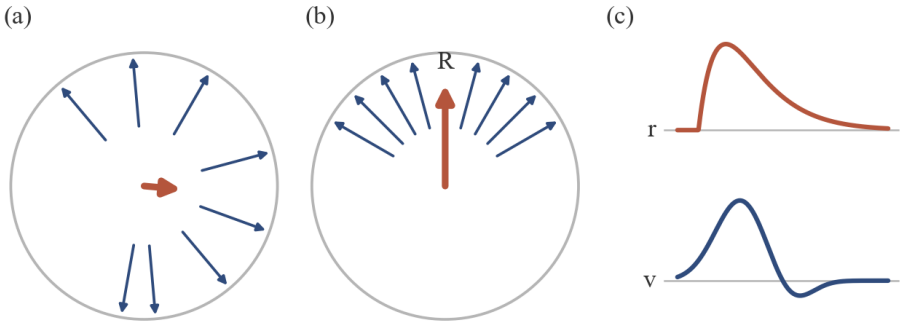


图 15　群体相位与宏观量

5.3.3　突触与 LGN 驱动

突触后电流由快兴奋与慢抑制两条 alpha 函数突触给出，两半球通过兴奋性连接 J 耦合：

$$I_{\mathrm{syn}} = G_e y_e - G_i y_i + J y'_e + u(t), \qquad \tau_x^2 \ddot{s}_x + 2\tau_x \dot{s}_x + s_x = \rho, \quad x = e, i, \tag{16}$$

y'_e 为对侧半球的兴奋。LGN 经视皮层送来的驱动，是提示方波先按时间尺度 T_d 平滑成一个短促的上升包络，再延迟 t_L 得到，两半球按驱动比例 P_L, P_R 分配：

$$u_k(t) = P_k \, F\left(\varepsilon_{T_d} * \mathbf{1}_{[0,\,0.2\,\mathrm{s}]}\right)(t - t_L), \qquad \varepsilon_\tau(t) = \frac{t}{\tau^2} e^{-t/\tau}, \quad k \in \{L, R\}. \tag{17}$$

5.4　头皮正演模型

头皮电位来自锥体细胞树突上的净突触电流。从 LGN 到电极的径向剖面如图 16 所示：视辐射纤维从 LGN 上行进入皮层，锥体细胞沿径向排列，同步的突触电流形成指向头皮的偶极，电场穿过脑脊液、颅骨与头皮到达电极。

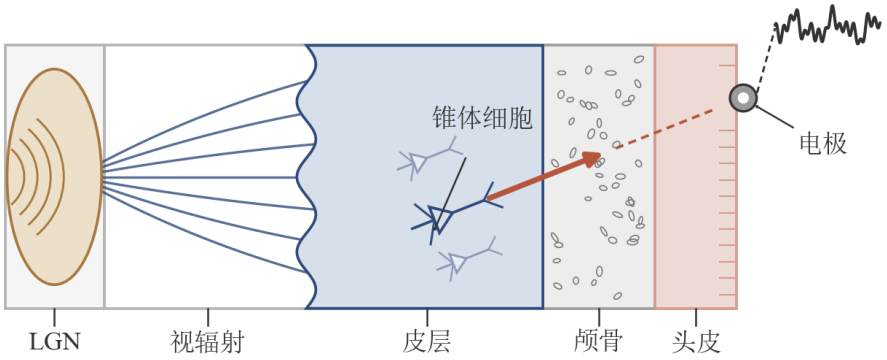


图 16　从 LGN 到头皮电极的径向剖面

两半球各放一个径向偶极，位于半径 7 cm 处的前额叶[11]，电极位于半径 9.2 cm 的球面上[12]，按 10-20 系统取 Fz、F3、F4 的方位。导联系数取无限均匀介质中的偶极场：

$$g_{ek} = \frac{\hat{\boldsymbol{n}}_k \cdot (\boldsymbol{r}_e - \boldsymbol{r}_k)}{4\pi\,|\boldsymbol{r}_e - \boldsymbol{r}_k|^3}, \tag{18}$$

球面只用来确定电极与源的位置。颅骨电导率低会把电场在头皮上摊开，这一效应没有计入；由于三个电极都在额区、彼此相距只有几厘米，它主要改变整体幅度，而整体幅度由下面的增益吸收。头皮电位是两个源的加权和，再经过与数据相同的滤波器 h：

$$U_e(t) = h * \Big[c \sum_{k\in\{L,R\}} g_{ek}\,(s_{e,k} - \gamma\, s_{i,k})(t) \Big]. \tag{19}$$

γ 为抑制电流的权重，c 为总增益。c 是纯线性系数，若作为参数会与驱动幅度 F 简并，所以每次求值时用闭式最小二乘解出，并限定 $c \ge 0$，偶极方向由锥体细胞的朝向定死，不允许靠翻转极性去凑波形。

综上，LGN—皮层—头皮三段式脑电计算模型为

$$\begin{cases} P_k = \displaystyle\sum_{(x,y)\in\Omega_k} \sum_{\theta} A_\theta(x,y), & \Omega_L \text{ 为右视野},\ \Omega_R \text{ 为左视野} \\ u_k(t) = P_k\, F\,(\varepsilon_{T_d} * \mathbf{1}_{[0,0.2\,\mathrm{s}]})(t - t_L) & \\ T_m \dot{r}_k = \dfrac{\sigma_\eta}{\pi T_m} + 2 r_k v_k & \\ T_m \dot{v}_k = v_k^2 + \eta_0 + T_m(G_e s_{e,k} - G_i s_{i,k} + K s_{e,\bar{k}} + u_k(t)) - (\pi T_m r_k)^2 & \\ \tau_x^2 \ddot{s}_{x,k} + 2\tau_x \dot{s}_{x,k} + s_{x,k} = r_k, & x = e, i \\ R_k = \dfrac{1-\overline{Q}_k}{1+\overline{Q}_k}, \quad Q_k = \pi T_m r_k + \mathrm{i} v_k & \\ U_e(t) = h * \Big[c \displaystyle\sum_{k} g_{ek}(s_{e,k} - \gamma s_{i,k}) \Big], & e \in \{\mathrm{Fz}, \mathrm{F3}, \mathrm{F4}\} \end{cases} \tag{20}$$

其中 $\bar{k}$ 为 k 的对侧。形状单元的响应 $\boldsymbol{\Phi}$ 由式 (10) 单独给出，作为形状表征，不进入前额叶的驱动。

5.5　模型的求解与检验

5.5.1　参数估计与多算法对比

拟合分两步。第一步拟合左右合并后的共模响应，此时两半球对称，只需积分一群，待估参数为驱动的 F, T_d, t_L，突触的 $T_e, T_i, G_e, G_i, \gamma$，以及 η_0, T_m，共 10 个，σ_η 固定为 1。第

二步用拟合好的参数加入两半球的驱动差，计算左右差。目标函数为三通道在 −0.3 到 0.65 s 内以标准误加权的残差平方和，提示前 0.3 s 也计入残差，模型若在提示前不处于静息态会被罚。

平均场用欧拉法积分，固定步长为 0.1 ms。积分从提示前 1.3 s 开始，前 1 s 用于趋近稳态，基线取提示前 0.3 s，避免初值的弛豫过渡被当成诱发响应。积分循环用 numba 编译，本次在相同步长下重新拟合，并通过有限脉冲网络对照检验源项。

目标函数有大片平台：参数落在神经元群不放电的区域时，模型输出为零，梯度也为零。为此比较六种做法，每种各跑 10 次，每次随机起点或随机种子不同，结果见表 5。命中指代价不超过全部运行中最好值的 1.01 倍。全局方法与多起点方法的调用预算在 2500 到 3000 次，单起点的局部方法在平台上会提前停止，实际调用少得多。

表 5　六种做法拟合项目一的对比

算法	最优代价	代价中位数	命中次数	单次用时/s	平均调用次数
单起点信赖域	313.4	974.7	0	1.2	289
多起点信赖域	274.4	275.2	6	23.5	3295
Nelder-Mead	279.3	974.7	0	4.0	1056
L-BFGS-B	316.6	974.7	0	2.2	501
差分进化	319.0	320.9	0	12.1	2481
双重退火	318.8	330.8	0	12.0	2501

同一目标函数的最好代价为 274.4。多起点 TRF 的最好代价为 274.4、中位数为 275.2，10 次中 6 次达到全体最好值的 1.01 倍以内，单次平均用时 23.5 s。不同优化器的函数调用数与终止条件也列在表中，比较反映本次计算预算下的表现，不能据此证明全局最优。

拟合代价不是选解的唯一依据。重跑的 8 起点拟合先找到代价 291.7 的候选解，但它没有通过稳定性检查。本文把原稿使用的检验落实到程序中：各参数随机扰动 5%，30 次源项形状相关的中位数至少为 0.90，且 4000 个神经元网络的源项相对误差不超过 10%，然后从通过检验的候选中取代价最低者。选出的代价为 315.5，独立复核与低代价候选的比较见表 6。

表 6　项目一的候选解

解	代价	诱发段 R^2	η_0	静息 $\lvert R\rvert$	扰动 5% 形状相关	脉冲网络误差
现用解	315.5	0.67，0.62，0.88	−9.89	0.97	0.98	2.0%
候选 1	274.9	0.69，0.66，0.90	1.79	0.73	0.80	95.2%
候选 2	275.0	0.69，0.67，0.90	−1.00	0.59	0.47	63.1%
候选 3	274.9	0.69，0.66，0.90	1.79	0.73	0.80	95.2%
候选 4	275.5	0.69，0.67，0.90	−0.65	0.62	0.34	17.3%

低代价候选的拟合优度与现用解接近，但参数扰动与有限网络检验暴露出差异。现用解的兴奋性中心为 $\eta_0 = -9.89$，30 次扰动的形状相关中位数为 0.982，4000 个神经元网络相对误差为 2.0%。表中的低代价解未同时通过上述两项条件，因此保留现用解，用于后续认知模型。该判据检验数值稳定性与有限网络可复现性，不等同于从头皮数据唯一识别生理参数。

求解步骤如下。Step1，总计取 8 个起点，包括一个默认起点、两个已知起点与五个范围内的随机起点。Step2，对每个起点用信赖域反射算法最小化加权残差，雅可比矩阵用有限差分。Step3，每次求值时先积分平均场，再按式 (19) 投到头皮并滤波。Step4，用闭式最小二乘解出增益 c，截断到非负。Step5，对代价最低的几组解检验参数扰动稳定性与脉冲

网络一致性，取满足两项检验且代价最低者。Step6，用选定的解在两半球分别积分，得到左右差。

5.5.2　拟合结果

项目一的拟合参数见表 7，模型与数据的对照以及群体动力学如图 17 所示。

表 7　项目一的拟合参数

参数	含义	取值	参数	含义	取值
F	驱动幅度	399.16	G_e	兴奋耦合	20.42
T_d	驱动时间常数	32.61 ms	G_i	抑制耦合	30.00
t_L	驱动延迟	136.90 ms	γ	抑制电流权重	2.048
T_e	兴奋突触时间常数	6.783 ms	η_0	兴奋性中心	-9.89
T_i	抑制突触时间常数	117.197 ms	T_m	膜时间常数	29.718 ms
c	增益	0.1039	σ_η	兴奋性半宽	1，固定

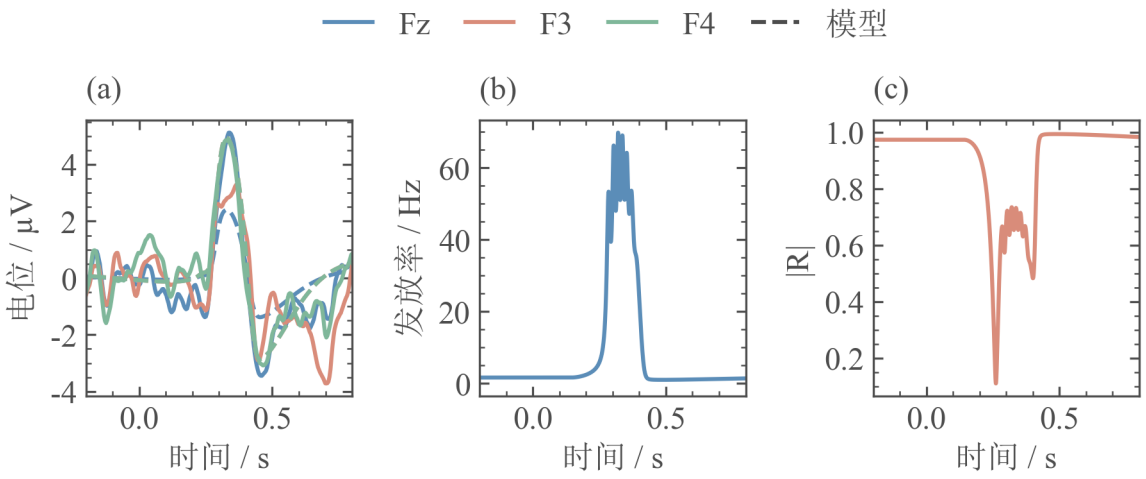


图 17　项目一的模型与数据

模型在诱发段 −0.1 到 0.65 s 对 Fz、F3、F4 的 R^2 分别为 0.673，0.623，0.883，模型峰潜伏期为 0.328 s。图 17(a) 对照平均响应与模型，(b) 给出发放率，(c) 给出序参数。提示前群体的发放率约为 1.66 Hz，$|R| = 0.973$；驱动约在 136.9 ms 后到达，发放率峰值为 69.8 Hz，序参数在 0.261 s 降到 0.110，随后在 0.485 s 回升到 0.993。头皮正峰对应兴奋电流的集中注入，随后慢抑制把群体拉回静息。图 18示意这一相位聚散过程。

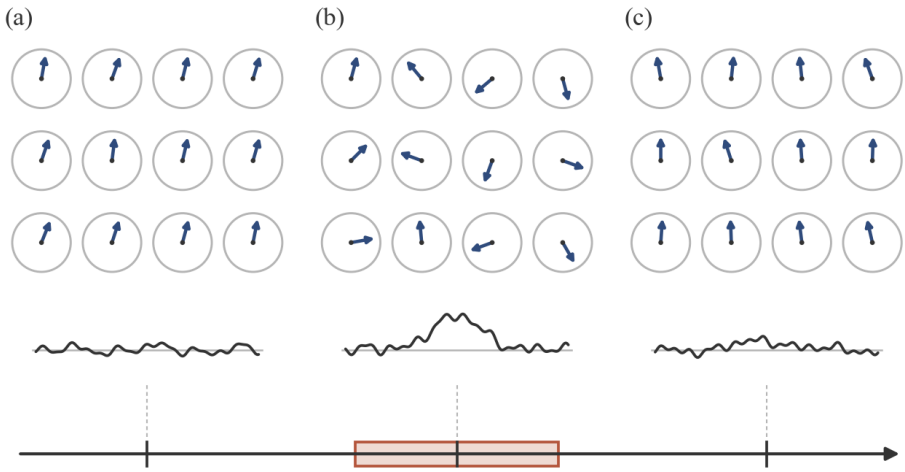


图 18　提示驱动下群体相位的变化

参数 F、G_i、η_0、T_m 接近或达到参数范围的边界。参数范围限制数值搜索域，不能凭一次局部拟合证明约束全局最优。表 6 的候选检验进一步说明，较低拟合代价不一定伴随稳定性与有限网络可复现性。

5.5.3　脉冲网络检验

精确平均场只在无穷大网络中严格成立。为检验它对有限网络的适用性，直接模拟 N 个满足式 (13) 的神经元，参数取表 7。V^2 项很陡，直接对 V 做欧拉积分会在发放后一步越过阈值，所以改积等价的相位 $\theta = 2\arctan V$，并按最快的神经元细分子步，洛伦兹分布的极端尾部截在 ± 200，约占 0.3%。结果见表 8，对照如图 19 所示。突触源项为投到头皮前的 $s_e - \gamma s_i$。16000 个神经元网络与平均场的源项相关为 0.9998，相对误差为 1.98%，峰值发放率为 66.8 Hz 与 69.8 Hz。不同网络规模的误差见表 8，并在图 19中对照。这一检验支持本组参数下用平均场近似有限网络；两边使用相同的头皮正演，不能据此检验导体模型。

表 8　脉冲网络与精确平均场的对照

神经元数 N	发放率相关	膜电位相关	$\lvert R\rvert$ 相关	突触源项相关	源项相对误差
1000	0.9960	0.9977	0.9984	0.9998	2.02%
4000	0.9974	0.9978	0.9986	0.9998	1.99%
16000	0.9976	0.9979	0.9986	0.9998	1.98%

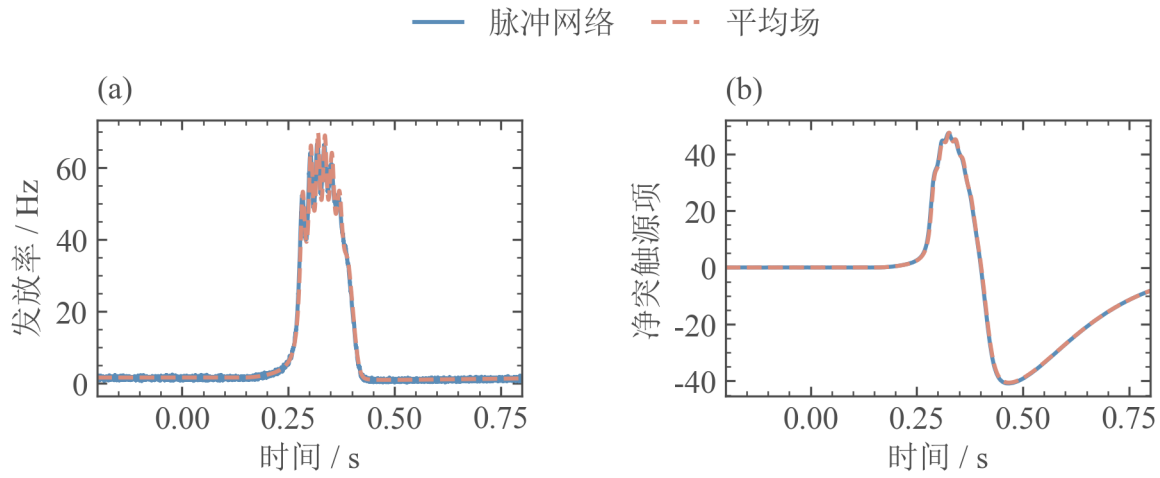


图 19　脉冲网络与精确平均场的波形对照

5.5.4　项目二的拟合

同一模型拟合项目二宽负波的诱发段 R^2 为 0.427，0.554，0.774，峰潜伏期为 0.414 s；参数扰动形状相关中位数为 0.487，4000 个神经元的源项相对误差为 83.6%。沿用项目一动力学并添加持续输入时，完整拟合窗内 R^2 为 0.422，0.052，0.521，峰潜伏期为 0.648 s。两种尝试均存在不足，不能视为已可靠解释项目二负波。第二个源参与是一种待检验的建模解释，第六章据此比较不同持续输入。

5.6　左右三角差异的形成机制与特征表示

5.6.1　形成机制

由模型可以把左右三角形的差异追溯到每一段。第一，左右三角互为镜像，竖直底边分别落在注视点的左侧或右侧，边缘能量更多地进入一侧视野。第二，交叉投射使两半球的驱动相差 15%，右指三角驱动右半球更强，左指三角驱动左半球更强。第三，两群前额叶神经元收到不同强度的驱动，发放与相位起落的幅度不同。第四，两个偶极到 F3 与 F4 的导联系数左右对称，两群的差在 F3 与 F4 上留下符号相反的偏转，左右三角互换时符号翻转。形状单元的 2.9 倍差异说明两种三角在颞下皮层被分得很开，但在本模型中它不参与头皮信号的形成。四步依次如图 20 所示。

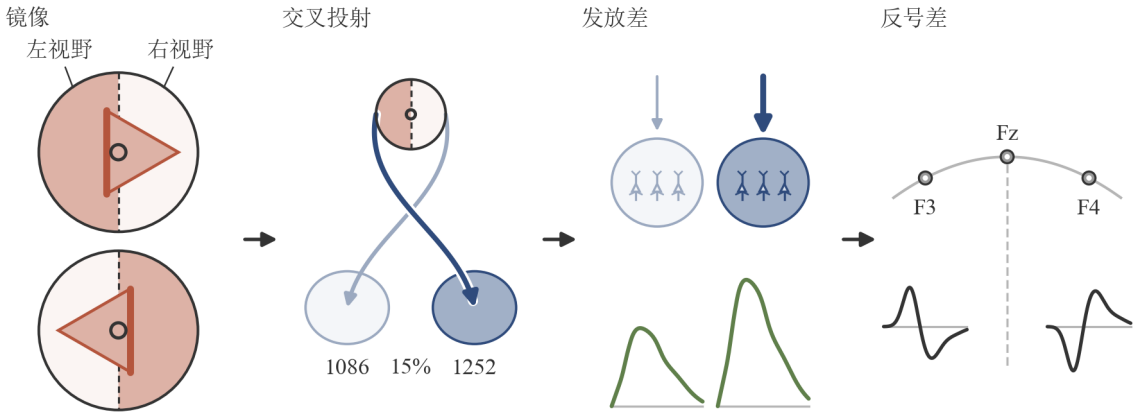


图 20　左右三角差异的形成链

把两半球驱动差代入式 (20)，300 到 500 ms 内 F3 减 F4 的左右差在 $J = 0$、0.5、1 时分别为 0.577，0.624，0.640 μV。该预测依赖半球耦合，本文以 $J = 0.5$ 的 0.624 μV 为代表值，同时报告三个设定的范围。

5.6.2 特征表示

能从头皮脑电上区分左右三角形的特征是 300 到 500 ms 内 F3 与 F4 之差的均值

$$\Lambda = \frac{1}{|\mathcal{T}|}\sum_{t\in\mathcal{T}}\left(V_{\mathrm{F3}}(t) - V_{\mathrm{F4}}(t)\right), \qquad \mathcal{T} = [0.3,\ 0.5]\ \mathrm{s}, \tag{21}$$

模型预言左右三角互换时偏侧化符号翻转；$J = 0.5$ 时左右差为 0.624 μV。这一特征源于两半球的驱动差。形状单元的响应向量 (S_R, S_L) 与额区偏侧化是不同层次的表征，额区三个电极无法直接测量形状单元本身。

5.6.3 可检出性与单试次解码

Λ 能否在本数据上测出，取决于单试次的噪声与试次数。单试次上 Λ 的标准差为 σ_Λ，每种三角各做 n_1, n_2 次，双侧检验水平 0.05、把握度 0.8 时能检出的最小左右差为

$$d_{\min} = (z_{0.975} + z_{0.8})\,\sigma_\Lambda\sqrt{\frac{1}{n_1}+\frac{1}{n_2}} = 2.80\,\sigma_\Lambda\sqrt{\frac{1}{n_1}+\frac{1}{n_2}}. \tag{22}$$

结果见表 9，两个项目的实测左右差与检出限比较如图 21 所示。

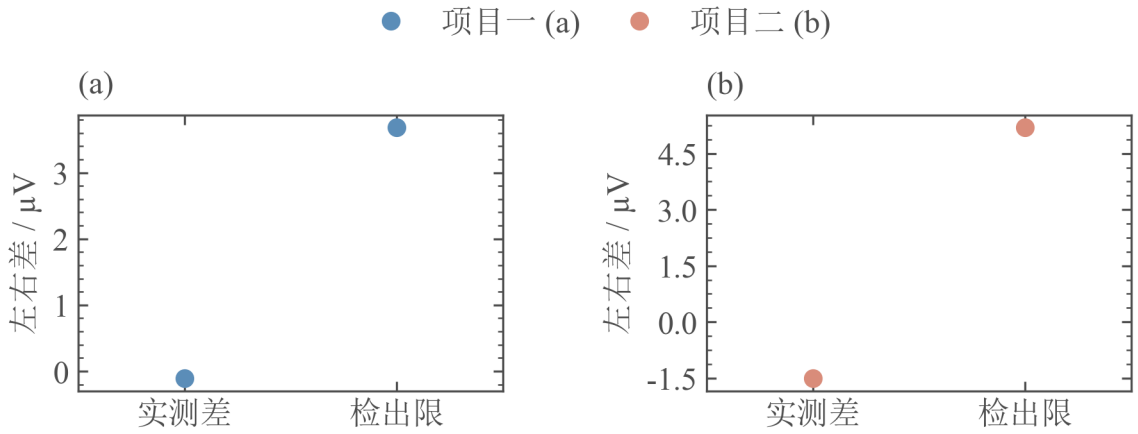


图 21　两个项目的实测左右差与检出限比较

表 9　头皮层左右差的可检出性

量	项目一	项目二
左、右指试次数	83、85	87、84
单试次 Λ 的标准差 σ_Λ	8.51 μV	12.13 μV
能测出的最小左右差 $d_{\min}$	3.68 μV	5.19 μV
模型预测的左右差	0.577 到 0.640 μV	0.38 到 2.52 μV
实测的左右差	−0.11 μV	−1.51 μV

$J = 0.5$ 时模型左右差为 0.624 μV，低于 3.68 μV 的检出限；实测差为 -0.11 μV。按相同噪声水平和式 (22) 反推，每侧约需 2917 次试验才能检出这一代表值。该估计只针对当前模型、耦合与噪声假设。项目二参数的稳定性不足，预测只作参考。

单试次上再做一次检验。从 0 到 0.8 s 的段中取四类特征：三通道降采样后的时域波形、F3 减 F4 的偏侧化波形、四个频带的对数能量及其左右差、三通道协方差。每类特征配四种分类器，分层五折交叉验证，换 10 个划分种子取 AUC 的平均，结果见表 10。SVM 的 AUC 用决策函数值计算，不用 Platt 校准后的概率，后者在小样本下会把排序整体翻转。

表 10　四类特征配四种分类器的单试次左右解码 AUC

特征	项目	逻辑回归	收缩 LDA	RBF-SVM	随机森林
时域波形	一	0.50	0.50	0.52	0.50
时域波形	二	0.45	0.44	0.49	0.46
偏侧化波形	一	0.39	0.40	0.37	0.38
偏侧化波形	二	0.42	0.43	0.38	0.46
频带能量	一	0.43	0.43	0.48	0.44
频带能量	二	0.47	0.47	0.46	0.46
通道协方差	一	0.50	0.52	0.51	0.48
通道协方差	二	0.55	0.56	0.57	0.57
单次交叉验证用时/s		0.02 到 0.03	0.02 到 0.03	0.02 到 0.02	4.60 到 5.03

32 个组合的 AUC 在 0.37 到 0.57 之间。偏侧化波形配 SVM 的 500 次标签置换检验，项目一和项目二的双侧 p 分别为 0.0439 与 0.0419，单侧检验都没有显示高于随机的区分能力。考虑 32 个组合的多重比较，本数据未提供可靠的单试次区分证据。

模型的偏侧化代表值为 0.624 μV，低于 3.68 μV 的检出限，单试次解码也未显示稳定的区分能力。改善实验需要增加有效试次数、降低伪影噪声或加入更接近视觉皮层的电极。

5.7　本章小结

本章建立 LGN—皮层—头皮三段式计算模型。项目一诱发段 R^2 为 0.673，0.623，0.883，峰潜伏期 0.328 s；参数稳定性与有限网络检验参与选解，16000 个神经元的源项误差为 1.98%。模型给出 300 到 500 ms 的 F3 减 F4 均值作为偏侧化特征，$J = 0.5$ 时左右差为 0.624 μV，低于本数据 3.68 μV 的检出限。项目二宽负波的单群体拟合仍有稳定性问题。

六、　问题三模型的建立、分析与求解

6.1　问题分析

问题三要求截取应答前的认知相关信号，结合问题二的机制，同时考虑海马与 LGN 两个来源，建立宏观认知模型并用数据验证。

题目把从刺激到动作分成看见视物、对照识别与应答三段，并说明只考虑前两段，应答只用来定反应时。本文照此办：模型覆盖前两段，应答时刻只作为截取的终点。

首先，要确定截到哪里以及每次试验的应答是否有效。第 4.2 节已查明目标显示段在点击时结束，由此得到两个项目逐次试验的应答时刻与反应时。截下来的信号从提示起算，到应答前 0.1 s 打住，再分成目标出现前的维持期与目标出现后的决策期。

然后，两个项目的认知负担不同。项目一记住方向即可，项目二还要把三角形的形状记在脑子里，目标出现后与左右两侧的候选逐一比对。记住形状属于工作记忆，Daume 等人在人海马中记录到维持期的 theta 与 gamma 相幅耦合，并且它控制着工作记忆的保持[14]；额区 theta 随记忆负荷升高[15]，alpha 则反映丘脑皮层回路对感觉输入的门控[16]。据此让问题二的前额叶群同时收到两路输入，一路来自丘脑的 alpha 节律，一路来自海马，后者是 theta 相位门控下的 gamma 爆发，行为层用**带失误的证据累加模型**[17]，合成**双源输入宏观认知模型**。

最后，用数据检验两件事：行为上项目二变慢的原因是决策变慢还是多了一段比对，脑电上两项目的差异来自哪一路输入。前者用两个联合模型的 BIC 比较，后者用六个模型比

较，其中包括不含 theta 的纯慢输入与只变背景两个对照，再用跨被试预测与频谱峰位置做样本外检验。

问题三的求解路线如图 22 所示，上一行是信号截取与行为层，中间一行是脑电层的数据与模型，末行是检验。

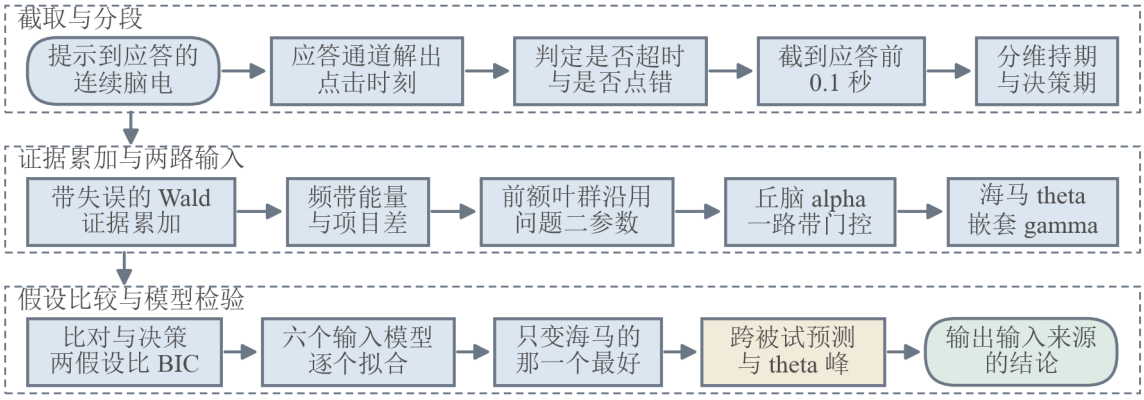


图 22　问题三思路框架图

6.2　信号截取与应答编码

四份数据共 400 次试验，每次都有目标显示段，而且都以点击结束：项目二每次都有点击标记，项目一每段 ±1 的长度在 0.82 到 1.94 s 之间逐次变化，没有停在某个固定时长上，说明也是被点击结束的。全部试次中最长的显示段为 1.97 s，找不到未及时应答的试次。题目未给出应答时限，本文把显示段以点击结束作为及时应答的判据。

对错只能在项目一中判定。项目一的目标就在提示那一侧，±1 的符号与提示方向不同即为点错，200 次中只有 B1 有 2 次，反应时为 1.26 s 与 1.55 s，都慢于该组的中位数 1.11 s。项目二的 ±1 与 ±2 符号逐次相同，数据只给出点击侧，没有给出与记忆形状相同的那个目标在哪一侧，所以项目二的对错无法判定，下文按全部应答处理，并把这一点作为限定。

截取的信号从提示开始，到应答前 0.1 s 为止，即题目所说终点在应答前约 100 ms。其中维持期定为提示后 0.6 至 2.0 s，此时提示诱发的响应已经结束，目标尚未出现，脑电主要反映记忆的保持；决策期取目标出现到应答前 0.1 s，包含比对与决策。项目一只用判为正确的试次，含饱和坏段的窗剔除。

6.3　认知过程的机理分析

应答之前，前额叶上同时汇合着两路信号，一次项目二试验中两路输入随时间的变化如图 23 所示。一路来自 LGN 所在的丘脑，丘脑皮层回路产生的 alpha 节律在整次试验中都在；另一路来自海马，它保持着提示给出的信息，以 theta 为载波、在波峰上骑着 gamma 爆发的方式送往前额叶[14]，这一路在提示后变强，维持期最强，目标出现后减弱。两路在前额叶群上叠加，头皮电位在维持期出现更多的低频与 theta 成分。图中下方的阶段条带把一次试验分成提示、等待、维持、应答四段，浅蓝底的维持段是本文的分析窗，截取的信号从提示开始，到应答前 0.1 s 为止。

两个项目在记忆内容上不同，如图 24 所示。项目一只需按方向在两侧中挑一侧，记住的是一个位置；项目二要把记忆里的形状叠到两个候选上，合得上的那个才是要点的，记住的是一个形状模板。这意味着项目二在维持期需要更强的记忆通路参与，在决策期需要多一步模板比对。

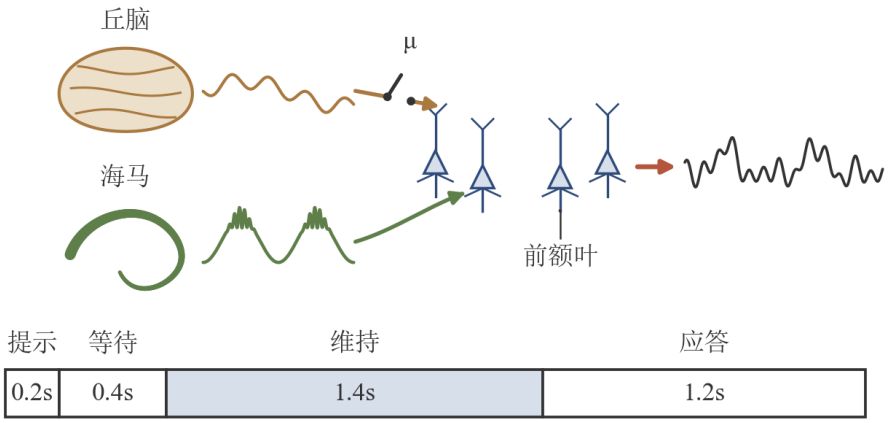


图 23　一次项目二试验的两路输入与头皮电位时程

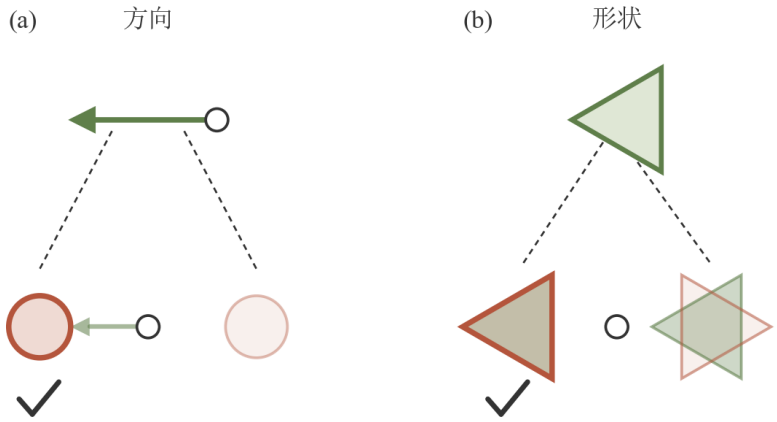


图 24　记忆模板与候选目标的比对

6.4　双源输入宏观认知模型

6.4.1　行为层：带失误的证据累加模型

目标出现后，证据 $X(t)$ 以漂移率 k、单位噪声累加，到达边界 θ_b 时应答，首达时间服从 Wald 分布。加上非决策时间 t_d，即感觉编码与动作执行所需的时间，反应时的密度为

$$f(t)=\frac{\theta_b}{\sqrt{2\pi(t-t_d)^3}}\exp\!\left(-\frac{(\theta_b-k(t-t_d))^2}{2(t-t_d)}\right),\qquad t>t_d. \tag{23}$$

题目提醒应答可能错误。扩散过程若在起点居中的上下两个边界之间进行，错误率为 $1/(1+e^{2k\theta_b})$；此外被试还可能因注意力失误随机点一侧，失误率为 ϵ。两者合起来

$$P_{\text{err}}=\frac{\epsilon}{2}+(1-\epsilon)\,\frac{1}{1+e^{2k\theta_b}}. \tag{24}$$

证据累加、到达边界与失误的过程如图 25 所示：轨迹从非决策时间之后起步，漂到任一边界即应答，失误表现为跳到另一侧。

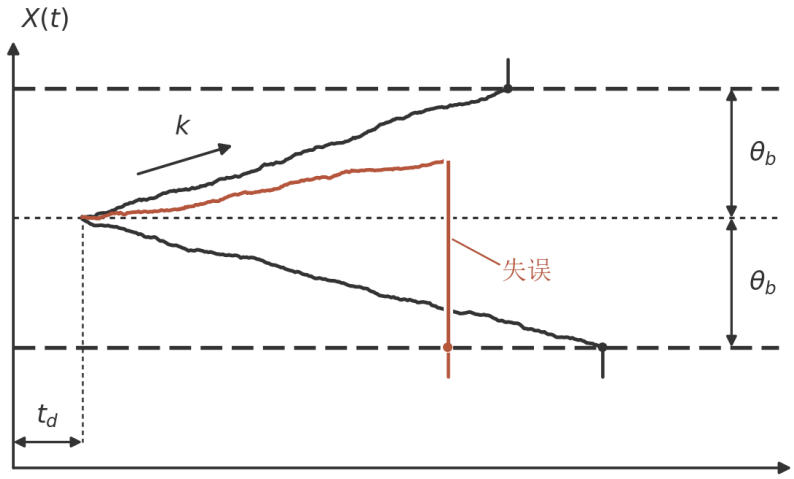


图 25　证据累加与应答

项目二比项目一慢，可能有两种原因。其一是决策变慢，两项目共用非决策时间 t_d，漂移率与边界各自取值，共 5 个参数；其二是多了一段固定时长的模板比对，两项目共用 (k, θ_b)，非决策时间各自取值，共 4 个参数，差值 $T_c = t_{0,2} - t_{0,1}$ 即比对时长。两者用对数似然与 BIC 比较。

6.4.2　脑电层：双源输入的前额叶群

前额叶群沿用问题二项目一的全部内在参数，即式 (14) 与式 (16) 及表 7，只改变输入。输入由三路组成：

$$u(t) = \sigma_0\, \zeta_B(t) + c_T\, \zeta_\alpha(t) + c_H\, h(t). \tag{25}$$

ζ_B 是时间常数 5 ms 的 Ornstein–Uhlenbeck 噪声，代表来自其它皮层的宽带背景，Steeghs-Turchina 等人表明头皮上的慢波可由长程连接的简单统计模型解释[18]；ζ_α 是白噪声经 10 Hz、品质因数 2 的谐振器得到的随机振子，代表丘脑皮层回路的 alpha 节律；$h(t)$ 为海马输入：

$$\begin{aligned} h(t) &= m(t)\Big[(1-\eta)\, g(t)\big(1 + \tfrac{1}{2} g(t)\cos 2\pi\omega_\gamma t\big) + \eta\Big], \\ g(t) &= \frac{1+\cos\phi(t)}{2}, \qquad \dot{\phi} = 2\pi\omega_\theta + \sqrt{2D_\phi}\,\xi(t). \end{aligned} \tag{26}$$

theta 相位 ϕ 带相位扩散，$D_\phi = 2$；gamma 频率 $\omega_\gamma = 90$ Hz，只在 theta 波峰附近出现；$m(t)$ 是时间常数 0.3 s 的慢包络；$\eta = 0.3$ 是不随 theta 起伏的慢调制所占份额。三路输入都归一到单位标准差，幅度由 σ_0, c_T, c_H 决定。

theta 频率 ω_θ 不参与拟合，取行为层给出的比对时长的倒数，即假定项目二多出的那段比对恰好是海马的一个 theta 周期。这把行为与脑电绑在同一个模型里，并留下一个可检验的预言：两个项目相减的频谱在 $1/T_c$ 附近应出现一个 theta 峰。

两个项目的差异只允许落在输入上。主模型让海马耦合随项目变化，$c_H \to c_H + \Delta c_H$；比较的对象有五个：都不变；只变丘脑门控，$c_T \to \mu c_T$；两者都变；纯慢输入，即海马一路取 $\eta = 1$，只留慢包络、去掉 theta 与 gamma；只变背景，$\sigma_0 \to \gamma\sigma_0$。

综上，双源输入宏观认知模型为

$$
\begin{cases}
\mathrm{RT} = t_d + T_c\,\mathbb{1}_{\{\text{项目二}\}} + \tau_a, \quad \tau_a = \inf\{t:\ X(t) \ge \theta_b\}, \quad dX = k\,dt + dW \\
P_{\mathrm{err}} = \dfrac{\epsilon}{2} + \dfrac{1-\epsilon}{1+e^{2k\theta_b}} \\
T_m \dot{r} = \dfrac{\Delta}{\pi T_m} + 2rv, \qquad T_m \dot{v} = v^2 + \eta_0 + T_m(G_e y_e - G_i y_i + u(t)) - (\pi T_m r)^2 \\
\tau_x^2 \ddot{s}_x + 2\tau_x \dot{s}_x + s_x = r, \qquad x = e, i \\
u(t) = \sigma_0\,\zeta_B(t) + c_T\,\zeta_a(t) + (c_H + \Delta c_H\,\mathbb{1}_{\{\text{项目二}\}})h(t), \quad h(t) \text{ 由式 (26) 给出}, \quad \omega_\theta = 1/T_c \\
V(t) \propto h * (y_e - \gamma y_i)
\end{cases}
\tag{27}
$$

脑电层的观测取三个电极的平均。拟合目标为 8 个数：项目一维持期四个频带的相对能量，以及项目二减项目一四个频带的对数绝对能量差。两者都与输出增益无关，所以 $V(t)$ 只写成正比关系；不拟合绝对幅度，是因为单试次的背景脑电大半来自这群神经元以外的皮层，问题二按平均响应标定的增益管不到它。频带能量由希尔伯特包络在连续信号上计算后按窗取均值，模型输出与数据用同一套滤波与包络算法。

6.5 模型的求解与检验

6.5.1 行为层结果

四份数据的反应时与 Wald 拟合见表 11，分布与拟合曲线如图 26 所示。

表 11　反应时与证据累加模型的拟合

量	A1	A2	B1	B2
反应时中位数/s	1.143	1.393	1.109	1.322
反应时标准差/s	0.170	0.149	0.184	0.154
点错次数	0	无法判定	2	无法判定
漂移率 k	4.16	5.55	3.81	5.19
边界 θ_b	1.86	3.73	1.64	3.35
非决策时间 t_d/s	0.712	0.728	0.690	0.684
分位点最大偏差	0.046	0.021	0.039	0.029

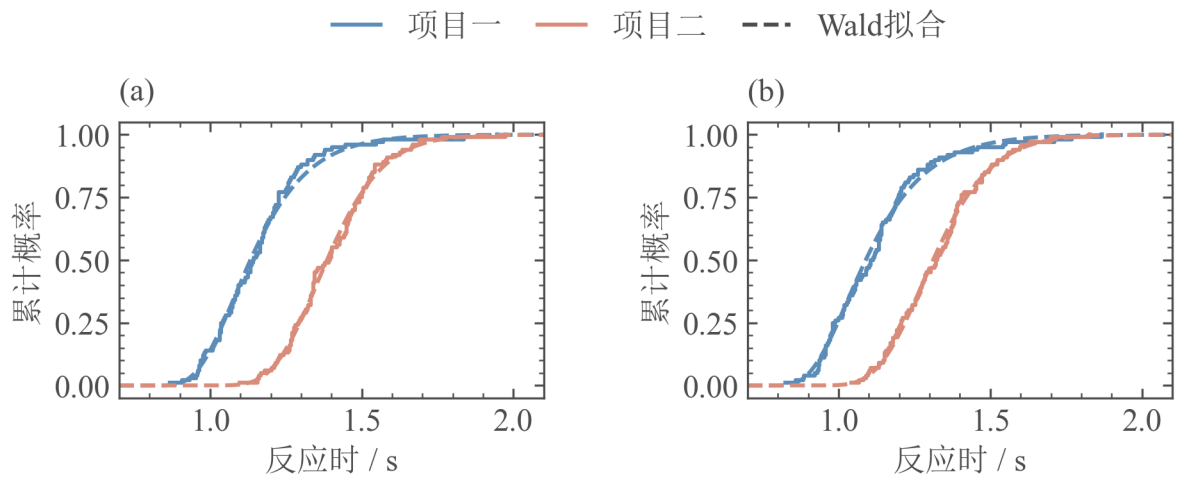


图 26　反应时分布与 Wald 拟合

Wald 分布在 10%、30%、50%、70%、90% 五个分位点上与经验分布的最大偏差不超过 0.046，拟合良好。项目二比项目一慢 0.21 到 0.25 s，标准差却小 0.02 到 0.03 s。

两种解释的联合拟合见表 12。

表 12　项目二变慢的两种解释

模型	参数数	被试 A		被试 B	
		负对数似然	BIC	负对数似然	BIC
两项目全共享	3	-43.78	-71.67	-47.40	-78.90
决策变慢	5	-102.76	-179.02	-91.68	-156.88
多一段比对	4	-101.97	-182.74	-90.75	-160.30

两项目全共享的拟合较差。多一段比对的模型比决策变慢模型少一个参数，BIC 在 A、B 上分别低 3.72 与 3.42，因此得到中等程度的相对支持。两被试平均比对时长为 0.225 s，取倒数得到 4.444 Hz。该解释仍不能完整说明两项目反应时标准差的差别。

B1 的 2 次错误，按表 11 的参数代入式 (24) 的扩散项，$1/(1+e^{2k\theta_b}) = 3.7\times10^{-6}$，100 次里出现 2 次的概率只有 6.6×10^{-8}，双侧二项检验的 $p = 1.3\times10^{-7}$，扩散过程解释不了。两次错误都慢于中位数，更像注意力失误。在可判定对错的项目一 200 次中，错误率为 1%，95% 精确区间为 0.12% 到 3.6%，由式 (24) 得失误率 $\epsilon \approx 2\%$。

6.5.2　脑电的项目差异

维持期与决策期的频带能量见表 13，频谱如图 27 所示。

表 13　两项目频带能量的变化

频带	维持期			决策期	
	绝对能量倍数	相对能量差	效应量 t_L	相对能量差	效应量 t_L
delta	3.85	+0.130	+0.94	+0.056	+0.38
theta	2.84	+0.038	+0.45	+0.017	+0.20
alpha	1.18	−0.123	−1.31	−0.068	−0.63
beta	1.08	−0.045	−0.96	−0.005	−0.11

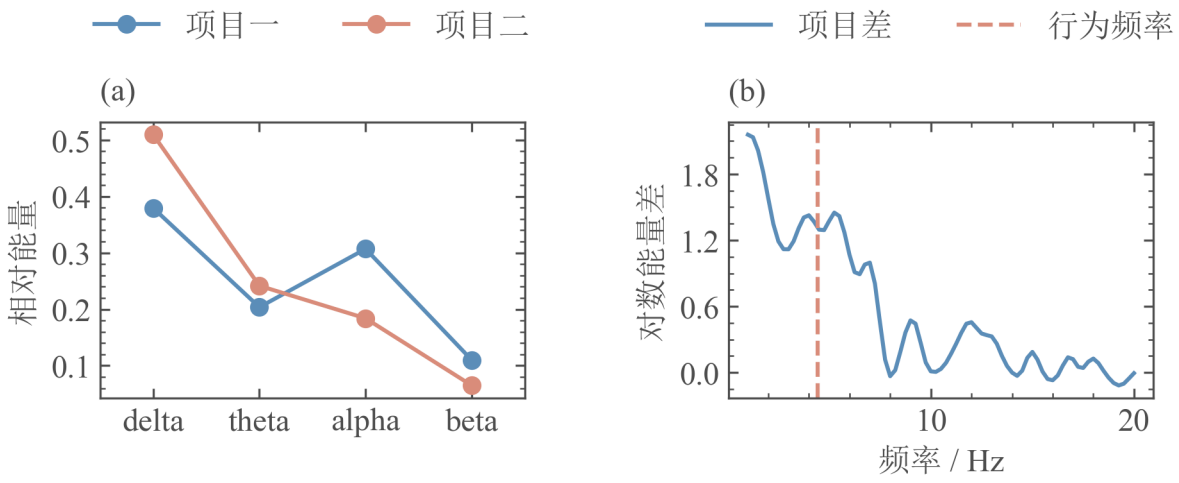


图 27　两项目维持期频带能量与项目差频谱

项目二维持期的 delta 与 theta 绝对能量分别升到项目一的 3.85 倍与 2.84 倍，alpha 与 beta 只有 1.18 倍与 1.08 倍，两被试方向一致。相对能量中 alpha 与 beta 的下降是被新增的低频能量挤小的，不是它们本身减弱。决策期方向相同，幅度约减半，beta 不再显著，说明维持期是项目差异最集中的阶段。

6.5.3 眼动的核查

项目二眨眼更多，被试也可能来回扫视，额区的低频升高需要核查眼动与眨眼残留。结果见表 14，偏侧信号取 F3 减 F4，对水平眼动敏感；共模信号取三者平均，对垂直眼动与眨眼敏感。

表 14　眼动核查的对数能量差

量	A 偏侧	A 共模	B 偏侧	B 共模
delta 对数能量差	0.90	1.17	0.80	1.53
theta 对数能量差	0.77	0.97	0.61	1.12
窗内线性趋势解释的方差，项目一	0.040	0.064	0.078	0.083
窗内线性趋势解释的方差，项目二	0.087	0.064	0.068	0.119

共模信号的升高大于偏侧信号，这与垂直眼动或眨眼残留的分布一致，不能用来排除它们；偏侧信号的 delta 也升高了 0.8 到 0.9，水平眼动同样不能排除。进一步把眨眼峰前后 0.3 s 内有落进维持期的窗口整窗剔除后复算：项目二几乎每次试验在维持期都有眨眼，94 与 96 个窗口中只剩 5 与 7 个，项目一剩 34 与 21 个。在这些不含眨眼的窗口上，delta 的对数差仍为 1.15，theta 降为 0.52，alpha 与 beta 为 −0.08 与 −0.19。delta 的升高在剔除眨眼后基本保留，theta 的升高减半，而且项目二只剩 12 个窗口。所以维持期低频升高中有一部分可能来自眨眼与眼动残留，本文不能把它全部归于脑源，下面的模型比较在这一限定下进行。

6.5.4 输入来源的比较

六个模型在输入端的差别如图 28 所示：三路输入里哪几路被改动、改成什么，是六个模型唯一的不同。六个模型的拟合见表 15，预测与实测的对照如图 29 所示。

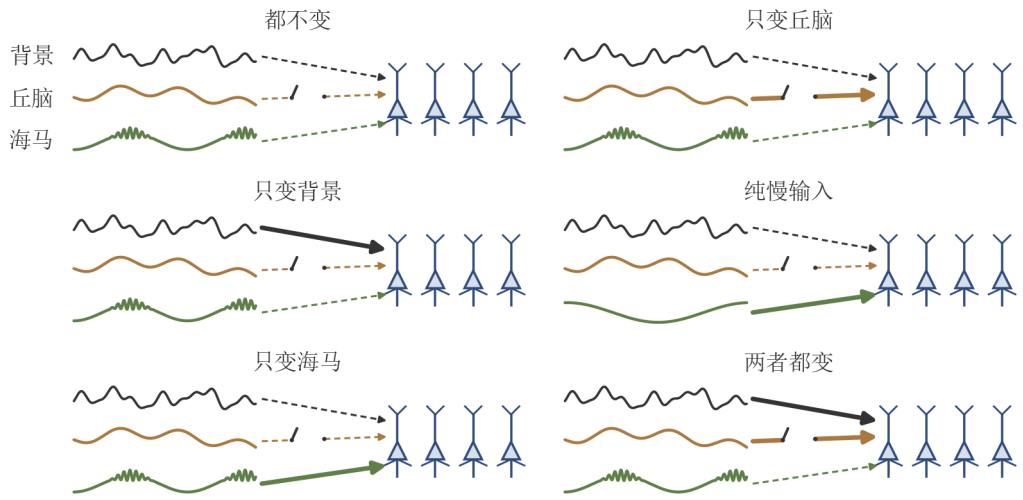


图 28　六个输入模型的结构

每次求值积分两段各 84.5 s 的连续信号，用两组固定的随机种子，使目标函数对参数是确定的。

表 15　六个输入模型的比较

模型	参数数	χ^2	BIC	对数差 $\delta,\theta,\alpha,\beta$	关键参数
都不变	3	453.6	459.9	0.00，0.00，0.00，0.00	—
只变丘脑	4	411.9	420.2	0.02，0.15，0.28，0.22	$\mu = 1.197$
只变背景	4	276.8	285.1	0.53，0.52，0.25，0.44	$\gamma = 1.512$
纯慢输入	4	139.8	148.2	1.58，0.68，0.15，0.18	$\Delta c_H = 2.414$
只变海马	4	115.1	123.4	1.29，1.07，0.11，0.15	$\Delta c_H = 2.303$
两者都变	5	114.9	125.3	1.30，1.07，0.10，0.14	$\mu = 0.984$
实测	—	—	—	1.35，1.04，0.17，0.08	—

注：主拟合 theta 频率由行为层推得，为 4.444 Hz；灵敏度表另固定为 4.44 Hz 或 5.5 Hz，并重新拟合自由参数。

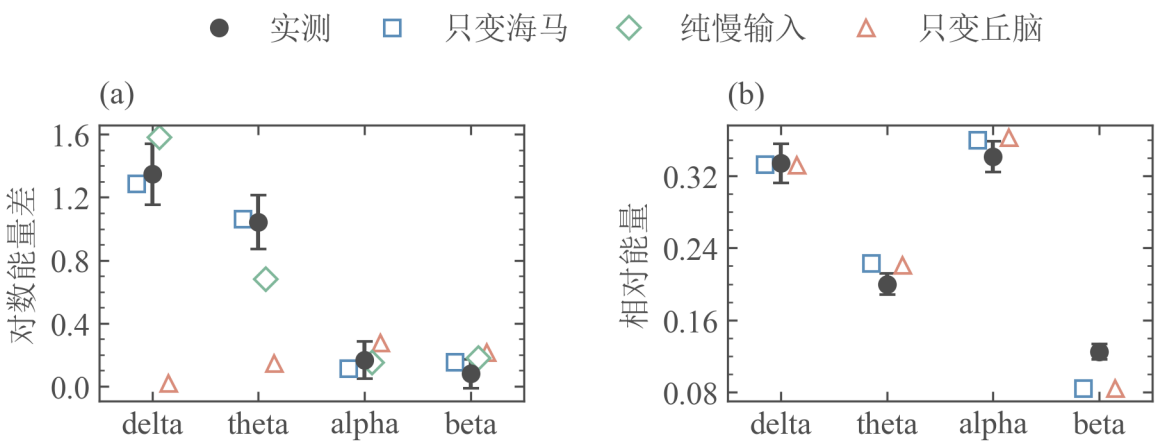


图 29　输入模型的预测与实测（误差棒为均值标准误的 1.96 倍）

六个输入模型中，只变海马的 BIC 最低。只变海马的模型 $\chi^2 = 115.1$，BIC 为 123.4；只变丘脑、只变背景与纯慢输入的 χ^2 分别为 411.9、276.8 与 139.8。只变海马模型的耦合从 0.100 升到 2.403。纯慢输入对 theta 项目差的预测为 0.68，实测为 1.04，说明当前模型下增加带 theta 节律的持续输入比只有慢包络更能解释数据。

这一比较有两个限定。第一，theta 节律不能证明来自海马，额叶皮层也产生随记忆负荷改变的 theta，所以结论是需要一路带 theta 节律的持续输入，与海马参与相容。第二，8 个目标与 4 个自由参数对应的约化 χ^2 为 28.8，绝对拟合仍不充分。项目一 beta 相对能量预测为 0.084、实测为 0.125。海马相幅耦合是模型输入的设定，不作为独立检验依据。

只变海马模型的 4 个参数用三种方法求解，结果见表 16。命中仅指 χ^2 不超过主拟合 115.1 的 1.01 倍，是达到当前参考值的次数，不表示已证明全局最优。

表 16　只变海马模型的三种求解方法对比

方法	运行次数	各次 χ^2	命中次数	平均用时/s
多起点信赖域	3	115.1，115.1，115.1	3	40.0
Nelder-Mead	3	306.1，115.1，115.1	2	34.5
差分进化	2	181.3，158.3	0	34.8

三种方法使用相同模型与目标函数。多起点信赖域和 Nelder-Mead 的各次结果及命中次数列在表中，差分进化在当前迭代预算下较难到达主拟合参考值。本文保留多起点信赖域，并报告随机起点间差异与实际用时。

6.5.5　模型检验

用三处没有参与拟合的数据检验模型，结果见表 17。

表 17　双源模型的样本外检验

检验	模型	实测
A 拟合预测 B 的对数差	1.15，0.94，0.10，0.14	1.52，1.11，0.20，0.14
项目差 theta 峰	4.444 Hz	5.50 Hz，区间 5.25，7.00 Hz
两被试分开的 theta 峰	4.444 Hz	5.50 与 5.25 Hz
B1 的错误数	失误成分约 2 次	2 次

第一，用 A 拟合后预测 B 的四个对数差，方向与实测一致，但 8 个目标中有 3 项的标准化偏差绝对值超过 1.96，不能认为跨被试数值预测已经准确。第二，对数频率去倾斜后的局部峰为 5.50 Hz，95% 自助区间为 5.25 到 7.00 Hz；行为推出的 4.444 Hz 未落入该区间，因此没有得到该频谱指标的支持。第三，失误成分用于解释单纯扩散模型难以解释的 2 次错误。

还有两项预言的结果为零。应答前 0.6 到 0.1 s 的电位按反应时快慢分组后，额区三个电极上都没有稳定的累加斜坡；各频带能量与单试次反应时的秩相关绝对值都不超过 0.12，$p > 0.08$。前者说明本数据未在额区检出稳定的证据累加斜坡，O’Connell 等人报告的累加信号位于中央顶区[19]；后者与模型一致，因为模型中海马耦合决定要不要比对，不决定单次比对的快慢。

6.6　本章小结

本章建立双源输入宏观认知模型。信号从提示截到应答前 0.1 s；项目一 B1 有 2 次错误，项目二对错无法由现有通道判定。平均比对时长为 0.225 s。维持期 delta、theta 绝对能量为项目一的 3.85 倍与 2.84 倍，眼电残留仍不能排除。只变海马模型 $\chi^2 = 115.1$，在当前模型集合中支持增加带 theta 节律的持续输入；跨被试数值预测与行为频率绑定仍有不足。

七、　各问结果汇总

按题目的问法，三问的答案汇总于表 18。

表 18　各问答案汇总

题目要求	答案
问题一 保留特征的降噪	分源模板去噪，幅度保留 0.86，误差 1.22 μV，相关 0.902
问题一 各电极有效响应	项目一：Fz +5.12 μV，0.340 s，F3 +3.28 μV，0.363 s，F4 +4.85 μV，0.324 s；项目二：Fz -11.01 μV，0.406 s，F3 -9.26 μV，0.480 s，F4 -16.69 μV，0.426 s
问题一 拟合曲线	滤波一致高斯和，式 (6) 与式 (7)，按有效样本数 BIC 选成分数
问题二 LGN 到头皮的机理	精确平均场与径向偶极正演，诱发段 R^2 为 0.673，0.623，0.883，16000 神经元源项误差 1.98%

续表

题目要求	答案
问题二 左右差与区分特征	300 到 500 ms 的 F3 减 F4 均值；$J = 0.5$ 时模型左右差 0.624 μV，检出限 3.68 μV，单试次 AUC 为 0.37 到 0.57
问题三 信号截取与应答	提示到应答前 0.1 s，目标显示段终点为应答时刻；项目一 B1 有 2 次错误，项目二对错无法判定
问题三 宏观认知模型	平均比对时长 0.225 s；只变海马模型 $\chi^2 = 115.1$，纯慢输入 139.8，只变丘脑 411.9
问题三 数据验证	跨被试有 3 项标准化偏差超过 1.96；频谱峰 5.50 Hz，行为推出的 4.444 Hz 未得到支持，眼电残留不能排除

八、 模型的灵敏度分析

8.1 高通截止频率

问题一的高通截止频率在 0.3 到 3 Hz 之间变化时的结果已在表 2 给出，三条指标随截止频率的走向如图 30 所示。

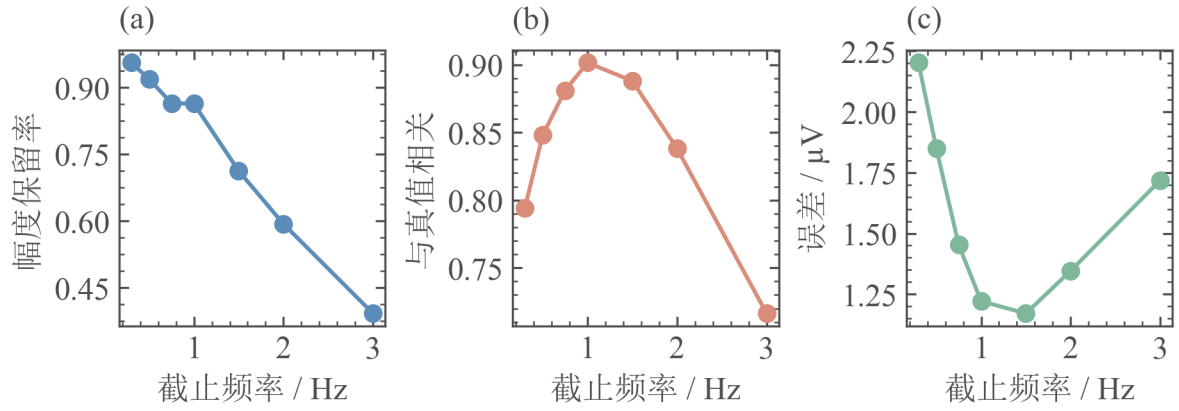


图 30 截止频率对注入检验三项指标的影响

截止频率在 0.75 到 1.5 Hz 之间，相关保持在 0.88 到 0.90，误差在 1.17 到 1.45 μV，结果稳定；截止频率不高于 0.5 Hz 时漂移残留使误差达到 1.85 μV 以上，达到 2 Hz 时幅度保留率降到 0.59 以下。1 Hz 处在稳定区间的中部，去噪模型对这一参数不敏感。

8.2 平均场参数

把项目一拟合出的 10 个参数逐一上调或下调 10%，其余不变，增益固定，看三通道 R^2 的平均值与诱发源项形状的变化，结果见表 19 与图 31。基准的平均 R^2 为 0.726。

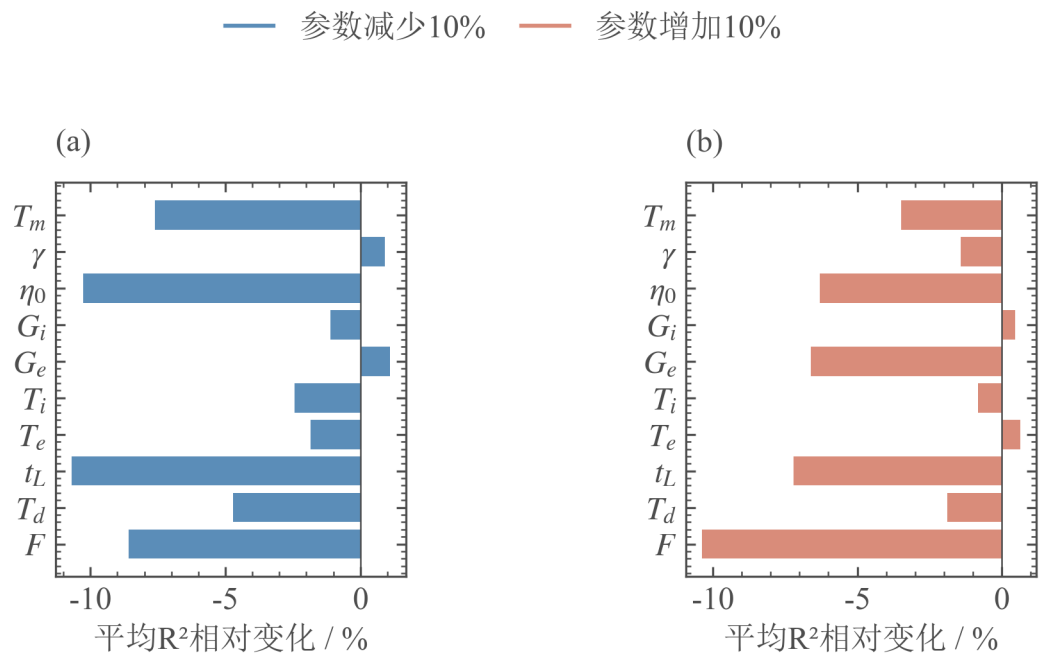


图 31　平均场参数变化 10% 时平均 R^2 的相对变化

表 19　平均场参数变化 10% 时的拟合优度与源项形状

参数	×0.9	×1.1	形状相关	参数	×0.9	×1.1	形状相关
F	0.664	0.651	0.958	G_e	0.734	0.678	0.998
T_d	0.692	0.713	0.989	G_i	0.718	0.730	0.999
t_L	0.649	0.674	0.972	η_0	0.652	0.681	0.967
T_e	0.713	0.731	0.998	γ	0.733	0.716	0.998
T_i	0.709	0.720	0.991	T_m	0.671	0.701	0.972

各参数逐一改变 10% 后的平均拟合优度与源项形状相关见表 19。全部参数同时随机扰动 5%，重复 40 次，形状相关中位数为 0.981，最小为 0.753；项目二独立检查的中位数为 0.487。这些检验针对本组参数，不能推出所有参数或被试都具有相同稳定性。

8.3　海马输入的形状

海马输入的形状由三个量决定：theta 频率 ω_θ、不随 theta 起伏的慢调制份额 η、背景噪声的时间常数 T_B。各取几档后重拟合只变海马的模型，结果见表 20 与图 32。

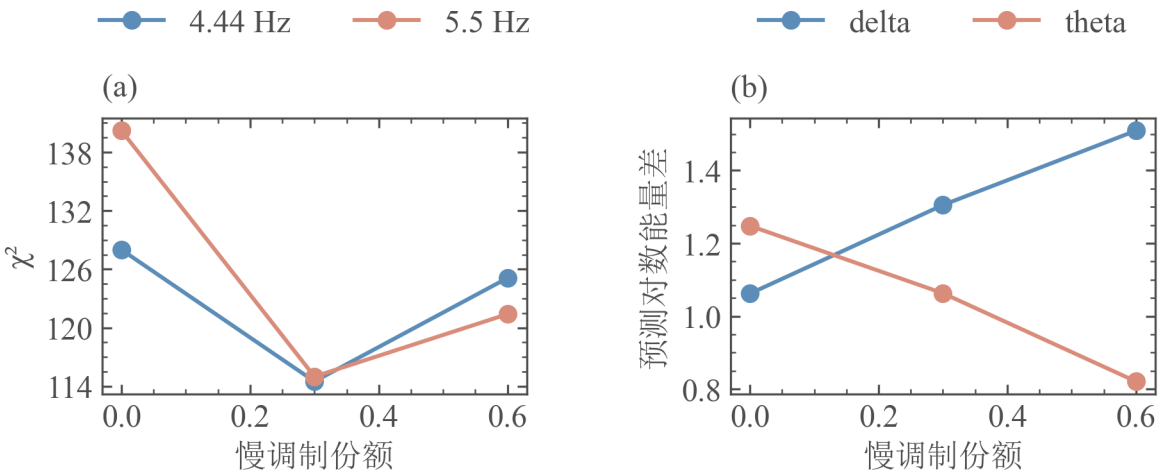


图 32　海马输入形状对双源模型的影响

每一行固定 theta 频率、慢调制份额与背景时间常数，再拟合其余四个参数。灵敏度网格的 4.44 Hz 是固定值，主拟合使用行为推出的 4.444 Hz，起点也有所不同，所以不要求两处的 χ^2 完全相同。

表 20　海马输入形状对双源模型的影响

ω_θ/Hz	η	T_B/ms	χ^2	预测对数差 $\delta,\theta,\alpha,\beta$	Δc_H
4.44	0	5	128.0	1.06，1.25，0.10，0.10	2.25
4.44	0.3	5	114.5	1.30，1.06，0.12，0.14	2.35
4.44	0.6	5	125.1	1.51，0.82，0.14，0.17	2.42
5.5	0	5	140.2	0.92，1.30，0.12，0.09	2.21
5.5	0.3	5	115.0	1.24，1.11，0.13，0.13	2.31
5.5	0.6	5	121.4	1.49，0.86，0.14，0.16	2.39
4.44	0.3	20	201.9	1.29，1.07，0.11，0.14	1.92
实测			—	1.35，1.04，0.17，0.08	—

慢调制份额控制 delta 与 theta 的能量分配，不同固定设定下重拟合的 χ^2 范围为 114.5 到 201.9，海马耦合增量范围为 1.92 到 2.42。该比较说明输入形状影响拟合优度与耦合幅度；频带汇总量不能替代对 theta 峰频率的独立检验。

九、　模型的评价

9.1　模型的优点

一是去噪按伪影时间结构分源去除，并用注入响应与 R 波残余定量检验。二是计算模型把视觉编码、神经元群与头皮正演连接起来，选解落实了稳定性与有限网络检验，16000 神经元的源项误差为 1.98%。三是把模型预测左右差 0.624 μV 与检出限 3.68 μV 比较，解释当前数据的区分能力限制。四是认知模型比较六类输入，并如实报告样本外检验未得到支持的部分。

9.2　模型的缺点

一是只有额区三个电极且没有眼电通道，源定位与眼电残留份额不能由数据确定，导体正演采用均匀介质近似。二是项目二的单群体解与共享动力学拟合仍有不足，第二个源尚未在诱发响应上得到可靠识别。三是项目二对错无法判定，失误率只能参考项目一。四

是认知模型约化 χ^2 为 28.8，跨被试有 3 项标准化偏差超过 1.96，行为频率绑定也未得到支持。五是仅有两名被试，不能据此验证疾病诊断效能。

9.3 模型的改进

若能增加顶枕区电极，视皮层的偏侧化成分会比额区大得多，左右三角的头皮层特征有望在单试次上测出；若有眼电通道，可以把眼动作为回归量直接去除。项目二的负波可以用两群神经元建模，一群接收感觉驱动，一群接收海马输入，两者的偶极方向不同，叠加后形成宽负波。theta 频率可以作为自由参数由频谱估计，再检验比对时长是否为整数个周期。

9.4 模型的推广

分源模板去噪与注入检验的框架可用于少通道生理记录，三段式模型可在更换感觉编码后用于其他诱发电位，可检出性分析可辅助脑机接口实验设计。双源认知模型可以作为后续患者与健康人数据比较的候选框架，但本次只有两名被试的任务数据，无法识别海马来源或验证疾病诊断效能。项目间低频能量增加不等同于疾病相关的信号变慢变弱，诊断用途仍须独立患者数据与外部验证。

参考文献

[1] Polich J，Updating P300: an integrative theory of P3a and P3b，Clinical Neurophysiology，118(10)：2128–2148，2007。

[2] Donoho D L, Johnstone I M，Ideal spatial adaptation by wavelet shrinkage，Biometrika，81(3)：425–455，1994。

[3] Branch M A, Coleman T F, Li Y，A subspace, interior, and conjugate gradient method for large-scale bound-constrained minimization problems，SIAM Journal on Scientific Computing，21(1)：1–23，1999。

[4] Jung T P, Makeig S, Humphries C, et al.，Removing electroencephalographic artifacts by blind source separation，Psychophysiology，37(2)：163–178，2000。

[5] Hyvärinen A, Oja E，Independent component analysis: algorithms and applications，Neural Networks，13(4-5)：411–430，2000。

[6] Azzopardi G, Petkov N，Ventral-stream-like shape representation: from pixel intensity values to trainable object-selective COSFIRE models，Frontiers in Computational Neuroscience，8：80，2014。

[7] Glomb K, Cabral J, Cattani A, et al.，Computational models in electroencephalography，Brain Topography，35(1)：142–161，2022。

[8] Montbrió E, Pazó D, Roxin A，Macroscopic description for networks of spiking neurons，Physical Review X，5(2)：021028，2015。

[9] Acebrón J A, Bonilla L L, Pérez Vicente C J, et al.，The Kuramoto model: a simple paradigm for synchronization phenomena，Reviews of Modern Physics，77(1)：137–185，2005。

[10] Byrne Á, Brookes M J, Coombes S，A mean field model for movement induced changes in the beta rhythm，Journal of Computational Neuroscience，43(2)：143–158，2017。

[11] Papargiri A, Kalantonis V S, Kaziki D, et al.，Revisiting an analytical solution for the three-shell spherical human head model in electroencephalography，Partial Differential Equations in Applied Mathematics，4：100178，2021。

[12] Dayarian N, Khadem A，A hybrid boundary element-finite element approach for solving the EEG forward problem in brain modeling，Frontiers in Systems Neuroscience，18：1327674，2024。

[13] Storn R, Price K，Differential evolution: a simple and efficient heuristic for global optimization over continuous spaces，Journal of Global Optimization，11(4)：341–359，1997。

[14] Daume J, Kamiński J, Schjetnan A G P, et al.，Control of working memory by phase-amplitude coupling of human hippocampal neurons，Nature，629(8011)：393–401，2024。

[15] Jensen O, Tesche C D，Frontal theta activity in humans increases with memory load in a working memory task，European Journal of Neuroscience，15(8)：1395–1399，2002。

[16] Klimesch W，EEG alpha and theta oscillations reflect cognitive and memory performance: a review and analysis，Brain Research Reviews，29(2-3)：169–195，1999。

[17] Ratcliff R, McKoon G，The diffusion decision model: theory and data for two-choice decision tasks，Neural Computation，20(4)：873–922，2008。

[18] Steeghs-Turchina M, Srinivasan R, Nunez P L, Nunez M D，Slow wave dynamics of scalp EEG can be explained by simple statistical models of long-range connections，NeuroImage，321：121418，2025。

[19] O'Connell R G, Dockree P M, Kelly S P，A supramodal accumulation-to-bound signal that determines perceptual decisions in humans，Nature Neuroscience，15(12)：1729–1735，2012。

AI 工具使用声明

本次代码重跑、选解规则修正、结果图生成及论文结果更新使用了 OpenAI 的 Codex 人工智能工具。计算在 AutoDL 服务器上执行，结果由程序输出，参数筛选、对照模型与检验的数值记录随支撑材料保存。人工智能辅助参与了代码修改和文字整理，参赛队需对模型假设、数据解释、引用与最终提交内容负责，并按比赛要求核验声明信息。

附录　支撑材料与程序清单

程序随支撑材料提交，包含 code/ 目录下的 26 个求解脚本与远程重跑、结果更新辅助程序，每个文件开头按规定注明了 AI 辅助信息。本次在 AutoDL 隔离环境使用 Python 3.12，numpy 1.26.4、scipy 1.14.1、numba 0.61.2、PyWavelets 1.8.0、scikit-learn 1.5.2，CPU 即可运行。完整依赖版本与运行日志保存在 rerun_20260926_autodl/ 中。按问题一、二、三的顺序运行，脚本会把中间结果写进 out/ 目录，后面的脚本再读它们，因此单个脚本不能单独跑。程序与功能见表 21，各程序输出的结果文件见表 22，完整的源程序见附录二。

表 21　程序清单

文件	功能
common.py	数据读取、提示与应答事件切分、截段
preprocess.py	分源模板去噪：饱和修补、高通、心电与眨眼模板相减、小波截断、低通、去趋势、剔除与截尾均值
q1_run.py、q1_extra.py	各组数据与两被试合并的平均响应，眨眼分组核查
q1_validate.py	注入检验与高通截止扫描
q1_compare.py	七种去噪方法的对比

续表

文件	功能
q1_refit.py、q1_revise.py	滤波一致的高斯和拟合，按有效样本数的 BIC 选成分数；心电模板与小波截断的作用检验
q2_visual.py	高斯差分、Gabor 朝向能量、顶点单元、COSFIRE 形状单元与两半球驱动
q2_dynamics.py	精确平均场、alpha 突触、头皮正演、多起点拟合与左右差预测
q2_compare.py、q2_refind.py、q2_candidates.py	六种做法的对比，低代价候选解的复现与检验
q2_spiking.py、q2_task2_check.py	脉冲网络与精确平均场的对照，两项目解的稳定性与网络检验
q2_shared.py	项目二共用项目一动力学的拟合尝试
q2_decode.py、q2_decode_compare.py、q2_svm_perm.py	检出限、四类特征配四种分类器的单试次解码与置换检验
q3_events.py、q3_behavior.py	应答通道解码、反应时、对错、Wald 拟合与两种假设的比较
q3_eeg.py、q3_ocular.py	维持期与决策期频带能量、应答锁时电位、脑与行为相关、眼动核查
q3_model.py、q3_revise.py	双源输入宏观认知模型、六个输入模型比较、剔除眨眼复算、优化器对比、跨被试预测与频谱峰检验
q_sens.py	平均场参数与海马输入形状的灵敏度分析
autodl_rerun.py、rerun_pipeline.py	AutoDL 上传与回收、按依赖关系执行全部 26 项计算
fig_results_rerun.py、update_paper_results.py	由新结果生成数值图、摘要与正文表格
srv.py	把本目录传到服务器并在那里运行，跑完把结果拉回本机；单机运行时用不到

表 22　结果文件清单

文件	内容
q1_erps.json、q1_summary.json、q1_pooled.json	各电极的平均响应与标准误，两被试合并后的结果
q1_extra.json、q1_revise.json	眨眼分组核查；按有效样本数选出的拟合参数，心电残余与小波截断的作用
q1_fit_relaxed.json	放宽幅度与宽度上界后的重拟合
q1_validate_all.json、q1_validate_final.json、q1_validate_sweep.json	注入检验、复算与高通截止扫描
q1_compare.json	七种去噪方法的注入检验对比
q2_visual.json	形状单元响应与两半球驱动

续表

文件	内容
q2_fit_Task1.json、 q2_fit_Task2.json、 q2_check.json	平均场拟合参数、诱发段拟合优度、稳定性与网络检验、左右差预测
q2_fit_Task1_params.npy、 q2_fit_Task2_params.npy	两个项目拟合出的动力学参数，供复算直接读取
q2_compare_Task1.json、 q2_candidates.json、 q2_refind.json	六种做法的对比、候选解检验与低代价解的复现
q2_spiking_Task1.json、 q2_spiking_Task2.json	两个项目的脉冲网络检验
q2_shared_Task2.json	项目二共用项目一动力学的拟合尝试
q2_decode.json、 q2_decode_compare.json、 q2_svm_perm.json	检出限、单试次解码与置换检验
q3_events.json、 q3_behavior.json	应答时刻与超时判定；带失误 Wald 模型的拟合与两种解释的比较
q3_eeg.json、q3_ocular.json	维持期与决策期频带能量、眼动核查
q3_model.json、q3_revise.json	双源模型的拟合、对照模型、剔除眨眼复算、优化器对比与样本外检验
q_sens.json	平均场参数与海马输入形状的灵敏度分析

附录二　程序源代码

下面按运行顺序列出支撑材料里 26 个求解脚本的完整源程序，都在 code/ 目录下。每个文件开头的注释即为按竞赛规定注明的 AI 辅助信息。

common.py

```
# 本程序及代码是在人工智能工具辅助下完成的：DeepSeek，模型 DeepSeek-V4，
# 开发机构 杭州深度求索人工智能基础技术研究有限公司，版本发布日期 2026 年 4 月 24 日。
"""数据读取与试次切分的公共函数。"""
import numpy as np, scipy.io as sio, os

ROOT = os.environ.get('YJS_C_ROOT', '/data1/liutianrui/yjs2026C')
if not os.path.isdir(ROOT):                 # 本机跑时落回项目目录
    ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
DATA = os.path.join(ROOT, 'data')
if not os.path.isdir(DATA):                 # 本机数据就用汇总目录里的赛题附件
    DATA = os.path.join(os.path.dirname(os.path.dirname(ROOT)), '赛题', 'C题')
PAPER_FIGS = os.path.join(os.path.dirname(ROOT), '初稿', '论文', 'figs')
SUMMARY_FIGS = os.path.join(os.path.dirname(ROOT), '初稿', 'figs_paper')
FIGS = os.path.join(ROOT, 'figs')
OUT = os.path.join(ROOT, 'out')
FS = 256
FILES = {'A1': 'VisualCogA_Task-1.mat', 'A2': 'VisualCogA_Task-2.mat',
         'B1': 'VisualCogB_Task-1.mat', 'B2': 'VisualCogB_Task-2.mat'}
CH = ['Fz', 'F3', 'F4']


def load(key):
    d = sio.loadmat(os.path.join(DATA, FILES[key]))['data']
    return d


def onsets(v):
```

```
    """非零段的起点索引与该段的值。"""
    nz = np.where(v != 0)[0]
    if len(nz) == 0: return np.array([], int), np.array([])
    on = nz[np.r_[True, np.diff(nz) > 1]]
    return on, v[on]


def trials(d):
    """返回每个试次的字典：cue 起点，方向(-1/+1)，目标显示起点，点击标记(±2，项目二才有)，是否正确."""
    cue_on, cue_v = onsets(d[7])
    act = d[8]
    act_on, act_v = onsets(act)
    two = np.where(np.abs(act) == 2)[0]
    out = []
    for i, c in enumerate(cue_on):
        nxt = act_on[act_on > c]
        t_on = int(nxt[0]) if len(nxt) else None
        click = None; correct = None
        if t_on is not None:
            seg_end = cue_on[i + 1] if i + 1 < len(cue_on) else len(act)
            tw = two[(two >= t_on) & (two < seg_end)]
            if len(tw):
                click = int(tw[0]); correct = bool(np.sign(act[click]) == np.sign(cue_v[i]))
            else:
                # 项目一：通道9 记的是目标区间且符号即所点方向
                correct = bool(np.sign(act[t_on]) == np.sign(cue_v[i]))
        out.append(dict(cue=int(c), dir=int(cue_v[i]), tgt=t_on, click=click, correct=correct))
    return out


def epochs(x, idx, pre, post):
    """x: (n,) 信号；idx: 事件采样点；取 [idx-pre, idx+post) 段，越界丢弃."""
    segs, keep = [], []
    for k, i in enumerate(idx):
        if i - pre >= 0 and i + post <= len(x):
            segs.append(x[i - pre:i + post]); keep.append(k)
    return np.array(segs), np.array(keep)
```

preprocess.py

```
# 本程序及代码是在人工智能工具辅助下完成的：DeepSeek，模型 DeepSeek-V4，
# 开发机构 杭州深度求索人工智能基础技术研究有限公司，版本发布日期 2026 年 4 月 24 日。
"""问题一：保留视觉响应特征的去噪流水线。

链条：饱和段修补 -> 高通去漂移(截止按注入实验的增益/误差权衡选) -> R 峰模板去心电
      -> 共模事件锁时模板去眨眼与眼动 -> 小波高频细节层幅度截断去肌电与动作尖峰 -> 20 Hz 低通去电极噪声
      -> 逐试次二次多项式去趋势 -> 稳健剔除 -> 修剪均值叠加
设计原则：视觉响应只有几微伏而伪影是几十到几百微伏，所有非线性步骤都只动幅度超过阈值的成分，
响应本身的系数永远在阈值以下不会被碰；心电和眨眼用锁时模板减法，只减去与事件锁时的部分。
"""
import numpy as np, pywt
from scipy.signal import butter, find_peaks, sosfiltfilt
from common import FS, epochs, CH


def fix_saturation(x, lim=999.9, pad=int(0.05 * FS)):
    bad = np.abs(x) >= lim
    if bad.any():
        idx = np.where(bad)[0]
        for i in idx[np.r_[True, np.diff(idx) > 1]]: bad[max(0, i - pad):i] = True
        for i in idx[np.r_[np.diff(idx) > 1, True]]: bad[i:i + pad] = True
        good = ~bad; x = x.copy(); x[bad] = np.interp(np.where(bad)[0], np.where(good)[0], x[good])
    return x, bad


def highpass(x, fc=1.0, order=2):
```

```
    sos = butter(order, fc / (FS / 2), 'high', output='sos'); return sosfiltfilt(sos, x)


def lowpass(x, fc=20, order=4):
    sos = butter(order, fc / (FS / 2), 'low', output='sos'); return sosfiltfilt(sos, x)


def r_peaks(ecg):
    h = np.percentile(np.abs(ecg), 99) * 0.5
    pk, _ = find_peaks(np.abs(ecg), height=h, distance=int(0.35 * FS))
    return pk


def _template_subtract(x, pk, pre_n, post_n, iters=2, shift_n=0, gmax=3.0, gmin=-1.0, sign_align=
    False):
    x = x.copy(); tmpl = None; base_n = int(0.1 * FS)
    for _ in range(iters):
        ep, keep = epochs(x, pk, pre_n, post_n)
        if len(ep) < 5: return x, None
        if sign_align:
            sg = np.sign(ep[:, pre_n]); sg[sg == 0] = 1; ep = ep * sg[:, None]
        ep = ep - ep[:, :base_n].mean(1, keepdims=True)
        tmpl = np.median(ep, 0); tmpl -= tmpl[:base_n].mean()
        w = np.ones_like(tmpl); r = int(0.06 * FS); w[:r] = np.linspace(0, 1, r); w[-r:] = np.
    linspace(1, 0, r); tmpl *= w
        den = tmpl @ tmpl + 1e-9
        for k in keep:
            i = pk[k]; best = (0.0, 0)
            for s in range(-shift_n, shift_n + 1, 2):
                a, b = i - pre_n + s, i + post_n + s
                if a < 0 or b > len(x): continue
                seg = x[a:b]; g = (seg - seg[:base_n].mean()) @ tmpl / den
                if abs(g) > abs(best[0]): best = (g, s)
            g, s = best; g = float(np.clip(g, gmin, gmax))
            if g != 0: a, b = i - pre_n + s, i + post_n + s; x[a:b] -= g * tmpl
    return x, tmpl


def remove_ecg(x, pk):
    return _template_subtract(x, pk, int(0.25 * FS), int(0.55 * FS), iters=2, shift_n=0, gmax=3.0,
    gmin=0.0)


def detect_blinks(chs, thr=35.0, lo=0.5, hi=8.0):
    """三通道共模信号(中位数)上找眨眼与眼动事件。"""
    sos = butter(3, [lo / (FS / 2), hi / (FS / 2)], 'band', output='sos')
    cm = np.median(np.stack([sosfiltfilt(sos, c) for c in chs]), 0)
    s = 1.4826 * np.median(np.abs(cm - np.median(cm)))
    T = max(thr, 5 * s)
    pk, _ = find_peaks(np.abs(cm), height=T, distance=int(0.25 * FS), width=(int(0.04 * FS), int
    (1.0 * FS)))
    return pk, cm


def remove_blinks(x, pk):
    return _template_subtract(x, pk, int(0.4 * FS), int(0.6 * FS), iters=2, shift_n=int(0.06 * FS),
     gmax=6.0, gmin=-6.0, sign_align=True)


def wavelet_clip(x, wavelet='sym8', level=6, k=5.0, levels_touch=(1, 2, 3)):
    """只在高频细节层(1=64-128 Hz, 2=32-64, 3=16-32)做幅度截断，去肌电与动作尖峰；低频层不动。"""
    coeffs = pywt.wavedec(x, wavelet, level=level); out = [coeffs[0]]
    for j, c in enumerate(coeffs[1:], start=1):
        lvl = level - j + 1
        if lvl not in levels_touch: out.append(c); continue
        s = 1.4826 * np.median(np.abs(c)) + 1e-9; T = k * s; out.append(np.clip(c, -T, T))
```

```
    return pywt.waverec(out, wavelet)[:len(x)]


def clean_all(d, hp=1.0, lp=20, k=5.0, blink_thr=35.0, do_blink=True, do_ecg=True, do_wave=True):
    pk = r_peaks(d[6]); xs, bads = [], []
    for ci in range(3):
        x, bad = fix_saturation(d[ci]); x = highpass(x, hp)
        if do_ecg: x, _ = remove_ecg(x, pk)
        xs.append(x); bads.append(bad)
    bpk, cm = detect_blinks(xs, thr=blink_thr) if do_blink else (np.array([], int), None)
    out, info = {}, dict(n_R=int(len(pk)), n_blink=int(len(bpk)), blink_pk=bpk, bad={})
    for ci, name in enumerate(CH):
        x = xs[ci]
        if do_blink: x, tmpl = remove_blinks(x, bpk); info[f'blink_tmpl_{name}'] = tmpl
        if do_wave: x = wavelet_clip(x, k=k)
        out[name] = lowpass(x, lp); info['bad'][name] = bads[ci]
    return out, info


def machine_like(x, hp=2.0):
    """模拟机器自带滤波: 约 2 Hz 高通 + 20 Hz 低通(与我们同一低通便于比较)。"""
    x, _ = fix_saturation(x); return lowpass(highpass(x, hp), 20)


def plain_band(x, lo=1.0, hi=20):
    x, _ = fix_saturation(x); return lowpass(highpass(x, lo), hi)


def detrend_epochs(ep, order=2):
    """逐试次多项式去趋势(整段拟合)。"""
    n = ep.shape[1]; t = np.linspace(-1, 1, n); V = np.vander(t, order + 1)
    coef, *_ = np.linalg.lstsq(V, ep.T, rcond=None)
    return ep - (V @ coef).T


def robust_mean(ep, trim=0.1):
    n = ep.shape[0]; k = int(np.floor(n * trim)); s = np.sort(ep, 0); return s[k:n - k].mean(0)


def epoch_reject(ep, bad_mask_ep, pre, win=(-0.2, 0.6), cap=120, nmad=3.0):
    a, b = pre + int(win[0] * FS), pre + int(win[1] * FS)
    p2p = np.ptp(ep[:, a:b], 1); med = np.median(p2p); mad = 1.4826 * np.median(np.abs(p2p - med))
    return (~bad_mask_ep.any(1)) & (p2p < min(cap, med + nmad * mad))


def make_epochs(x, idx, pre, post, base, bad=None, detrend=True, reject=True):
    """一步到位: 切段 -> 去趋势 -> 基线 -> 剔除。返回 ep, keep(原索引), ok(布尔)。"""
    ep, keep = epochs(x, idx, pre, post)
    if detrend: ep = detrend_epochs(ep)
    ep = ep - ep[:, base].mean(1, keepdims=True)
    if bad is not None:
        bm, _ = epochs(bad.astype(float), idx, pre, post); bm = bm > 0
    else: bm = np.zeros(ep.shape, bool)
    ok = epoch_reject(ep, bm, pre) if reject else ~bm.any(1)
    return ep, keep, ok
```

q1_run.py

```
# 本程序及代码是在人工智能工具辅助下完成的: DeepSeek, 模型 DeepSeek-V4,
# 开发机构 杭州深度求索人工智能基础技术研究有限公司, 版本发布日期 2026 年 4 月 24 日。
"""问题一主程序: 四份数据 -> 去噪 -> 按左右提示叠加 -> 有效响应曲线 -> 滤波一致的高斯和拟合 -> 与机器滤
    波对照。"""
import numpy as np, json, sys
import matplotlib; matplotlib.use('Agg'); import matplotlib.pyplot as plt
from scipy.optimize import curve_fit
from scipy.signal import welch, butter, sosfiltfilt
```

```
from common import *
from preprocess import *

PRE, POST = int(0.5 * FS), int(1.2 * FS); TT = (np.arange(PRE + POST) - PRE) / FS
BASE = slice(PRE - int(0.2 * FS), PRE)
HP = float(sys.argv[1]) if len(sys.argv) > 1 else 1.0
SOS_HP = butter(2, HP / (FS / 2), 'high', output='sos'); SOS_LP = butter(4, 20 / (FS / 2), 'low',
    output='sos')
COMP = [('P1', 0.10, 0.03), ('N1', 0.18, 0.04), ('P2', 0.33, 0.05), ('N2', 0.46, 0.06)]
FIT_END = 0.6; PADN = 3 * FS


def gsum_raw(t, p):
    y = np.zeros_like(t)
    for i in range(0, len(p), 3): y += p[i] * np.exp(-0.5 * ((t - p[i + 1]) / p[i + 2]) ** 2)
    return y


def gsum_filt(t, *p):
    """模型曲线先过与数据同样的高通与低通(零相位)，再取对应时间点，这样拟出的幅度已经补偿了滤波衰减。
    """
    tl = np.arange(-PADN, PRE + POST + PADN) / FS + t[0] - (PRE / FS if len(t) == PRE + POST else
    0)
    tl = np.arange(-3.0, 3.0, 1 / FS)
    y = gsum_raw(tl, p); y = sosfiltfilt(SOS_LP, sosfiltfilt(SOS_HP, y))
    return np.interp(t, tl, y)


def fit_curve(t, y):
    m = (t >= -0.1) & (t <= FIT_END); t, y = t[m], y[m]
    p0, lo, hi = [], [], []
    for name, mu, sd in COMP: p0 += [0.0, mu, sd]; lo += [-30, mu - 0.06, 0.015]; hi += [30, mu +
    0.06, 0.12]
    try: p, _ = curve_fit(gsum_filt, t, y, p0=p0, bounds=(lo, hi), maxfev=20000)
    except Exception: p = np.array(p0)
    yh = gsum_filt(t, *p); r2 = 1 - ((y - yh) ** 2).sum() / ((y - y.mean()) ** 2).sum()
    comps = {COMP[i][0]: dict(amp=round(float(p[3 * i]), 2), lat=round(float(p[3 * i + 1]), 3), sd=
    round(float(p[3 * i + 2]), 3)) for i in range(len(COMP))}
    return p, float(r2), comps


def perm_test(a, b, n=4000, seed=0):
    obs = b.mean() - a.mean(); allv = np.r_[a, b]; rng = np.random.default_rng(seed); cnt = 0
    for _ in range(n): rng.shuffle(allv); cnt += abs(allv[len(a):].mean() - allv[:len(a)].mean())
    >= abs(obs)
    return float(obs), cnt / n


WINS = [(0.25, 0.45), (0.3, 0.5), (0.15, 0.3)]
summary, erps = {}, {}
for key in FILES:
    d = load(key); tr = trials(d)
    cues = np.array([k['cue'] for k in tr]); dirs = np.array([k['dir'] for k in tr])
    clean, info = clean_all(d, hp=HP)
    s = dict(n_R=info['n_R'], n_blink=info['n_blink'], blink_per_min=round(info['n_blink'] / (d.
    shape[1] / FS / 60), 1))
    mach, plain = {}, {}
    for ci, name in enumerate(CH):
        mach[name] = machine_like(d[ci]); plain[name] = plain_band(d[ci], lo=HP)
        f, p = welch(d[ci], FS, nperseg=FS * 4); s[f'noise_floor_20_40_{name}'] = round(float(np.
    median(p[(f > 20) & (f < 40)])), 2)
        s[f'sat_frac_{name}'] = round(float(info['bad'][name].mean()), 4)
    res = {}
    fig, ax = plt.subplots(3, 3, figsize=(16, 10), sharex=True)
    for ci, name in enumerate(CH):
```

```
    for row, (lab, src) in enumerate([('cleaned', clean), (f'{HP} Hz HP only', plain), ('
machine-like 2 Hz HP', mach)]):
        ep, keep, ok = make_epochs(src[name], cues, PRE, POST, BASE, info['bad'][name], detrend
=(row == 0), reject=(row == 0)); dd = dirs[keep]
        for sgn, col, cl in [(-1, 'b', 'L'), (1, 'r', 'R')]:
            sel = (dd == sgn) & ok; m = robust_mean(ep[sel]); se = ep[sel].std(0) / np.sqrt(sel
.sum())
            ax[row, ci].plot(TT, m, col, label=f'{cl} n={sel.sum()}'); ax[row, ci].fill_between
(TT, m - se, m + se, color=col, alpha=.15)
            if row == 0:
                res[f'{name}_{cl}'] = m.tolist(); res[f'{name}_{cl}_se'] = se.tolist(); res[f'{
name}_{cl}_n'] = int(sel.sum())
                p, r2, comps = fit_curve(TT, m); res[f'{name}_{cl}_fit'] = comps; res[f'{name}_
{cl}_fit_r2'] = round(r2, 3); res[f'{name}_{cl}_fit_p'] = p.tolist()
                mm = (TT >= -0.1) & (TT <= FIT_END); ax[row, ci].plot(TT[mm], gsum_filt(TT[mm],
 *p), col, ls='--', lw=1)
        if row == 0:
            selL, selR = (dd == -1) & ok, (dd == 1) & ok
            mall = robust_mean(ep[ok]); res[f'{name}_all'] = mall.tolist(); res[f'{name}_all_n
'] = int(ok.sum())
            p, r2, comps = fit_curve(TT, mall); res[f'{name}_all_fit'] = comps; res[f'{name}
_all_fit_r2'] = round(r2, 3); res[f'{name}_all_fit_p'] = p.tolist()
            for (w0, w1) in WINS:
                w = (TT >= w0) & (TT <= w1); obs, pv = perm_test(ep[selL][:, w].mean(1), ep[
selR][:, w].mean(1))
                res[f'{name}_RminusL_{int(w0*1000)}_{int(w1*1000)}'] = round(obs, 2); res[f'{
name}_RminusL_{int(w0*1000)}_{int(w1*1000)}_p'] = round(pv, 3)
            res[f'{name}_single_trial_rms_0_600'] = round(float(np.sqrt((ep[ok][:, PRE:PRE +
int(.6 * FS)] ** 2).mean())), 2)
            # P2 峰的显著性: 0.28-0.40 s 均值 vs 0 的 bootstrap
            w = (TT >= 0.28) & (TT <= 0.40); v = ep[ok][:, w].mean(1); res[f'{name}_P2_mean'] =
 round(float(v.mean()), 2); res[f'{name}_P2_t'] = round(float(v.mean() / (v.std() / np.sqrt(len
(v)))), 2)
        ax[row, ci].axvline(0, color='k', lw=.6); ax[row, ci].axvline(.2, color='k', lw=.6, ls
='--'); ax[row, ci].legend(fontsize=8); ax[row, ci].set_title(f'{name}  {lab}')
        if row > 0: ax[row, ci].set_ylim(-40, 60)
        ax[row, ci].set_xlim(-0.3, 1.0)
ax[-1, 0].set_xlabel('s from cue'); fig.suptitle(f'{key}  ERP by cue direction (row 1 cleaned,
dashed = filter-consistent Gaussian-sum fit)'); fig.tight_layout()
fig.savefig(f'{FIGS}/q1_{key}_erp.png', dpi=120); plt.close(fig)
# 偏侧化 F3-F4
ep3, keep3, ok3 = make_epochs(clean['F3'], cues, PRE, POST, BASE, info['bad']['F3'] | info['bad
']['F4'])
ep4, _, ok4 = make_epochs(clean['F4'], cues, PRE, POST, BASE, info['bad']['F3'] | info['bad']['
F4'])
lat = ep3 - ep4; dd = dirs[keep3]; ok = ok3 & ok4
fig, ax = plt.subplots(figsize=(8, 4))
for sgn, col, cl in [(-1, 'b', 'L'), (1, 'r', 'R')]:
    sel = (dd == sgn) & ok; m = robust_mean(lat[sel]); se = lat[sel].std(0) / np.sqrt(sel.sum()
)
    ax.plot(TT, m, col, label=f'{cl} n={sel.sum()}'); ax.fill_between(TT, m - se, m + se, color
=col, alpha=.15); res[f'lat_{cl}'] = m.tolist(); res[f'lat_{cl}_n'] = int(sel.sum())
for (w0, w1) in WINS:
    w = (TT >= w0) & (TT <= w1); obs, pv = perm_test(lat[(dd == -1) & ok][:, w].mean(1), lat[(
dd == 1) & ok][:, w].mean(1), seed=1)
    res[f'lat_RminusL_{int(w0*1000)}_{int(w1*1000)}'] = round(obs, 2); res[f'lat_RminusL_{int(
w0*1000)}_{int(w1*1000)}_p'] = round(pv, 4)
ax.axvline(0, color='k', lw=.6); ax.axvspan(.3, .5, color='y', alpha=.15); ax.legend(); ax.
set_xlim(-0.3, 1.0)
ax.set_title(f'{key}  F3-F4,  R-L 300-500 ms = {res["lat_RminusL_300_500"]:.2f} uV, p={res["
lat_RminusL_300_500_p"]:.3f}')
fig.tight_layout(); fig.savefig(f'{FIGS}/q1_{key}_lateral.png', dpi=120); plt.close(fig)
# 清洗前后示例
fig, ax = plt.subplots(3, 1, figsize=(14, 7), sharex=True); t0 = tr[12]['cue'] - 2 * FS; sl =
slice(t0, t0 + 25 * FS); t = np.arange(d.shape[1]) / FS
for ci, name in enumerate(CH):
```

```
        ax[ci].plot(t[sl], plain_band(d[ci], lo=0.3)[sl], color='0.6', lw=.5, label='0.3-20 Hz only
    '); ax[ci].plot(t[sl], clean[name][sl], 'k', lw=.6, label='cleaned'); ax[ci].set_ylabel(name)
        for kk in tr:
            if t0 <= kk['cue'] < t0 + 25 * FS: ax[ci].axvline(kk['cue'] / FS, color='r', lw=.5)
        ax[ci].set_ylim(-150, 150)
    ax[0].legend(); ax[-1].set_xlabel('s'); fig.suptitle(f'{key} cleaning example'); fig.
    tight_layout(); fig.savefig(f'{FIGS}/q1_{key}_clean_example.png', dpi=110); plt.close(fig)
    s.update({k: v for k, v in res.items() if not isinstance(v, list)}); summary[key] = s; erps[key
    ] = res
    print(key, json.dumps({k: v for k, v in s.items() if 'fit' not in k}, ensure_ascii=False))
    for name in CH: print(f'   {name} all r2={res[f"{name}_all_fit_r2"]}', res[f'{name}_all_fit'])
json.dump(summary, open(f'{OUT}/q1_summary.json', 'w'), indent=1, ensure_ascii=False)
json.dump(dict(t=TT.tolist(), erps=erps), open(f'{OUT}/q1_erps.json', 'w'))
```

q1_extra.py

```
# 本程序及代码是在人工智能工具辅助下完成的：DeepSeek，模型 DeepSeek-V4，
# 开发机构 杭州深度求索人工智能基础技术研究有限公司，版本发布日期 2026 年 4 月 24 日。
"""问题一补充：(1) 项目二负波与眨眼无关的检验 (2) 两被试按项目合并的 ERP 与偏侧化 (3) BIC 选成分数的自
    由高斯和拟合。"""
import numpy as np, json
import matplotlib; matplotlib.use('Agg'); import matplotlib.pyplot as plt
from scipy.optimize import least_squares
from scipy.signal import butter, sosfiltfilt
from common import *
from preprocess import *

PRE, POST = int(0.5 * FS), int(1.2 * FS); TT = (np.arange(PRE + POST) - PRE) / FS; BASE = slice(PRE
     - int(0.2 * FS), PRE)
SOS_HP = butter(2, 1.0 / (FS / 2), 'high', output='sos'); SOS_LP = butter(4, 20 / (FS / 2), 'low',
    output='sos')
TL = np.arange(-3.0, 3.0, 1 / FS)
# 滤波器(零相位高通+低通)的脉冲响应，模型曲线用卷积过滤波器，比逐次 sosfiltfilt 快一个量级
_imp = np.zeros(len(TL)); _imp[len(TL) // 2] = 1.0; HIMP = sosfiltfilt(SOS_LP, sosfiltfilt(SOS_HP,
    _imp))
HIMP = HIMP[len(TL) // 2 - 2 * FS: len(TL) // 2 + 2 * FS]
from scipy.signal import fftconvolve


MIN_GAP = 0.06


def unpack(q):
    """q = [amp_1, lat_1, sd_1, amp_2, gap_2, sd_2, ...], lat_i = lat_{i-1} + MIN_GAP + gap_i, 保证
    成分潜伏期有序且不重合。"""
    p = np.array(q, float).copy()
    for i in range(3, len(p), 3): p[i + 1] = p[i - 2] + MIN_GAP + p[i + 1]
    return p


def model(p, t):
    y = np.zeros_like(TL)
    for i in range(0, len(p), 3): y += p[i] * np.exp(-0.5 * ((TL - p[i + 1]) / p[i + 2]) ** 2)
    y = fftconvolve(y, HIMP, mode='same'); return np.interp(t, TL, y)


def free_fit(t, y, kmax=4, t0=-0.1, t1=0.65, seed=0, starts=8):
    """K=1..kmax 个高斯，潜伏期自由在 [0.05,0.6] 且相邻至少隔 MIN_GAP，幅度 |a|<=25，多起点，按 BIC
    选 K。"""
    m = (t >= t0) & (t <= t1); t, y = t[m], y[m]; n = len(y); rng = np.random.default_rng(seed);
    best = None
    for K in range(1, kmax + 1):
        bk = None
        for s in range(starts):
            q0 = [rng.uniform(-8, 8), rng.uniform(0.08, 0.35), rng.uniform(0.03, 0.08)]
            lo = [-25, 0.05, 0.02]; hi = [25, 0.6, 0.15]
```

```
            for i in range(1, K): q0 += [rng.uniform(-8, 8), rng.uniform(0.0, 0.15), rng.uniform
    (0.03, 0.08)]; lo += [-25, 0.0, 0.02]; hi += [25, 0.4, 0.15]
            try: r = least_squares(lambda q: model(unpack(q), t) - y, q0, bounds=(lo, hi), max_nfev
    =600)
            except Exception: continue
            if bk is None or r.cost < bk.cost: bk = r
        p = unpack(bk.x)
        if p[1::3].max() > 0.62: continue
        rss = 2 * bk.cost; bic = n * np.log(rss / n) + 3 * K * np.log(n)
        if best is None or bic < best[0]: best = (bic, K, p, rss)
    bic, K, p, rss = best; r2 = 1 - rss / ((y - y.mean()) ** 2).sum()
    comps = [dict(amp=round(float(p[3 * i]), 2), lat=round(float(p[3 * i + 1]), 3), sd=round(float(
    p[3 * i + 2]), 3)) for i in range(K)]
    return dict(K=int(K), r2=round(float(r2), 3), comps=comps, p=p.tolist())


out = {}
# ---------- 每份数据的段 ----------
EP = {}
for key in FILES:
    d = load(key); tr = trials(d); cues = np.array([k['cue'] for k in tr]); dirs = np.array([k['dir
    '] for k in tr])
    clean, info = clean_all(d); bpk = info['blink_pk']
    # 每个试次：提示后 0.3-1.0 s 内是否有眨眼事件
    has_blink = np.array([np.any((bpk > c + int(0.3 * FS)) & (bpk < c + int(1.0 * FS))) for c in
    cues])
    EP[key] = {}
    for name in CH:
        ep, keep, ok = make_epochs(clean[name], cues, PRE, POST, BASE, info['bad'][name])
        EP[key][name] = dict(ep=ep, dirs=dirs[keep], ok=ok, blink=has_blink[keep])
    out[key] = dict(n_trials_with_blink_300_1000=int(has_blink.sum()))

# ---------- (1) 眨眼分组检验 ----------
fig, ax = plt.subplots(2, 3, figsize=(15, 7), sharex=True)
for r, key in enumerate(['A2', 'B2']):
    for ci, name in enumerate(CH):
        e = EP[key][name]; w = (TT >= 0.3) & (TT <= 0.5)
        for flag, col, lab in [(False, 'g', 'no blink 0.3-1.0 s'), (True, 'm', 'blink present')]:
            sel = e['ok'] & (e['blink'] == flag)
            if sel.sum() < 5: continue
            m = robust_mean(e['ep'][sel]); se = e['ep'][sel].std(0) / np.sqrt(sel.sum())
            ax[r, ci].plot(TT, m, col, label=f'{lab} n={sel.sum()}'); ax[r, ci].fill_between(TT, m
    - se, m + se, color=col, alpha=.15)
            v = e['ep'][sel][:, w].mean(1); out[key][f'{name}_neg300_500_{"blink" if flag else "
    noblink"}'] = dict(mean=round(float(v.mean()), 2), t=round(float(v.mean() / (v.std() / np.sqrt(
    len(v)))), 2), n=int(sel.sum()))
        ax[r, ci].axvline(0, color='k', lw=.6); ax[r, ci].axvspan(.3, .5, color='y', alpha=.15); ax
    [r, ci].legend(fontsize=8); ax[r, ci].set_title(f'{key} {name}'); ax[r, ci].set_xlim(-0.3, 1.0)
fig.suptitle('Task 2 negativity: trials split by blink occurrence after the cue'); fig.tight_layout
    (); fig.savefig(f'{FIGS}/q1_task2_blink_split.png', dpi=120); plt.close(fig)

# ---------- (2) 合并两被试 ----------
pool = {}
fig, ax = plt.subplots(2, 4, figsize=(20, 8), sharex=True)
for r, (task, keys) in enumerate([('Task1', ['A1', 'B1']), ('Task2', ['A2', 'B2'])]):
    pool[task] = {}
    for ci, name in enumerate(CH + ['F3-F4']):
        if name == 'F3-F4':
            eps = [EP[k]['F3']['ep'] - EP[k]['F4']['ep'] for k in keys]; oks = [EP[k]['F3']['ok'] &
     EP[k]['F4']['ok'] for k in keys]
        else:
            eps = [EP[k][name]['ep'] for k in keys]; oks = [EP[k][name]['ok'] for k in keys]
        ep = np.concatenate(eps); ok = np.concatenate(oks); dd = np.concatenate([EP[k]['Fz']['dirs
    '] for k in keys])
        rec = {}
        for sgn, col, cl in [(-1, 'b', 'L'), (1, 'r', 'R'), (0, 'k', 'all')]:
```

```
            sel = ok if sgn == 0 else (ok & (dd == sgn)); m = robust_mean(ep[sel]); se = ep[sel].
    std(0) / np.sqrt(sel.sum())
            ax[r, ci].plot(TT, m, col, label=f'{cl} n={sel.sum()}', lw=1.6 if sgn == 0 else 1); ax[
    r, ci].fill_between(TT, m - se, m + se, color=col, alpha=.12)
            rec[cl] = m.tolist(); rec[cl + '_se'] = se.tolist(); rec[cl + '_n'] = int(sel.sum())
            if name != 'F3-F4' or sgn != 0:
                f = free_fit(TT, m); rec[cl + '_fit'] = f
                mm = (TT >= -0.1) & (TT <= 0.65); ax[r, ci].plot(TT[mm], model(np.array(f['p']), TT
    [mm]), col, ls='--', lw=1)
        for (w0, w1) in [(0.15, 0.3), (0.25, 0.45), (0.3, 0.5)]:
            w = (TT >= w0) & (TT <= w1); a, b = ep[ok & (dd == -1)][:, w].mean(1), ep[ok & (dd ==
    1)][:, w].mean(1)
            obs = b.mean() - a.mean(); allv = np.r_[a, b]; rng = np.random.default_rng(2); cnt = 0
            for _ in range(4000): rng.shuffle(allv); cnt += abs(allv[len(a):].mean() - allv[:len(a)
    ].mean()) >= abs(obs)
            rec[f'RminusL_{int(w0*1000)}_{int(w1*1000)}'] = round(float(obs), 2); rec[f'RminusL_{
    int(w0*1000)}_{int(w1*1000)}_p'] = round(cnt / 4000, 4)
            rec[f'RminusL_{int(w0*1000)}_{int(w1*1000)}_se'] = round(float(np.sqrt(a.var() / len(a)
     + b.var() / len(b))), 2)
        ax[r, ci].axvline(0, color='k', lw=.6); ax[r, ci].axvline(.2, color='k', lw=.6, ls='--');
    ax[r, ci].legend(fontsize=8); ax[r, ci].set_title(f'{task} {name}'); ax[r, ci].set_xlim(-0.3,
    1.0)
        pool[task][name] = rec
        print(task, name, 'all fit', rec['all_fit'] if 'all_fit' in rec else '', 'R-L 300-500', rec
    ['RminusL_300_500'], 'p', rec['RminusL_300_500_p'])
fig.suptitle('Pooled over two subjects (dashed = BIC-selected Gaussian-sum fit, filter-consistent)
    '); fig.tight_layout(); fig.savefig(f'{FIGS}/q1_pooled_erp.png', dpi=120); plt.close(fig)
out['pooled'] = pool
# ---------- (3) 每份数据 all/L/R 自由拟合表 ----------
tab = {}
for key in FILES:
    tab[key] = {}
    for name in CH:
        e = EP[key][name]
        for cl, sel in [('all', e['ok']), ('L', e['ok'] & (e['dirs'] == -1)), ('R', e['ok'] & (e['
    dirs'] == 1))]:
            tab[key][f'{name}_{cl}'] = free_fit(TT, robust_mean(e['ep'][sel]))
        print(key, name, 'all', tab[key][f'{name}_all']['K'], tab[key][f'{name}_all']['r2'], tab[
    key][f'{name}_all']['comps'])
out['fits'] = tab
print(json.dumps({k: v for k, v in out.items() if k in FILES}, indent=1))
json.dump(out, open(f'{OUT}/q1_extra.json', 'w'), indent=1)
json.dump(dict(t=TT.tolist(), pooled=pool), open(f'{OUT}/q1_pooled.json', 'w'))
```

q1_validate.py

```
# 本程序及代码是在人工智能工具辅助下完成的：DeepSeek，模型 DeepSeek-V4，
# 开发机构 杭州深度求索人工智能基础技术研究有限公司，版本发布日期 2026 年 4 月 24 日。
"""问题一验证：往真实原始数据里注入已知的合成视觉响应，看各处理方案还原了多少，并扫描高通截止做权衡。
合成响应 = P2 正峰(0.33 s, 6 uV) + N2 负波(0.45 s, -4 uV) + 偏侧成分(F3 与 F4 反号, 0.4 s, ±3 uV)。
注入位置取每个试次目标显示后 3.5 s 处(离真实提示与眨眼都远)，随机赋左右方向。"""
import numpy as np, json, sys
import matplotlib; matplotlib.use('Agg'); import matplotlib.pyplot as plt
from common import *
from preprocess import *

PRE, POST = int(0.5 * FS), int(1.2 * FS); TT = (np.arange(PRE + POST) - PRE) / FS; BASE = slice(PRE
     - int(0.2 * FS), PRE)
g = lambda t, a, mu, sd: a * np.exp(-0.5 * ((t - mu) / sd) ** 2)
tt = np.arange(0, 0.8, 1 / FS)
common = g(tt, 6, 0.33, 0.045) + g(tt, -4, 0.45, 0.06); lat = g(tt, 3, 0.40, 0.06)
truth_c = np.zeros(PRE + POST); truth_c[PRE:PRE + len(tt)] = common
truth_l = np.zeros(PRE + POST); truth_l[PRE:PRE + len(tt)] = lat
M = (TT >= 0) & (TT <= 0.6)
```

```
def inject(key, seed=0):
    d = load(key).copy(); tr = trials(d); rng = np.random.default_rng(seed)
    inj = np.array([k['tgt'] + int(3.5 * FS) for k in tr if k['tgt'] and k['tgt'] + int(5 * FS) < d
    .shape[1]])
    sgn = rng.choice([-1, 1], size=len(inj))
    for i, sg in zip(inj, sgn):
        d[0, i:i + len(tt)] += common; d[1, i:i + len(tt)] += common + sg * lat; d[2, i:i + len(tt)
    ] += common - sg * lat
    return d, inj, sgn


def score(x, inj, sgn, bad, detrend=True, reject=True, sign=0):
    ep, keep, ok = make_epochs(x, inj, PRE, POST, BASE, bad, detrend=detrend, reject=reject); ss =
    sgn[keep]
    mR, mL = robust_mean(ep[(ss > 0) & ok]), robust_mean(ep[(ss < 0) & ok]); mAll = (mR + mL) / 2;
    mDiff = sign * (mR - mL) / 2
    r = dict(n=int(ok.sum()), corr=float(np.corrcoef(mAll[M], truth_c[M])[0, 1]), gain=float(mAll[M
    ] @ truth_c[M] / (truth_c[M] @ truth_c[M])),
             rmse=float(np.sqrt(((mAll[M] - truth_c[M]) ** 2).mean())), wave=mAll)
    if sign:
        r['lat_corr'] = float(np.corrcoef(mDiff[M], truth_l[M])[0, 1]); r['lat_gain'] = float(mDiff
    [M] @ truth_l[M] / (truth_l[M] @ truth_l[M]))
        r['lat_rmse'] = float(np.sqrt(((mDiff[M] - truth_l[M]) ** 2).mean()))
    return r


mode = sys.argv[1] if len(sys.argv) > 1 else 'all'
out = {}
if mode in ('sweep', 'all'):
    # 高通截止扫描: 完整流水线 vs 只高通低通
    hps = [0.3, 0.5, 0.75, 1.0, 1.5, 2.0, 3.0]; sweep = {}
    for key in FILES:
        d, inj, sgn = inject(key); sweep[key] = {}
        for hp in hps:
            clean, info = clean_all(d, hp=hp)
            row = {}
            for ci, name in enumerate(CH):
                r = score(clean[name], inj, sgn, info['bad'][name], sign=(0 if ci == 0 else (1 if
    ci == 1 else -1)))
                r2 = score(plain_band(d[ci], lo=hp), inj, sgn, info['bad'][name], detrend=False,
    reject=False)
                row[name] = {k: round(v, 3) for k, v in r.items() if k != 'wave'}; row[name + '
    _plainHP'] = {k: round(v, 3) for k, v in r2.items() if k != 'wave'}
            sweep[key][str(hp)] = row
            print(key, hp, ' '.join(f'{n}: g={row[n]["gain"]:.2f} rmse={row[n]["rmse"]:.2f} n={row[
    n]["n"]}' for n in CH), '| plain', ' '.join(f'{n}: g={row[n+"_plainHP"]["gain"]:.2f} rmse={row[
    n+"_plainHP"]["rmse"]:.2f}' for n in CH))
    out['sweep'] = sweep
    fig, ax = plt.subplots(1, 2, figsize=(12, 4))
    for key in FILES:
        gm = [np.mean([sweep[key][str(h)][n]['gain'] for n in CH]) for h in hps]; rm = [np.mean([
    sweep[key][str(h)][n]['rmse'] for n in CH]) for h in hps]
        ax[0].plot(hps, gm, 'o-', label=key); ax[1].plot(hps, rm, 'o-', label=key)
        gm = [np.mean([sweep[key][str(h)][n + '_plainHP']['gain'] for n in CH]) for h in hps]; rm =
     [np.mean([sweep[key][str(h)][n + '_plainHP']['rmse'] for n in CH]) for h in hps]
        ax[0].plot(hps, gm, 'x--', color=ax[0].lines[-1].get_color(), alpha=.6); ax[1].plot(hps, rm
    , 'x--', color=ax[1].lines[-1].get_color(), alpha=.6)
    ax[0].set_xlabel('high-pass cutoff Hz'); ax[0].set_ylabel('recovered gain'); ax[0].axhline(1,
    color='k', lw=.5); ax[1].set_xlabel('high-pass cutoff Hz'); ax[1].set_ylabel('RMSE uV')
    ax[0].legend(); ax[0].set_title('solid = full pipeline, dashed = high-pass only'); fig.
    tight_layout(); fig.savefig(f'{FIGS}/q1_validate_sweep.png', dpi=120); plt.close(fig)
if mode in ('final', 'all'):
    res = {}
    fig, axs = plt.subplots(4, 3, figsize=(15, 13), sharex=True)
    for fi, key in enumerate(FILES):
        d, inj, sgn = inject(key); clean, info = clean_all(d); res[key] = {}
```

```
        for ci, name in enumerate(CH):
            sign = 0 if ci == 0 else (1 if ci == 1 else -1)
            variants = [('cleaned', score(clean[name], inj, sgn, info['bad'][name], sign=sign)),
                        ('1 Hz HP only', score(plain_band(d[ci], lo=1.0), inj, sgn, info['bad'][
    name], detrend=False, reject=False, sign=sign)),
                        ('machine-like 2 Hz HP', score(machine_like(d[ci]), inj, sgn, info['bad'][
    name], detrend=False, reject=False, sign=sign))]
            for lab, r in variants:
                res[key][f'{name}|{lab}'] = {k: round(v, 3) for k, v in r.items() if k != 'wave'}
                axs[fi, ci].plot(TT, r['wave'], label=f'{lab}: corr={r["corr"]:.2f} gain={r["gain
    "]:.2f} rmse={r["rmse"]:.1f}')
            axs[fi, ci].plot(TT, truth_c, 'k--', label='truth'); axs[fi, ci].set_title(f'{key} {
    name}'); axs[fi, ci].legend(fontsize=7); axs[fi, ci].set_ylim(-12, 12); axs[fi, ci].set_xlim
    (-0.3, 1.0)
        print(key, json.dumps(res[key]))
    axs[-1, 0].set_xlabel('s'); fig.suptitle('Injected synthetic response recovery (common part of
    F3/F4/Fz)'); fig.tight_layout(); fig.savefig(f'{FIGS}/q1_validate_injection.png', dpi=120)
    out['final'] = res
json.dump(out, open(f'{OUT}/q1_validate_{mode}.json', 'w'), indent=1)
```

q1_compare.py

```
# 本程序及代码是在人工智能工具辅助下完成的：DeepSeek，模型 DeepSeek-V4，
# 开发机构 杭州深度求索人工智能基础技术研究有限公司，版本发布日期 2026 年 4 月 24 日。
"""问题一 多方法对比：同一套注入检验下比较本文流水线、它的三个消融版本与三种常用做法。

注入与评分沿用 q1_validate：在每个试次目标出现后 3.5 s 处加入已知合成响应，看叠加后能找回多少。
对比的方法：
  本文方法                完整流水线
  去掉小波截断            do_wave=False
  去掉眨眼模板            do_blink=False
  去掉心电模板            do_ecg=False
  带通加同样的剔除        1 到 20 Hz 带通，再做与本文相同的逐段去趋势与剔除
  仿机器滤波              2 Hz 高通，不剔除
  ICA 去眼电              1 到 20 Hz 带通后三通道 FastICA，去掉与眨眼时刻最相关的一个成分
指标：幅度保留率、与真值的相关、均方根误差、单份数据处理用时。
"""
import numpy as np, json, time
from sklearn.decomposition import FastICA
from common import *
from preprocess import *

# 注入与评分与 q1_validate 相同。那个脚本导入时会整份重跑，所以这里抄一份
PRE, POST = int(0.5 * FS), int(1.2 * FS); TT = (np.arange(PRE + POST) - PRE) / FS; BASE = slice(PRE
    - int(0.2 * FS), PRE)
g_ = lambda t, a, mu, sd: a * np.exp(-0.5 * ((t - mu) / sd) ** 2)
tt = np.arange(0, 0.8, 1 / FS)
common_ = g_(tt, 6, 0.33, 0.045) + g_(tt, -4, 0.45, 0.06); lat_ = g_(tt, 3, 0.40, 0.06)
truth_c = np.zeros(PRE + POST); truth_c[PRE:PRE + len(tt)] = common_
M = (TT >= 0) & (TT <= 0.6)


def inject(key, seed=0):
    d = load(key).copy(); tr = trials(d); rng = np.random.default_rng(seed)
    inj = np.array([k['tgt'] + int(3.5 * FS) for k in tr if k['tgt'] and k['tgt'] + int(5 * FS) < d
    .shape[1]])
    sgn = rng.choice([-1, 1], size=len(inj))
    for i, sg in zip(inj, sgn):
        d[0, i:i + len(tt)] += common_; d[1, i:i + len(tt)] += common_ + sg * lat_; d[2, i:i + len(
    tt)] += common_ - sg * lat_
    return d, inj, sgn


def score(x, inj, sgn, bad, detrend=True, reject=True):
    ep, keep, ok = make_epochs(x, inj, PRE, POST, BASE, bad, detrend=detrend, reject=reject); ss =
    sgn[keep]
```

```
    mAll = (robust_mean(ep[(ss > 0) & ok]) + robust_mean(ep[(ss < 0) & ok])) / 2
    return dict(corr=float(np.corrcoef(mAll[M], truth_c[M])[0, 1]),
                gain=float(mAll[M] @ truth_c[M] / (truth_c[M] @ truth_c[M])),
                rmse=float(np.sqrt(((mAll[M] - truth_c[M]) ** 2).mean())))


def ica_clean(d):
    X = np.array([plain_band(d[ci]) for ci in range(3)])
    ica = FastICA(n_components=3, random_state=0, max_iter=1000)
    S = ica.fit_transform(X.T)                          # (n, 3)
    pk, cm = detect_blinks([X[0], X[1], X[2]])
    m = np.zeros(X.shape[1]); w = int(0.2 * FS)
    for p in pk: m[max(0, p - w):p + w] = 1.0
    k = int(np.argmax([abs(np.corrcoef(np.abs(S[:, j]), m)[0, 1]) for j in range(3)]))
    S[:, k] = 0.0
    Y = ica.inverse_transform(S).T
    return {name: Y[ci] for ci, name in enumerate(CH)}


METHODS = ['本文方法', '去掉小波截断', '去掉眨眼模板', '去掉心电模板', '带通加同样的剔除', '仿机器滤波
    ', 'ICA 去眼电']

if __name__ == '__main__':
    res = {m: dict(gain=[], corr=[], rmse=[], sec=[]) for m in METHODS}
    for key in FILES:
        d, inj, sgn = inject(key)
        _, info = clean_all(d)
        for m in METHODS:
            t0 = time.time()
            if m == '本文方法': x, _ = clean_all(d)
            elif m == '去掉小波截断': x, _ = clean_all(d, do_wave=False)
            elif m == '去掉眨眼模板': x, _ = clean_all(d, do_blink=False)
            elif m == '去掉心电模板': x, _ = clean_all(d, do_ecg=False)
            elif m == '带通加同样的剔除': x = {n: plain_band(d[ci]) for ci, n in enumerate(CH)}
            elif m == '仿机器滤波': x = {n: machine_like(d[ci]) for ci, n in enumerate(CH)}
            else: x = ica_clean(d)
            res[m]['sec'].append(time.time() - t0)
            full = m in ('本文方法', '去掉小波截断', '去掉眨眼模板', '去掉心电模板', '带通加同样的剔除
    ', 'ICA 去眼电')
            for ci, name in enumerate(CH):
                r = score(x[name], inj, sgn, info['bad'][name], detrend=full, reject=full)
                for k in ('gain', 'corr', 'rmse'): res[m][k].append(r[k])
        print(key, 'done', flush=True)
    out = {}
    for m in METHODS:
        g = np.array(res[m]['gain'])
        out[m] = dict(gain_mean=round(float(g.mean()), 3), gain_min=round(float(g.min()), 3),
    gain_max=round(float(g.max()), 3),
                      gain_abs_dev=round(float(np.abs(g - 1).mean()), 3),
                      corr_mean=round(float(np.mean(res[m]['corr'])), 3), rmse_mean=round(float(np.
    mean(res[m]['rmse'])), 3),
                      sec_per_file=round(float(np.mean(res[m]['sec'])), 2))
        print(f'{m:10s}', out[m], flush=True)
    json.dump(out, open(f'{OUT}/q1_compare.json', 'w', encoding='utf-8'), indent=1, ensure_ascii=
    False)
```

q1_refit.py

```
# 本程序及代码是在人工智能工具辅助下完成的：DeepSeek，模型 DeepSeek-V4，
# 开发机构 杭州深度求索人工智能基础技术研究有限公司，版本发布日期 2026 年 4 月 24 日。
"""问题一 曲线拟合复核：放宽幅度与宽度的上界再拟合一次。

原拟合把幅度限在 25 uV、宽度限在 0.15 s，项目二的宽负波两项成分都顶到了上界，
拟合曲线在提示前偏离数据。这里放宽到 80 uV 与 0.30 s，其余与 q1_extra 相同：
滤波一致，潜伏期有序且相邻至少隔 60 ms，K=1..4 按 BIC 选，每个 K 12 个起点。
"""
```

```
import numpy as np, json
from scipy.optimize import least_squares
from scipy.signal import butter, sosfiltfilt, fftconvolve
from common import FS, OUT, CH

TL = np.arange(-3.0, 3.0, 1 / FS)
_imp = np.zeros(len(TL)); _imp[len(TL) // 2] = 1.0
HIMP = sosfiltfilt(butter(4, 20 / (FS / 2), 'low', output='sos'), sosfiltfilt(butter(2, 1.0 / (FS /
     2), 'high', output='sos'), _imp))
HIMP = HIMP[len(TL) // 2 - 2 * FS: len(TL) // 2 + 2 * FS]
MIN_GAP = 0.06
AMAX, SMAX = 80.0, 0.30


def unpack(q):
    p = np.array(q, float).copy()
    for i in range(3, len(p), 3): p[i + 1] = p[i - 2] + MIN_GAP + p[i + 1]
    return p


def model(p, t):
    y = np.zeros_like(TL)
    for i in range(0, len(p), 3): y += p[i] * np.exp(-0.5 * ((TL - p[i + 1]) / p[i + 2]) ** 2)
    return np.interp(t, TL, fftconvolve(y, HIMP, mode='same'))


def free_fit(t, y, kmax=4, t0=-0.1, t1=0.65, seed=0, starts=12):
    m = (t >= t0) & (t <= t1); t, y = t[m], y[m]; n = len(y); rng = np.random.default_rng(seed);
    best = None; allk = {}
    for K in range(1, kmax + 1):
        bk = None
        for s in range(starts):
            q0 = [rng.uniform(-10, 10), rng.uniform(0.08, 0.35), rng.uniform(0.03, 0.12)]
            lo = [-AMAX, 0.05, 0.02]; hi = [AMAX, 0.6, SMAX]
            for i in range(1, K):
                q0 += [rng.uniform(-10, 10), rng.uniform(0.0, 0.15), rng.uniform(0.03, 0.12)]
                lo += [-AMAX, 0.0, 0.02]; hi += [AMAX, 0.4, SMAX]
            try: r = least_squares(lambda q: model(unpack(q), t) - y, q0, bounds=(lo, hi), max_nfev
    =800)
            except Exception: continue
            if bk is None or r.cost < bk.cost: bk = r
        p = unpack(bk.x)
        if p[1::3].max() > 0.62: continue
        rss = 2 * bk.cost; bic = n * np.log(rss / n) + 3 * K * np.log(n)
        allk[K] = dict(bic=round(float(bic), 1), r2=round(float(1 - rss / ((y - y.mean()) ** 2).sum
    ()), 3))
        if best is None or bic < best[0]: best = (bic, K, p, rss)
    bic, K, p, rss = best
    comps = [dict(amp=round(float(p[3 * i]), 2), lat=round(float(p[3 * i + 1]), 3), sd=round(float(
    p[3 * i + 2]), 3)) for i in range(K)]
    at_bound = bool(np.any(np.abs(p[0::3]) >= AMAX - 1e-3) or np.any(p[2::3] >= SMAX - 1e-4))
    return dict(K=int(K), r2=round(float(1 - rss / ((y - y.mean()) ** 2).sum()), 3), comps=comps, p
    =p.tolist(),
                by_K=allk, at_bound=at_bound)


if __name__ == '__main__':
    P = json.load(open(f'{OUT}/q1_pooled.json', encoding='utf-8')); t = np.array(P['t'])
    out = {}
    for task in ('Task1', 'Task2'):
        out[task] = {}
        for c in CH:
            f = free_fit(t, np.array(P['pooled'][task][c]['all']))
            out[task][c] = f
            old = P['pooled'][task][c]['all_fit']
```

```
            print(task, c, 'K', f['K'], 'R2', f['r2'], '原', old['K'], old['r2'], '顶界', f['
    at_bound'], f['comps'], flush=True)
    json.dump(out, open(f'{OUT}/q1_fit_relaxed.json', 'w', encoding='utf-8'), indent=1,
    ensure_ascii=False)
```

q1_revise.py

```
# 本程序及代码是在人工智能工具辅助下完成的：DeepSeek，模型 DeepSeek-V4，
# 开发机构 杭州深度求索人工智能基础技术研究有限公司，版本发布日期 2026 年 4 月 24 日。
"""问题一修订计算。
一 成分数按有效样本数选：数据经 20 Hz 低通后以 256 Hz 采样，相邻点强相关，有效独立样本约为
   n_e = n * 2 * 20 / 256。BIC_e = n_e ln(RSS/n) + 3K ln n_e。每个 K 都保留解，不因潜伏期越界丢掉。
二 心电模板的作用：按 R 波锁时平均，比较有无心电模板时 -0.25 到 0.55 s 的峰峰值。
三 小波截断的作用：有无小波截断时，提示锁时段被峰峰值准则剔除的试次数与 13 到 20 Hz 能量。
"""
import numpy as np, json
from scipy.optimize import least_squares
from scipy.signal import butter, sosfiltfilt
from common import *
from preprocess import *
from q1_refit import model, unpack, MIN_GAP, AMAX, SMAX

FC_LP = 20.0


def fit_allK(t, y, kmax=4, t0=-0.1, t1=0.65, seed=0, starts=16):
    m = (t >= t0) & (t <= t1); t, y = t[m], y[m]; n = len(y); ne = n * 2 * FC_LP / FS
    rng = np.random.default_rng(seed); sst = ((y - y.mean()) ** 2).sum(); res = {}
    for K in range(1, kmax + 1):
        # 潜伏期间隔的上界随 K 收紧，保证最后一个成分不超过 0.6 s
        gap_hi = max(0.02, (0.6 - 0.05 - MIN_GAP * (K - 1)) / max(K - 1, 1))
        bk = None
        for s in range(starts):
            q0 = [rng.uniform(-10, 10), rng.uniform(0.06, 0.30), rng.uniform(0.03, 0.12)]
            lo = [-AMAX, 0.05, 0.02]; hi = [AMAX, 0.6, SMAX]
            for i in range(1, K):
                q0 += [rng.uniform(-10, 10), rng.uniform(0.0, gap_hi), rng.uniform(0.03, 0.12)]
                lo += [-AMAX, 0.0, 0.02]; hi += [AMAX, gap_hi, SMAX]
            q0 = np.clip(q0, lo, hi)
            try: r = least_squares(lambda q: model(unpack(q), t) - y, q0, bounds=(lo, hi), max_nfev
    =800)
            except Exception: continue
            if bk is None or r.cost < bk.cost: bk = r
        p = unpack(bk.x); rss = 2 * bk.cost
        res[K] = dict(rss=float(rss), r2=round(float(1 - rss / sst), 3),
                      bic=round(float(n * np.log(rss / n) + 3 * K * np.log(n)), 1),
                      bic_e=round(float(ne * np.log(rss / n) + 3 * K * np.log(ne)), 1),
                      comps=[dict(amp=round(float(p[3 * i]), 2), lat=round(float(p[3 * i + 1]), 3),
     sd=round(float(p[3 * i + 2]), 3)) for i in range(K)],
                      at_bound=bool(np.any(np.abs(p[0::3]) >= AMAX - 1e-3) or np.any(p[2::3] >=
    SMAX - 1e-4) or np.any(p[2::3] <= 0.02 + 1e-4) or p[1] <= 0.05 + 1e-4))
    Ke = min(res, key=lambda k: res[k]['bic_e']); Kn = min(res, key=lambda k: res[k]['bic'])
    return dict(n=int(n), n_e=round(float(ne), 1), K_bic=int(Kn), K_bic_e=int(Ke), byK=res)


if __name__ == '__main__':
    out = {}
    P = json.load(open(f'{OUT}/q1_pooled.json', encoding='utf-8')); t = np.array(P['t'])
    out['fit'] = {}
    for task in ('Task1', 'Task2'):
        out['fit'][task] = {}
        for c in CH:
            f = fit_allK(t, np.array(P['pooled'][task][c]['all']))
            out['fit'][task][c] = f
            k = f['K_bic_e']
```

```
            print(task, c, 'K(BIC)', f['K_bic'], 'K(BIC_e)', k, 'R2', f['byK'][k]['r2'], 'bound', f
    ['byK'][k]['at_bound'],
                  {K: (v['r2'], v['bic_e']) for K, v in f['byK'].items()}, flush=True)
    # 心电与小波的作用
    ecg, wav = {}, {}
    for key in FILES:
        d = load(key); pk = r_peaks(d[6])
        full, info = clean_all(d); noecg, _ = clean_all(d, do_ecg=False); nowav, _ = clean_all(d,
    do_wave=False)
        bad = info['bad']['Fz'] | info['bad']['F3'] | info['bad']['F4']
        e = {}
        for c in CH:
            vals = []
            for x in (noecg[c], full[c]):
                ep, keep = epochs(x, pk, int(0.25 * FS), int(0.55 * FS))
                ep = ep - ep[:, :int(0.1 * FS)].mean(1, keepdims=True); vals.append(float(np.ptp(np
    .median(ep, 0))))
            e[c] = [round(v, 2) for v in vals]
        ecg[key] = e
        cues = np.array([k['cue'] for k in trials(d)]); w = {}
        sos = butter(3, [13 / (FS / 2), 20 / (FS / 2)], 'band', output='sos')
        for c in CH:
            r = []
            for x in (nowav[c], full[c]):
                ep, keep, ok = make_epochs(x, cues, int(0.5 * FS), int(1.2 * FS), slice(int(0.3 *
    FS), int(0.5 * FS)), bad)
                hf = float(np.mean(sosfiltfilt(sos, x) ** 2))
                r.append((int((~ok).sum()), round(hf, 3)))
            w[c] = r
        wav[key] = w
        print(key, 'ECG 锁时残余峰峰值 [无模板, 有模板]', e, '| 剔除试次与 13-20 Hz 能量 [无小波, 有
    小波]', w, flush=True)
    out['ecg_residual_p2p'] = ecg; out['wavelet_reject_hf'] = wav
    json.dump(out, open(f'{OUT}/q1_revise.json', 'w', encoding='utf-8'), indent=1, ensure_ascii=
    False)
```

q2_visual.py

```
# 本程序及代码是在人工智能工具辅助下完成的：DeepSeek，模型 DeepSeek-V4，
# 开发机构 杭州深度求索人工智能基础技术研究有限公司，版本发布日期 2026 年 4 月 24 日。
"""问题二 第一段：视觉刺激 -> 视网膜/LGN -> V1 方向柱 -> 顶点单元 -> COSFIRE 形状单元。
给出左右三角形各自激活的皮层神经元空间分布，以及可区分左右的特征表示(顶点元组集合)。"""
import numpy as np
from scipy.ndimage import gaussian_filter, shift as ndshift, maximum_filter

N = 97                          # 视网膜像素网格(奇数，中心像素落在注视点上)
YX = np.indices((N, N)).astype(float); Y, X = YX[0] - (N - 1) / 2, YX[1] - (N - 1) / 2


def triangle_image(direction=+1, size=22, cy=-14, cx=0, with_ring=True):
    """等腰三角形，direction=+1 尖朝右，-1 尖朝左。cy<0 表示在注视圆上方。返回 0/1 图。"""
    h = size * np.sqrt(3) / 2
    ax, ay = cx + direction * 2 * h / 3, cy
    bx = cx - direction * h / 3; b1y, b2y = cy - size / 2, cy + size / 2
    P = np.array([[ax, ay], [bx, b1y], [bx, b2y]])
    def edge(p, q, x, y): return (q[0] - p[0]) * (y - p[1]) - (q[1] - p[1]) * (x - p[0])
    s1, s2, s3 = edge(P[0], P[1], X, Y), edge(P[1], P[2], X, Y), edge(P[2], P[0], X, Y)
    inside = ((s1 >= 0) & (s2 >= 0) & (s3 >= 0)) | ((s1 <= 0) & (s2 <= 0) & (s3 <= 0))
    img = inside.astype(float)
    circ = ((X - cx) ** 2 + (Y - (cy + 24)) ** 2 <= 5 ** 2) & ((X - cx) ** 2 + (Y - (cy + 24)) ** 2
     >= 3 ** 2)
    if with_ring: img[circ] = 1.0
    return img, P


def lgn(img, sc=1.0, ss=2.5):
```

```
    """视网膜神经节细胞/LGN: 中心环绕 DoG, 分 on/off 两路。"""
    dog = gaussian_filter(img, sc) - gaussian_filter(img, ss)
    return np.maximum(dog, 0), np.maximum(-dog, 0)


def gabor_bank(n_orient=8, lam=7.0, sigma=2.5, gamma=0.6, size=15):
    ks = []; r = size // 2; yy, xx = np.mgrid[-r:r + 1, -r:r + 1].astype(float)
    for k in range(n_orient):
        th = np.pi * k / n_orient
        xr = xx * np.cos(th) + yy * np.sin(th); yr = -xx * np.sin(th) + yy * np.cos(th)
        env = np.exp(-(xr ** 2 + gamma ** 2 * yr ** 2) / (2 * sigma ** 2))
        ke = env * np.cos(2 * np.pi * xr / lam); ko = env * np.sin(2 * np.pi * xr / lam)
        ke -= ke.mean(); ks.append((ke, ko, th))
    return ks


def v1(on, off, bank):
    """V1 简单细胞取 LGN on-off 差为输入, 复杂细胞取正交对能量。返回 (n_orient, N, N) 能量图。"""
    from scipy.signal import fftconvolve
    inp = on - off; out = []
    for ke, ko, th in bank:
        e = fftconvolve(inp, ke, mode='same'); o = fftconvolve(inp, ko, mode='same'); out.append(np
    .sqrt(e ** 2 + o ** 2))
    out = np.array(out); return out / (out.max() + 1e-9)


def polar_shift(m, rho, phi):
    """把响应图沿 phi+pi 方向平移 rho, 即把位于 (rho,phi) 处的响应搬到中心。图像 y 向下, phi 以数学方
    向定义。"""
    dx, dy = -rho * np.cos(phi), rho * np.sin(phi)   # y 轴向下所以取反
    return ndshift(m, (dy, dx), order=1, mode='constant')


def vertex_units(v1maps, bank, aperture_deg=60, rho=5.0, sigma_blur=1.0):
    """V2/V4 顶点单元: 张角 aperture 的顶点, 由两条边的方向能量沿各自边方向向外移 rho 后取几何平均。
    返回字典 (orientation_deg of bisector) -> map。方向按顶点指向(平分线朝向)分 12 档。"""
    n = len(bank); ths = np.array([b[2] for b in bank]); out = {}
    half = np.deg2rad(aperture_deg / 2)
    for k in range(12):
        bis = np.deg2rad(30 * k)                      # 顶点指向
        e1, e2 = bis + np.pi - half, bis + np.pi + half   # 两条边从顶点出发的方向
        maps = []
        for e in (e1, e2):
            # 边的方向作为 Gabor 取向(模 pi), 取最近的方向通道
            o = (e % np.pi); j = int(np.argmin(np.abs(np.angle(np.exp(1j * 2 * (ths - o))))))
            m = gaussian_filter(v1maps[j], sigma_blur)
            maps.append(polar_shift(m, rho, e))       # 边在顶点的 e 方向 rho 处
        out[30 * k] = np.sqrt(maps[0] * maps[1])
    return out


def configure_scosfire(vmaps, center, n_keep=3, t2=0.5):
    """按原型形状配置 S-COSFIRE: 选出响应最强的 n_keep 个顶点(类型, rho, phi)。center=(cy,cx)。"""
    stack = np.array([vmaps[k] for k in sorted(vmaps)]); keys = sorted(vmaps)
    mx = stack.max(0); loc = (mx == maximum_filter(mx, 7)) & (mx > t2 * mx.max())
    ys, xs = np.where(loc); order = np.argsort(-mx[ys, xs])[:n_keep]; tuples = []
    for i in order:
        y, x = ys[i], xs[i]; k = keys[int(np.argmax(stack[:, y, x]))]
        dy, dx = y - center[0], x - center[1]; rho = np.hypot(dx, dy); phi = np.arctan2(-dy, dx)
    # y 向下
        tuples.append(dict(vertex_dir=int(k), rho=float(rho), phi=float(phi)))
    return tuples


def scosfire_response(vmaps, tuples, sigma0=1.0, alpha=0.1):
    """S-COSFIRE 输出: 模糊(sigma=sigma0+alpha*rho)并平移后的顶点响应的加权几何平均。"""
```

```
    rmax = max(t['rho'] for t in tuples) + 1e-9; sig_p = np.sqrt(-rmax ** 2 / (2 * np.log(0.5)))
    acc = None; wsum = 0
    for t in tuples:
        m = gaussian_filter(vmaps[t['vertex_dir']], sigma0 + alpha * t['rho'])
        m = polar_shift(m, t['rho'], t['phi']); w = np.exp(-t['rho'] ** 2 / (2 * sig_p ** 2)); wsum
     += w
        acc = np.log(m + 1e-6) * w if acc is None else acc + np.log(m + 1e-6) * w
    return np.exp(acc / wsum)


def reflect_tuples(tuples):
    """关于竖直轴反射: 顶点指向 theta -> 180-theta, 位置 phi -> pi-phi。"""
    out = []
    for t in tuples:
        out.append(dict(vertex_dir=int((180 - t['vertex_dir']) % 360), rho=t['rho'], phi=float(np.
    pi - t['phi'])))
    return out


def hemisphere_drive(v1maps, vmaps):
    """把视网膜坐标的响应分到两半球(左视野->右半球)。返回 dict 层 -> (left_hemi, right_hemi) 总活动。
    """
    left_field = X < 0; right_field = X > 0
    res = {}
    v1sum = v1maps.sum(0); res['V1'] = (float(v1sum[right_field].sum()), float(v1sum[left_field].
    sum()))
    vs = np.array([vmaps[k] for k in vmaps]).max(0); res['V4'] = (float(vs[right_field].sum()),
    float(vs[left_field].sum()))
    return res


def run_pipeline():
    bank = gabor_bank(); out = {}
    for d, lab in [(+1, 'R'), (-1, 'L')]:
        img, P = triangle_image(d); on, off = lgn(img); v = v1(on, off, bank); vm = vertex_units(v,
     bank)
        out[lab] = dict(img=img, P=P, on=on, off=off, v1=v, vmaps=vm, drive=hemisphere_drive(v, vm)
    )
    # 用只含三角形的原型图配置形状单元(不含注视环), 反射得左三角单元
    cy, cx = int((N - 1) / 2 - 14), int((N - 1) / 2)
    img0, _ = triangle_image(+1, with_ring=False); on0, off0 = lgn(img0); v0 = v1(on0, off0, bank);
     vm0 = vertex_units(v0, bank)
    tR = configure_scosfire(vm0, (cy, cx)); tL = reflect_tuples(tR)
    for lab in 'RL':
        out[lab]['S_R'] = scosfire_response(out[lab]['vmaps'], tR); out[lab]['S_L'] =
    scosfire_response(out[lab]['vmaps'], tL)
    out['tuples_R'] = tR; out['tuples_L'] = tL
    return out


if __name__ == '__main__':
    import matplotlib; matplotlib.use('Agg'); import matplotlib.pyplot as plt, json
    from common import FIGS, OUT
    o = run_pipeline()
    fig, ax = plt.subplots(2, 6, figsize=(20, 7))
    for r, lab in enumerate('RL'):
        d = o[lab]
        ax[r, 0].imshow(d['img'], cmap='gray'); ax[r, 0].set_title(f'stimulus {lab}')
        ax[r, 1].imshow(d['on'] - d['off'], cmap='RdBu_r'); ax[r, 1].set_title('LGN on-off')
        ax[r, 2].imshow(d['v1'].max(0), cmap='magma'); ax[r, 2].set_title('V1 orientation energy (
    max)')
        ax[r, 3].imshow(np.argmax(d['v1'], 0) * (d['v1'].max(0) > 0.3), cmap='hsv'); ax[r, 3].
    set_title('V1 preferred orientation')
        vm = np.array([d['vmaps'][k] for k in sorted(d['vmaps'])]); ax[r, 4].imshow(vm.max(0), cmap
    ='magma'); ax[r, 4].set_title('V4 vertex units (max)')
```

```
    ax[r, 5].imshow(np.stack([d['S_R'] / (o['R']['S_R'].max() + 1e-9), d['S_L'] / (o['L']['S_L
'].max() + 1e-9), np.zeros_like(d['S_R'])], -1).clip(0, 1)); ax[r, 5].set_title('IT shape units
: red=R-selective, green=L-selective')
    for a in ax[r]: a.axis('off')
fig.tight_layout(); fig.savefig(f'{FIGS}/q2_visual_pipeline.png', dpi=120)
summ = {lab: dict(drive=o[lab]['drive'], S_R_max=float(o[lab]['S_R'].max()), S_L_max=float(o[
lab]['S_L'].max())) for lab in 'RL'}
summ['tuples_R'] = o['tuples_R']; summ['tuples_L'] = o['tuples_L']
print(json.dumps(summ, indent=1)); json.dump(summ, open(f'{OUT}/q2_visual.json', 'w'), indent
=1)
```

q2_dynamics.py

```
# 本次重跑的稳定性与有限网络选解规则由 OpenAI Codex 辅助补充。
# 本程序及代码是在人工智能工具辅助下完成的：DeepSeek，模型 DeepSeek-V4，
# 开发机构 杭州深度求索人工智能基础技术研究有限公司，版本发布日期 2026 年 4 月 24 日。
"""问题二 第二段：皮层群体动力学 + 头皮正演 + 与实测 ERP 拟合。

一群 QIF 神经元用 Montbrio-Pazo-Roxin 2015 的精确平均场写成 (r, v) 两个方程；
突触按 Byrne 2017 用 alpha 函数，分快兴奋与慢抑制两条；头皮源取净突触后电流而不是膜电位，
膜电位变化太慢给不出 ERP 的双相形状。序参数 Z 由共形映射 Z=(1-conj(W))/(1+conj(W)) 得到。

拟合分两步。第一步只拟合左右合并后的共模响应，两半球对称所以只需积分一群，参数 11 个。
第二步拿拟合好的参数，把视觉模型给出的不对称驱动加上去，两半球用 Jx 耦合，算出模型预测的
F3 减 F4 偏侧化，再和问题二实测的检出限比，看模型是不是自洽地解释了那个零结果。
"""
import numpy as np, json, sys
import matplotlib; matplotlib.use('Agg'); import matplotlib.pyplot as plt
from numba import njit
from scipy.optimize import least_squares
from scipy.signal import butter, sosfiltfilt
from common import FS, FIGS, OUT, CH

# 固定步长 0.1 ms；重跑后的有限网络检验另存 q2_check 与 q2_spiking 输出。
DT = 0.1e-3
# 从 -1.3 s 起积分，前 1 s 空跑让系统落到稳态，基线只取提示前 0.3 s。
# 不空跑的话，某些参数下初值的弛豫过渡会跨进基线乃至刺激期，被拟合当成诱发响应来用。
T0, T1 = -1.30, 1.05
NT = int((T1 - T0) / DT); tm = T0 + np.arange(NT) * DT
CUE = ((tm >= 0) & (tm < 0.2)).astype(float)
NB = int(-T0 / DT)                    # 提示出现的下标
B0 = int((-0.30 - T0) / DT)           # 基线起点，提示前 0.3 s
DELTA = 1.0


def alpha_kernel(tau):
    n = int(8 * tau / DT) + 2
    t = np.arange(n) * DT
    k = t / tau ** 2 * np.exp(-t / tau)
    return k / (k.sum() + 1e-12)


def drive_of(A, tau_d, delay):
    """外侧膝状体核送上来的前馈驱动：提示方波过一个 alpha 核再延迟。tau_d 与 delay 都按毫秒给。"""
    k = alpha_kernel(tau_d * 1e-3)
    x = np.roll(CUE, int(delay * 1e-3 / DT))
    return A * np.convolve(x, k, mode='full')[:NT]


@njit(cache=True)
def _one(tau_e, tau_i, Je, Ji, eta, tau_m, I, NT, DT, DELTA):
    ae, ai = 1.0 / tau_e, 1.0 / tau_i
    r = np.empty(NT); v = np.empty(NT); se = np.empty(NT); si = np.empty(NT)
    rr, vv, e, de, s, ds = 0.02, -1.0, 0.0, 0.0, 0.0, 0.0
    c1 = DELTA / (np.pi * tau_m)
    for n in range(NT):
```

```
        r[n] = rr; v[n] = vv; se[n] = e; si[n] = s
        rec = Je * e - Ji * s + I[n]
        dr = (c1 + 2.0 * rr * vv) / tau_m
        dv = (vv * vv + eta + tau_m * rec - (np.pi * tau_m * rr) ** 2) / tau_m
        rr += DT * dr
        if rr < 0.0: rr = 0.0
        elif rr > 500.0: rr = 500.0
        vv += DT * dv
        if vv < -80.0: vv = -80.0
        elif vv > 80.0: vv = 80.0
        de += DT * (ae * ae * r[n] - 2.0 * ae * de - ae * ae * e); e += DT * de
        ds += DT * (ai * ai * r[n] - 2.0 * ai * ds - ai * ai * s); s += DT * ds
    return r, v, se, si


@njit(cache=True)
def _pair(tau_e, tau_i, Je, Ji, eta, tau_m, I0, I1, Jx, NT, DT, DELTA):
    ae, ai = 1.0 / tau_e, 1.0 / tau_i
    out = np.empty((2, 4, NT))
    st = np.zeros((2, 6))
    st[:, 0] = 0.02; st[:, 1] = -1.0
    c1 = DELTA / (np.pi * tau_m)
    for n in range(NT):
        e0, e1 = st[0, 2], st[1, 2]
        for h in range(2):
            rr, vv, e, de, s, ds = st[h, 0], st[h, 1], st[h, 2], st[h, 3], st[h, 4], st[h, 5]
            out[h, 0, n] = rr; out[h, 1, n] = vv; out[h, 2, n] = e; out[h, 3, n] = s
            cross = e1 if h == 0 else e0
            rec = Je * e - Ji * s + Jx * cross + (I0[n] if h == 0 else I1[n])
            rr2 = rr + DT * (c1 + 2.0 * rr * vv) / tau_m
            vv2 = vv + DT * (vv * vv + eta + tau_m * rec - (np.pi * tau_m * rr) ** 2) / tau_m
            rr2 = min(max(rr2, 0.0), 500.0); vv2 = min(max(vv2, -80.0), 80.0)
            de += DT * (ae * ae * rr - 2.0 * ae * de - ae * ae * e); e += DT * de
            ds += DT * (ai * ai * rr - 2.0 * ai * ds - ai * ai * s); s += DT * ds
            st[h, 0] = rr2; st[h, 1] = vv2; st[h, 2] = e; st[h, 3] = de; st[h, 4] = s; st[h, 5] =
    ds
    return out


def run_one(p, I):
    """单群积分。p=(tau_e,tau_i,Je,Ji,eta,tau_m)。返回 r, v, se, si。"""
    return _one(p[0], p[1], p[2], p[3], p[4], p[5], np.ascontiguousarray(I), NT, DT, DELTA)


def run_pair(p, I0, I1, Jx):
    """两群通过 Jx 互相耦合，返回每一群的 r v se si。"""
    o = _pair(p[0], p[1], p[2], p[3], p[4], p[5], np.ascontiguousarray(I0),
              np.ascontiguousarray(I1), Jx, NT, DT, DELTA)
    return o[0], o[1]


def kuramoto_Z(r, v, tau_m):
    W = np.pi * tau_m * r + 1j * v
    return (1 - np.conj(W)) / (1 + np.conj(W))


# ---------------- 头模型：均匀球，半径 9.2 cm，两侧前额叶径向偶极 ----------------
def sph(theta_deg, phi_deg, R=9.2):
    t, p = np.deg2rad(theta_deg), np.deg2rad(phi_deg)
    return R * np.array([np.sin(t) * np.cos(p), np.sin(t) * np.sin(p), np.cos(t)])


ELEC = {'F3': sph(50, 140), 'Fz': sph(36, 90), 'F4': sph(50, 40)}
SRC = {'L': sph(45, 125, R=7.0), 'R': sph(45, 55, R=7.0)}
LF = np.array([[(SRC[h] / np.linalg.norm(SRC[h])) @ (ELEC[e] - SRC[h])
                / (4 * np.pi * np.linalg.norm(ELEC[e] - SRC[h]) ** 3) for h in 'LR'] for e in CH])
```

```
LF = LF / np.abs(LF).max()
SOS_HP = butter(2, 1.0 / (FS / 2), 'high', output='sos')
SOS_LP = butter(4, 20 / (FS / 2), 'low', output='sos')

# 整体增益 gain 是一个纯线性系数，留在参数里会和驱动幅度 A 简并，所以把它从向量里拿掉，
# 每次求值时用一次最小二乘闭式解出来。剩下十个参数都有各自的生理含义。
NAMES = ['A', 'tau_d_ms', 'delay_ms', 'tau_e_ms', 'tau_i_ms', 'Je', 'Ji', 'eta', 'wi', 'tau_m_ms']
LO = [0.5, 3.0, 20.0, 2.0, 10.0, 0.0, 0.0, -10.0, 0.0, 6.0]
HI = [400.0, 40.0, 170.0, 25.0, 150.0, 30.0, 30.0, 4.0, 4.0, 30.0]
P0 = [15.0, 12.0, 95.0, 6.0, 32.0, 8.0, 6.0, -2.0, 1.0, 12.0]


def split(q):
    A, tau_d, delay, tau_e, tau_i, Je, Ji, eta, wi, tau_m = q
    dyn = (tau_e * 1e-3, tau_i * 1e-3, Je, Ji, eta, tau_m * 1e-3)
    return dyn, drive_of(A, tau_d, delay), wi


def scalp_common(q, t_out, gain=1.0):
    """共模：两半球同样被驱动，只需积分一群。"""
    dyn, I, wi = split(q)
    r, v, se, si = run_one(dyn, I)
    src = se - wi * si; src -= src[B0:NB].mean()
    eeg = gain * (LF.sum(1)[:, None] * src[None, :])
    y = np.array([np.interp(t_out, tm, ch) for ch in eeg])
    y = sosfiltfilt(SOS_LP, sosfiltfilt(SOS_HP, y, axis=1), axis=1)
    return y - y[:, (t_out >= -0.2) & (t_out < 0)].mean(1, keepdims=True), (r, v, se, si, dyn[-1])


def scalp_lateral(q, D, Jx, t_out, gain=1.0):
    """不对称驱动下的两半球，返回三通道头皮电位。D=[左半球权重，右半球权重]。"""
    dyn, I, wi = split(q)
    o0, o1 = run_pair(dyn, D[0] * I, D[1] * I, Jx)
    s0 = o0[2] - wi * o0[3]; s1 = o1[2] - wi * o1[3]
    s0 -= s0[B0:NB].mean(); s1 -= s1[B0:NB].mean()
    eeg = gain * (LF @ np.vstack([s0, s1]))
    y = np.array([np.interp(t_out, tm, ch) for ch in eeg])
    y = sosfiltfilt(SOS_LP, sosfiltfilt(SOS_HP, y, axis=1), axis=1)
    return y - y[:, (t_out >= -0.2) & (t_out < 0)].mean(1, keepdims=True)


if __name__ == '__main__':
    task = sys.argv[1] if len(sys.argv) > 1 else 'Task1'
    nstart = int(sys.argv[2]) if len(sys.argv) > 2 else 8
    pooled = json.load(open(f'{OUT}/q1_pooled.json', encoding='utf-8'))
    vis = json.load(open(f'{OUT}/q2_visual.json', encoding='utf-8'))
    t_out = np.array(pooled['t']); P = pooled['pooled'][task]
    # 取提示诱发的那一段，并把提示前 0.3 s 也算进残差，模型在提示前若不静息会被罚
    m = (t_out >= -0.3) & (t_out <= 0.65)
    Y = np.array([P[c]['all'] for c in CH])[:, m]
    SE = np.array([P[c]['all_se'] for c in CH])[:, m] + 0.25

    def best_gain(q):
        """整体增益的闭式最小二乘解，三通道共用一个。"""
        yh, _ = scalp_common(q, t_out)
        a, b = yh[:, m] / SE, Y / SE
        den = float((a * a).sum())
        # 偶极子方向由锥体细胞的朝向定死，增益只许非负，不允许靠翻转极性去凑波形
        return max(0.0, float((a * b).sum() / den)) if den > 1e-12 and np.isfinite(den) else 0.0

    def resid(q):
        yh, _ = scalp_common(q, t_out, best_gain(q))
        v = ((yh[:, m] - Y) / SE).ravel()
        return np.where(np.isfinite(v), v, 1e3)

    # 起点：默认值，0.5 ms 步长时两项目各自的最优解，其余随机扰动
```

```
KNOWN = [[400.0, 32.0, 136.9, 6.7, 116.7, 20.2, 30.0, -10.0, 1.94, 29.5],
         [7.0, 40.0, 55.5, 25.0, 51.5, 20.9, 12.5, -2.07, 1.26, 9.6]]
best, rng = None, np.random.default_rng(0)
candidates = []
for s in range(nstart):
    if s == 0: q0 = np.array(P0)
    elif s <= len(KNOWN): q0 = np.clip(np.array(KNOWN[s - 1]), LO, HI)
    else: q0 = np.clip(np.array(P0) * rng.uniform(0.5, 1.9, len(P0)), LO, HI)
    try:
        rr = least_squares(resid, q0, bounds=(LO, HI), max_nfev=400, x_scale='jac')
    except Exception as ex:
        print('start', s, 'fail', ex, flush=True); continue
    print('start', s, 'cost', round(float(rr.cost), 1), flush=True)
    candidates.append((s, rr))
    if best is None or rr.cost < best.cost: best = rr
# 项目一的内在参数还用于问题三，必须先通过论文声明的稳定性和有限网络检验。
# 固定阈值先于检验确定，不按结果调整：5% 扰动形状相关中位数 >=0.9，网络源项误差 <=10%。
selection = []
if task == 'Task1':
    from q2_spiking import simulate
    chosen = None
    mm = tm > -0.3
    for start_id, candidate in sorted(candidates, key=lambda z: z[1].cost):
        qq = candidate.x
        _, (_, _, ee, ii, _) = scalp_common(qq, t_out)
        source = ee - qq[8] * ii
        perturb_rng = np.random.default_rng(0)
        correlations = []
        for _ in range(30):
            pp = qq * (1 + 0.05 * perturb_rng.standard_normal(len(qq)))
            _, (_, _, ep, ip, _) = scalp_common(pp, t_out)
            correlations.append(float(np.corrcoef((ep - pp[8] * ip)[mm], source[mm])[0, 1]))
        median = float(np.median(correlations))
        row = dict(start=start_id, cost=float(candidate.cost), robust_median=median,
                   robust_min=float(np.min(correlations)), net_relerr=None, accepted=False)
        if np.isfinite(median) and median >= 0.9:
            (_, _, _, en, inn), _, _, wi = simulate(4000, qq)
            sn = en - wi * inn
            sm = source.copy()
            sn -= sn[B0:NB].mean(); sm -= sm[B0:NB].mean()
            error = float(np.sqrt(np.mean((sn[mm] - sm[mm]) ** 2)) / np.std(sm[mm]))
            row['net_relerr'] = error
            row['accepted'] = bool(np.isfinite(error) and error <= 0.1)
        selection.append(row)
        print('selection', row, flush=True)
        if row['accepted']:
            chosen = candidate
            break
    if chosen is None:
        raise RuntimeError('No Task1 solution passed stability and finite-network checks.')
    best = chosen
q = best.x; gain = best_gain(q)
yh, (r, v, se, si, tau_m) = scalp_common(q, t_out, gain)
Z = kuramoto_Z(r, v, tau_m)
res = dict(task=task, params={n: round(float(x), 3) for n, x in zip(NAMES, q)},
           gain=round(gain, 4), cost=float(best.cost), selection=selection)
res['r2'] = {c: round(float(1 - ((yh[i, m] - Y[i]) ** 2).sum() / ((Y[i] - Y[i].mean()) ** 2).
sum()), 3)
             for i, c in enumerate(CH)}
res['peak_lat_s'] = {c: round(float(t_out[m][np.argmax(np.abs(yh[i, m]))]), 3) for i, c in
enumerate(CH)}
res['data_peak_lat_s'] = {c: round(float(t_out[m][np.argmax(np.abs(Y[i]))]), 3) for i, c in
enumerate(CH)}
res['Z_baseline'] = round(float(np.abs(Z[B0:NB]).mean()), 3)
res['Z_min'] = round(float(np.abs(Z[NB:]).min()), 3)
res['Z_max'] = round(float(np.abs(Z[NB:]).max()), 3)
```

```
    res['Z_min_t'] = round(float(tm[NB + int(np.argmin(np.abs(Z[NB:])))]), 3)
    res['Z_max_t'] = round(float(tm[NB + int(np.argmax(np.abs(Z[NB:])))]), 3)
    res['r_peak'] = round(float(r.max()), 2)
    # 第二步：不对称驱动下模型预测的偏侧化
    Dv = {lab: np.array(vis[lab]['drive']['V1']) for lab in 'RL'}
    nrm = max(Dv['R'].max(), Dv['L'].max())
    D = {lab: (Dv[lab] / nrm) for lab in 'RL'}
    w = (t_out >= 0.3) & (t_out <= 0.5)
    lat_pred = {}
    for Jx in (0.0, 0.5, 1.0):
        yl = scalp_lateral(q, D['L'], Jx, t_out, gain); yr = scalp_lateral(q, D['R'], Jx, t_out,
    gain)
        dl = (yl[1] - yl[2])[w].mean(); dr = (yr[1] - yr[2])[w].mean()
        lat_pred[f'Jx={Jx}'] = round(float(dr - dl), 3)
        print(f'Jx={Jx}: 模型预测 F3-F4 的右减左 = {dr-dl:+.3f} uV', flush=True)
    res['model_lateralization_uV'] = lat_pred
    res['drive_ratio_L_over_R'] = round(float(D['L'][0] / D['L'][1]), 3)
    dec = json.load(open(f'{OUT}/q2_decode.json', encoding='utf-8'))
    res['min_detectable_uV'] = dec['detectability'][task]['min_detectable_uV']
    res['observed_uV'] = dec['detectability'][task]['observed_uV']
    print(f"实测检出限 {res['min_detectable_uV']} uV, 实测差 {res['observed_uV']} uV", flush=True)

    fig, ax = plt.subplots(1, 4, figsize=(19, 4))
    for i, c in enumerate(CH):
        ax[0].plot(t_out, Y[i] if False else P[c]['all'], lw=1.0, alpha=.6, label=f'data {c}')
        ax[0].plot(t_out, yh[i], ls='--', lw=1.6, color=ax[0].lines[-1].get_color(), label=f'model
    {c}')
    ax[0].set_xlim(-0.2, 0.8); ax[0].axvline(0, color='k', lw=.5); ax[0].legend(fontsize=7)
    ax[0].set_title(f'{task}  共模 ERP,  $R^2$ = ' + ', '.join(f'{v:.2f}' for v in res['r2'].values
    ()))
    ax[1].plot(tm, r, color='#5B8DB8'); ax[1].set_title('firing rate $r$'); ax[1].axvspan(0, .2,
    color='y', alpha=.15)
    ax[2].plot(tm, np.abs(Z), color='#D98C7A'); ax[2].set_title('Kuramoto $|Z|$'); ax[2].axvspan(0,
     .2, color='y', alpha=.15)
    ax[3].plot(tm, se, label='$s_e$'); ax[3].plot(tm, si, label='$s_i$')
    ax[3].plot(tm, se - q[8] * si, 'k', lw=1.4, label='net'); ax[3].legend(fontsize=7)
    ax[3].set_title('synaptic currents'); ax[3].axvspan(0, .2, color='y', alpha=.15)
    for a in ax[1:]: a.set_xlim(-0.2, 0.8)
    fig.tight_layout(); fig.savefig(f'{FIGS}/q2_fit_{task}.png', dpi=120)
    print(json.dumps(res, indent=1, ensure_ascii=False))
    json.dump(res, open(f'{OUT}/q2_fit_{task}.json', 'w', encoding='utf-8'), indent=1, ensure_ascii
    =False)
    np.save(f'{OUT}/q2_fit_{task}_params.npy', q)
```

q2_compare.py

```
# 本程序及代码是在人工智能工具辅助下完成的：DeepSeek，模型 DeepSeek-V4，
# 开发机构 杭州深度求索人工智能基础技术研究有限公司，版本发布日期 2026 年 4 月 24 日。
"""问题二 多算法对比：同一个目标函数，比较五种优化器拟合项目一 ERP 的效果。

目标函数与 q2_dynamics 相同：平均场加头皮正演，与实测三通道 ERP 的加权残差平方和，增益闭式解出。
每种算法各跑 10 次，每次随机起点或随机种子不同，函数调用预算相近。
指标：最优代价、10 次代价的中位数、命中率（代价不超过全体最好值的 1.01 倍）、单次平均用时。
"""
import numpy as np, json, time, sys
from scipy.optimize import least_squares, minimize, differential_evolution, dual_annealing
import q2_dynamics as Q
from common import OUT, CH

task = sys.argv[1] if len(sys.argv) > 1 else 'Task1'
NRUN = int(sys.argv[2]) if len(sys.argv) > 2 else 10
pooled = json.load(open(f'{OUT}/q1_pooled.json', encoding='utf-8'))
t_out = np.array(pooled['t']); P = pooled['pooled'][task]
m = (t_out >= -0.3) & (t_out <= 0.65)
Y = np.array([P[c]['all'] for c in CH])[:, m]
SE = np.array([P[c]['all_se'] for c in CH])[:, m] + 0.25
```

```
LO, HI = np.array(Q.LO), np.array(Q.HI)
NCALL = [0]


def resid(q):
    NCALL[0] += 1
    yh, _ = Q.scalp_common(q, t_out)
    a, b = yh[:, m] / SE, Y / SE; den = float((a * a).sum())
    g = max(0.0, float((a * b).sum() / den)) if den > 1e-12 and np.isfinite(den) else 0.0
    v = (g * a - b).ravel()
    return np.where(np.isfinite(v), v, 1e3)


def cost(q):
    q = np.clip(q, LO, HI); r = resid(q); return 0.5 * float(r @ r)


def rand_start(rng):
    return LO + (HI - LO) * rng.random(len(LO))


def multistart_trf(rng, k=8):
    best = None
    for _ in range(k):
        r = least_squares(resid, rand_start(rng), bounds=(LO, HI), max_nfev=150, x_scale='jac')
        if best is None or r.cost < best.cost: best = r
    return best.x


METHODS = {
    '单起点 TRF': lambda rng: least_squares(resid, rand_start(rng), bounds=(LO, HI), max_nfev=150,
    x_scale='jac').x,
    '多起点 TRF': multistart_trf,
    'Nelder-Mead': lambda rng: minimize(cost, rand_start(rng), method='Nelder-Mead',
                                        options=dict(maxfev=2500, xatol=1e-4, fatol=1e-6)).x,
    'L-BFGS-B': lambda rng: minimize(cost, rand_start(rng), method='L-BFGS-B', bounds=list(zip(LO,
    HI)),
                                     options=dict(maxfun=2500)).x,
    '差分进化': lambda rng: differential_evolution(cost, list(zip(LO, HI)), popsize=8, maxiter=30,
    tol=1e-8,
                                                   seed=int(rng.integers(1 << 30)), polish=False).x,
    '双重退火': lambda rng: dual_annealing(cost, list(zip(LO, HI)), maxfun=2500,
                                       seed=int(rng.integers(1 << 30)), no_local_search=True).x,
}

if __name__ == '__main__':
    import os
    only = sys.argv[3].split(',') if len(sys.argv) > 3 else list(METHODS)
    rawf = f'{OUT}/q2_compare_{task}_raw.json'
    raw = json.load(open(rawf, encoding='utf-8')) if os.path.exists(rawf) else {}
    for name, run in METHODS.items():
        if name not in only: continue
        rng = np.random.default_rng(123); cs, ts, ns = [], [], []
        for k in range(NRUN):
            NCALL[0] = 0; t0 = time.time()
            try: x = run(rng); c = cost(x)
            except Exception as ex: c = np.inf; print(name, 'fail', ex)
            ts.append(time.time() - t0); cs.append(c); ns.append(NCALL[0])
            print(f'{name} run {k} cost {c:.1f} calls {NCALL[0]} {ts[-1]:.1f}s', flush=True)
        raw[name] = dict(cost=[float(c) for c in cs], sec=ts, calls=ns)
        json.dump(raw, open(rawf, 'w', encoding='utf-8'), ensure_ascii=False)
    raw = {k: raw[k] for k in METHODS if k in raw}
    best = min(min(v['cost']) for v in raw.values())
    out = dict(task=task, nrun=NRUN, best_cost=round(best, 2), methods={})
    for name, v in raw.items():
        c = np.array(v['cost'])
```

```
        out['methods'][name] = dict(best=round(float(c.min()), 2), median=round(float(np.median(c))
    , 2),
                                    hit=int((c <= best * 1.01).sum()), sec_mean=round(float(np.mean
    (v['sec'])), 1),
                                    calls_mean=int(np.mean(v['calls'])))
        print(name, out['methods'][name], flush=True)
    json.dump(out, open(f'{OUT}/q2_compare_{task}.json', 'w', encoding='utf-8'), indent=1,
    ensure_ascii=False)
```

q2_refind.py

```
# 本程序及代码是在人工智能工具辅助下完成的：DeepSeek，模型 DeepSeek-V4，
# 开发机构 杭州深度求索人工智能基础技术研究有限公司，版本发布日期 2026 年 4 月 24 日。
"""复现多起点 TRF 在优化器对比中找到的更低代价解，并与现用解比较。
按 q2_compare 的同一随机序列重放多起点 TRF 的前 7 次运行，记下每次的解。"""
import numpy as np, json
from scipy.optimize import least_squares
import q2_compare as C
import q2_dynamics as Q
from common import OUT, CH

rng = np.random.default_rng(123)
for _ in range(10):                  # 单起点 TRF 那一项先消耗了同一个 rng? 不，每种方法各自新建 rng
    break
rng = np.random.default_rng(123)
sols = []
for k in range(7):
    best = None
    for _ in range(8):
        r = least_squares(C.resid, C.rand_start(rng), bounds=(C.LO, C.HI), max_nfev=150, x_scale='
    jac')
        if best is None or r.cost < best.cost: best = r
    sols.append((float(best.cost), best.x.tolist())); print(k, round(best.cost, 2), np.round(best.x
    , 2), flush=True)
json.dump(sols, open(f'{OUT}/q2_refind.json', 'w', encoding='utf-8'))
```

q2_candidates.py

```
# 本程序及代码是在人工智能工具辅助下完成的：DeepSeek，模型 DeepSeek-V4，
# 开发机构 杭州深度求索人工智能基础技术研究有限公司，版本发布日期 2026 年 4 月 24 日。
"""比较项目一的几组候选解：诱发段 R2、峰潜伏期、峰值发放率、参数扰动 5% 后源项形状的稳定性、
与 4000 个脉冲神经元直接模拟的一致性。候选解来自现用拟合与优化器对比中代价约 274 的几组。"""
import numpy as np, json
import q2_dynamics as Q
from q2_spiking import simulate
from common import OUT, CH

P = json.load(open(f'{OUT}/q1_pooled.json', encoding='utf-8')); t = np.array(P['t']); PP = P['
    pooled']['Task1']
m = (t >= -0.1) & (t <= 0.65); mf = (t >= -0.3) & (t <= 0.65)
Y = np.array([PP[c]['all'] for c in CH]); SE = np.array([PP[c]['all_se'] for c in CH]) + 0.25


def gain(q, mask):
    yh, _ = Q.scalp_common(q, t); a, b = yh[:, mask] / SE[:, mask], Y[:, mask] / SE[:, mask]
    return max(0.0, float((a * b).sum() / (a * a).sum()))


def evaluate(name, q):
    q = np.array(q); g = gain(q, mf)
    yh, (r, v, se, si, tau_m) = Q.scalp_common(q, t, g)
    res = ((yh[:, mf] - Y[:, mf]) / SE[:, mf]).ravel(); cost = 0.5 * float(res @ res)
    r2 = [round(float(1 - ((yh[i, m] - Y[i, m]) ** 2).sum() / ((Y[i, m] - Y[i, m].mean()) ** 2).sum
    ()), 3) for i in range(3)]
    src = se - q[8] * si; mm = Q.tm > -0.3; rng = np.random.default_rng(0); cs = []
    for _ in range(30):
```

```
        qq = q * (1 + 0.05 * rng.standard_normal(len(q))); _, (r2_, v2, s2, i2, _) = Q.scalp_common
    (qq, t)
        c = s2 - qq[8] * i2; cs.append(float(np.corrcoef(c[mm], src[mm])[0, 1]))
    (rn, vn, Zn, sen, sin), dyn, I, wi = simulate(4000, q)
    a_ = sen - wi * sin; b_ = se - q[8] * si
    a_ = a_ - a_[Q.B0:Q.NB].mean(); b_ = b_ - b_[Q.B0:Q.NB].mean()
    net_err = float(np.sqrt(np.mean((a_[mm] - b_[mm]) ** 2)) / np.std(b_[mm]))
    Z = np.abs(Q.kuramoto_Z(r, v, tau_m))
    out = dict(name=name, cost=round(cost, 1), r2=r2, peak=round(float(t[m][np.argmax(np.abs(yh[1,
    m]))]), 3),
               r_base=round(float(r[Q.B0:Q.NB].mean()), 2), r_peak=round(float(r[Q.NB:].max()), 1),
               Z_base=round(float(Z[Q.B0:Q.NB].mean()), 3), Z_min=round(float(Z[Q.NB:].min()), 3),
               robust_median=round(float(np.median(cs)), 3), robust_min=round(float(np.min(cs)), 3)
    ,
               net_relerr=round(net_err, 3), params=[round(float(x), 2) for x in q])
    print(out, flush=True)
    return out


if __name__ == '__main__':
    cands = [('现用解', np.load(f'{OUT}/q2_fit_Task1_params.npy').tolist())]
    for k, (c, x) in enumerate(json.load(open(f'{OUT}/q2_refind.json', encoding='utf-8'))):
        if c < 280: cands.append((f'候选{k}', x))
    res = [evaluate(n, q) for n, q in cands]
    json.dump(res, open(f'{OUT}/q2_candidates.json', 'w', encoding='utf-8'), indent=1, ensure_ascii
    =False)
```

q2_spiking.py

```
# 本程序及代码是在人工智能工具辅助下完成的：DeepSeek，模型 DeepSeek-V4，
# 开发机构 杭州深度求索人工智能基础技术研究有限公司，版本发布日期 2026 年 4 月 24 日。
"""问题二 验证：用 N 个 QIF 脉冲神经元直接模拟，与精确平均场对照。

每个神经元 dV/dt = (V^2 + eta_j + tau_m (Je s_e - Ji s_i + I)) / tau_m, V 到 Vp 发放并跳到 -Vp,
其间留 2 tau_m / Vp 的不应期，近似从 Vp 飞到无穷再从负无穷回来的时间。eta_j 取洛伦兹分布，
中心与半宽与平均场相同。突触、驱动、参数全部取问题二项目一的拟合值。
比四样东西：发放率 r，平均膜电位 v（取中位数，即洛伦兹分布的中心），序参数 |Z|，
以及头皮电位的源项 s_e - wi s_i。再换几个 N 看误差是否按 1/sqrt(N) 收敛。
"""
import numpy as np, json, sys
import matplotlib; matplotlib.use('Agg'); import matplotlib.pyplot as plt
from numba import njit
from common import FIGS, OUT
from q2_dynamics import run_one, kuramoto_Z, drive_of, DT, NT, tm, NB, B0


@njit(cache=True)
def network(eta_j, th0, I, tau_e, tau_i, Je, Ji, tau_m, NT, DT, sub):
    """theta 神经元写法，与 QIF 在 V = tan(theta/2) 下等价；theta 越过 pi 记一次发放。
    V^2 项很陡，直接对 V 做欧拉积分会在发放后一步越过阈值，所以改积 theta 并细分子步。"""
    N = len(eta_j); th = th0.copy(); h = DT / sub
    r = np.zeros(NT); vmed = np.zeros(NT); Zr = np.zeros(NT); Zi = np.zeros(NT)
    se = np.zeros(NT); si = np.zeros(NT); Vt = np.zeros(N)
    e = 0.0; de = 0.0; s = 0.0; ds = 0.0
    ae, ai = 1.0 / tau_e, 1.0 / tau_i
    for n in range(NT):
        se[n] = e; si[n] = s
        rec = Je * e - Ji * s + I[n]; nsp = 0
        for k in range(sub):
            for j in range(N):
                c = np.cos(th[j])
                th[j] += h * ((1.0 - c) + (1.0 + c) * (eta_j[j] + tau_m * rec)) / tau_m
                if th[j] > np.pi:
                    th[j] -= 2.0 * np.pi; nsp += 1
        rn = nsp / (N * DT); r[n] = rn
        cr = 0.0; ci = 0.0
```

```
        for j in range(N):
            cr += np.cos(th[j]); ci += np.sin(th[j]); Vt[j] = np.tan(th[j] / 2.0)
        Zr[n] = cr / N; Zi[n] = ci / N; vmed[n] = np.median(Vt)
        de += DT * (ae * ae * rn - 2.0 * ae * de - ae * ae * e); e += DT * de
        ds += DT * (ai * ai * rn - 2.0 * ai * ds - ai * ai * s); s += DT * ds
    return r, vmed, Zr + 1j * Zi, se, si


ETA_CLIP = 200.0


def simulate(N, q, seed=0, sub=None):
    A, tau_d, delay, tau_e, tau_i, Je, Ji, eta, wi, tau_m = q
    tau_e, tau_i, tau_m = tau_e * 1e-3, tau_i * 1e-3, tau_m * 1e-3
    if sub is None:        # 最快的神经元每个子步转角不超过 0.2 弧度
        sub = int(np.ceil(DT * 2 * (ETA_CLIP + abs(eta) + 30) / (0.2 * tau_m)))
    I = drive_of(A, tau_d, delay)
    rng = np.random.default_rng(seed)
    # 洛伦兹分布用分位点均匀取样，比随机抽样的有限规模涨落小；极端尾部截在 ±ETA_CLIP，约占 0.3%
    u = (np.arange(N) + 0.5) / N; eta_j = eta + np.clip(np.tan(np.pi * (u - 0.5)), -ETA_CLIP,
    ETA_CLIP)
    rng.shuffle(eta_j)
    th0 = 2 * np.arctan(-1.0 + 0.1 * np.tan(np.pi * (rng.random(N) - 0.5)))
    return network(eta_j, th0, I, tau_e, tau_i, Je, Ji, tau_m, NT, DT, sub), (tau_e, tau_i, Je, Ji,
     eta, tau_m), I, wi


def smooth(x, ms=2.0):
    k = max(1, int(ms * 1e-3 / DT)); return np.convolve(x, np.ones(k) / k, 'same')


if __name__ == '__main__':
    task = sys.argv[1] if len(sys.argv) > 1 else 'Task1'
    q = np.load(f'{OUT}/q2_fit_{task}_params.npy')
    res = dict(task=task, conv={})
    m = tm > -0.3
    for N in (1000, 4000, 16000):
        (r_n, v_n, Z_n, se_n, si_n), dyn, I, wi = simulate(N, q)
        r, v, se, si = run_one(dyn, I); Z = kuramoto_Z(r, v, dyn[-1])
        src_n = se_n - wi * si_n; src = se - wi * si
        src_n -= src_n[B0:NB].mean(); src -= src[B0:NB].mean()
        rel = lambda a, b: float(np.sqrt(np.mean((a[m] - b[m]) ** 2)) / np.std(b[m]))
        c = dict(corr_r=round(float(np.corrcoef(smooth(r_n)[m], r[m])[0, 1]), 4),
                 corr_v=round(float(np.corrcoef(v_n[m], v[m])[0, 1]), 4),
                 corr_Z=round(float(np.corrcoef(np.abs(Z_n[m]), np.abs(Z[m]))[0, 1]), 4),
                 corr_src=round(float(np.corrcoef(src_n[m], src[m])[0, 1]), 4),
                 relerr_src=round(rel(src_n, src), 4), relerr_r=round(rel(smooth(r_n), r), 4),
                 peak_r_net=round(float(smooth(r_n).max()), 2), peak_r_mf=round(float(r.max()), 2))
        res['conv'][N] = c
        print(N, c, flush=True)
        if N == 16000: keep = (r_n, v_n, Z_n, src_n, r, v, Z, src)
    Ns = np.array(list(res['conv'].keys())); e = np.array([res['conv'][n]['relerr_src'] for n in Ns
    ])
    res['slope_log_err_vs_log_N'] = round(float(np.polyfit(np.log(Ns), np.log(e), 1)[0]), 3)
    print('源项相对误差随 N 的对数斜率', res['slope_log_err_vs_log_N'], '(1/sqrt(N) 对应 -0.5)')
    r_n, v_n, Z_n, src_n, r, v, Z, src = keep
    fig, ax = plt.subplots(4, 1, figsize=(10, 11), sharex=True)
    ax[0].plot(tm, smooth(r_n), 'k', lw=.6, label='network'); ax[0].plot(tm, r, 'r', label='mean
    field'); ax[0].set_ylabel('r')
    ax[1].plot(tm, v_n, 'k', lw=.6); ax[1].plot(tm, v, 'r'); ax[1].set_ylabel('v')
    ax[2].plot(tm, np.abs(Z_n), 'k', lw=.6); ax[2].plot(tm, np.abs(Z), 'r'); ax[2].set_ylabel('|Z
    |')
    ax[3].plot(tm, src_n, 'k', lw=.6); ax[3].plot(tm, src, 'r'); ax[3].set_ylabel('source')
    ax[0].legend(); ax[3].set_xlabel('s'); ax[3].set_xlim(-0.2, 1.0)
    fig.tight_layout(); fig.savefig(f'{FIGS}/q2_spiking_{task}.png', dpi=110)
```

```
    json.dump(res, open(f'{OUT}/q2_spiking_{task}.json', 'w', encoding='utf-8'), indent=1,
    ensure_ascii=False)
```

q2_task2_check.py

```
# 本程序及代码是在人工智能工具辅助下完成的：DeepSeek，模型 DeepSeek-V4，
# 开发机构 杭州深度求索人工智能基础技术研究有限公司，版本发布日期 2026 年 4 月 24 日。
"""项目二现用解的检验，口径与 q2_candidates 相同：诱发段 R2（增益在 -0.3 到 0.65 s 上解出）、
峰潜伏期、参数扰动 5% 后源项形状相关、与 4000 个脉冲神经元直接模拟的相对误差。"""
import numpy as np, json
import q2_dynamics as Q
from q2_spiking import simulate
from common import OUT, CH

P = json.load(open(f'{OUT}/q1_pooled.json', encoding='utf-8')); t = np.array(P['t'])
res = {}
for task in ('Task1', 'Task2'):
    PP = P['pooled'][task]
    m = (t >= -0.1) & (t <= 0.65); mf = (t >= -0.3) & (t <= 0.65)
    Y = np.array([PP[c]['all'] for c in CH]); SE = np.array([PP[c]['all_se'] for c in CH]) + 0.25
    q = np.load(f'{OUT}/q2_fit_{task}_params.npy')
    yh0, _ = Q.scalp_common(q, t); a, b = yh0[:, mf] / SE[:, mf], Y[:, mf] / SE[:, mf]
    g = max(0.0, float((a * b).sum() / (a * a).sum()))
    yh, (r, v, se, si, tau_m) = Q.scalp_common(q, t, g)
    r2 = [round(float(1 - ((yh[i, m] - Y[i, m]) ** 2).sum() / ((Y[i, m] - Y[i, m].mean()) ** 2).sum
    ()), 3) for i in range(3)]
    src = se - q[8] * si; mm = Q.tm > -0.3; rng = np.random.default_rng(0); cs = []
    for _ in range(30):
        qq = q * (1 + 0.05 * rng.standard_normal(len(q))); _, (_, _, s2, i2, _) = Q.scalp_common(qq
    , t)
        cs.append(float(np.corrcoef((s2 - qq[8] * i2)[mm], src[mm])[0, 1]))
    (rn, vn, Zn, sen, sin), dyn, I, wi = simulate(4000, q)
    a_ = sen - wi * sin; b_ = src.copy(); a_ -= a_[Q.B0:Q.NB].mean(); b_ -= b_[Q.B0:Q.NB].mean()
    res[task] = dict(r2_evoked=r2, gain=round(g, 4), peak=round(float(t[m][np.argmax(np.abs(yh[1, m
    ]))]), 3),
                     robust_median=round(float(np.median(cs)), 3), robust_min=round(float(np.min(cs
    )), 3),
                     net_corr=round(float(np.corrcoef(a_[mm], b_[mm])[0, 1]), 3),
                     net_relerr=round(float(np.sqrt(np.mean((a_[mm] - b_[mm]) ** 2)) / np.std(b_[mm
    ])), 3))
    print(task, res[task], flush=True)
json.dump(res, open(f'{OUT}/q2_check.json', 'w', encoding='utf-8'), indent=1, ensure_ascii=False)
```

q2_shared.py

```
# 本程序及代码是在人工智能工具辅助下完成的：DeepSeek，模型 DeepSeek-V4，
# 开发机构 杭州深度求索人工智能基础技术研究有限公司，版本发布日期 2026 年 4 月 24 日。
"""问题二 项目二：与项目一共用同一群前额叶神经元的动力学参数。

两个项目记录的是同一被试同一块皮层，神经元与突触不会因任务而换一套。项目一的拟合解稳健，
扰动与脉冲网络都验证过，所以把它的六个动力学参数与 wi 固定下来，项目二只放开两样：
  一是刺激驱动 A、tau_d、delay，目标形状不同，驱动可以不同；
  二是一个持续的背景输入 I0，从空跑阶段就加上，代表问题三里项目二维持期才有的海马持续输入，
  它把这群神经元推到另一个工作点，同一个刺激在新工作点上会给出不同的诱发波形。
比较两个版本：只放开驱动，驱动加 I0。再做与项目一相同的稳健性检查与脉冲网络对照。
"""
import numpy as np, json, sys
from scipy.optimize import least_squares
import q2_dynamics as Q
from q2_dynamics import split, run_one, drive_of, LF, tm, NB, B0, SOS_HP, SOS_LP, kuramoto_Z
from scipy.signal import sosfiltfilt
from common import OUT, CH

q1 = np.load(f'{OUT}/q2_fit_Task1_params.npy')        # A tau_d delay tau_e tau_i Je Ji eta wi
    tau_m
DYN = (q1[3] * 1e-3, q1[4] * 1e-3, q1[5], q1[6], q1[7], q1[9] * 1e-3); WI = q1[8]
```

```
def hippo(g, t_on, tau):
    """提示后 t_on 秒开始的海马持续输入，以时间常数 tau 升到 g。"""
    x = np.clip(tm - t_on, 0, None)
    return g * (1 - np.exp(-x / max(tau, 1e-3)))


def scalp(p, t_out, gain=1.0):
    if len(p) == 4:
        A, tau_d, delay, I0 = p; I = drive_of(A, tau_d, delay) + I0
    else:
        A, tau_d, delay, gH, tH, tauH = p; I = drive_of(A, tau_d, delay) + hippo(gH, tH, tauH)
    r, v, se, si = run_one(DYN, I)
    src = se - WI * si; src = src - src[B0:NB].mean()
    y = np.array([np.interp(t_out, tm, gain * c * src) for c in LF.sum(1)])
    y = sosfiltfilt(SOS_LP, sosfiltfilt(SOS_HP, y, axis=1), axis=1)
    return y - y[:, (t_out >= -0.2) & (t_out < 0)].mean(1, keepdims=True), (r, v, se, si, src)


if __name__ == '__main__':
    task = sys.argv[1] if len(sys.argv) > 1 else 'Task2'
    pooled = json.load(open(f'{OUT}/q1_pooled.json', encoding='utf-8'))
    t_out = np.array(pooled['t']); P = pooled['pooled'][task]
    m = (t_out >= -0.3) & (t_out <= 0.65)
    Y = np.array([P[c]['all'] for c in CH])[:, m]
    SE = np.array([P[c]['all_se'] for c in CH])[:, m] + 0.25

    def gain_of(p):
        yh, _ = scalp(p, t_out); a, b = yh[:, m] / SE, Y / SE; den = float((a * a).sum())
        return max(0.0, float((a * b).sum() / den)) if den > 1e-12 and np.isfinite(den) else 0.0

    def resid(p):
        yh, _ = scalp(p, t_out, gain_of(p)); v = ((yh[:, m] - Y) / SE).ravel()
        return np.where(np.isfinite(v), v, 1e3)

    res = dict(task=task, fixed_dynamics_from='Task1')
    rng = np.random.default_rng(0)
    for kind, lo, hi in (('drive', [0.5, 3.0, 20.0, 0.0], [400.0, 40.0, 170.0, 1e-9]),
                         ('drive+I0', [0.5, 3.0, 20.0, -30.0], [400.0, 40.0, 170.0, 60.0]),
                         ('drive+H', [0.5, 3.0, 20.0, 0.0, 0.05, 0.01], [400.0, 40.0, 170.0, 200.0,
     0.6, 0.5])):
        best = None
        for s in range(24):
            p0 = np.array([rng.uniform(a, b) for a, b in zip(lo, hi)])
            if s == 0: p0[:3] = q1[:3]
            p0 = np.clip(p0, lo, np.maximum(hi, np.array(lo) + 1e-9))
            try:
                rr = least_squares(resid, p0, bounds=(lo, np.maximum(hi, np.array(lo) + 1e-9)),
    max_nfev=300, x_scale='jac')
            except Exception:
                continue
            if best is None or rr.cost < best.cost: best = rr
        p = best.x; g = gain_of(p); yh, (r, v, se, si, src) = scalp(p, t_out, g)
        r2 = {c: round(float(1 - ((yh[i, m] - Y[i]) ** 2).sum() / ((Y[i] - Y[i].mean()) ** 2).sum()
    ), 3) for i, c in enumerate(CH)}
        k = {'drive': 3, 'drive+I0': 4, 'drive+H': 6}[kind]; n = Y.size
        # 稳健性：自由参数各扰动 5%，看源项形状
        mm = tm > -0.3; cs = []
        for _ in range(20):
            pp = p * (1 + 0.05 * rng.standard_normal(len(p))); _, (_, _, _, _, s2) = scalp(pp,
    t_out)
            cs.append(np.corrcoef(s2[mm], src[mm])[0, 1])
        Z = kuramoto_Z(r, v, DYN[-1])
        names = ['A', 'tau_d_ms', 'delay_ms', 'I0'] if len(p) == 4 else ['A', 'tau_d_ms', 'delay_ms
    ', 'g_H', 't_H_s', 'tau_H_s']
```

```
        res[kind] = dict(params={a: round(float(b), 4) for a, b in zip(names, p)},
                         gain=round(g, 4), chi2=round(float(2 * best.cost), 1),
                         bic=round(float(2 * best.cost + k * np.log(n)), 1), r2=r2,
                         peak_lat_s=round(float(t_out[m][np.argmax(np.abs(yh[1, m]))]), 3),
                         r_base=round(float(r[B0:NB].mean()), 2), r_peak=round(float(r[NB:].max()),
     2),
                         Z_base=round(float(np.abs(Z[B0:NB]).mean()), 3),
                         robust_corr_median=round(float(np.median(cs)), 3), robust_corr_min=round(
    float(np.min(cs)), 3))
        print(kind, json.dumps(res[kind], ensure_ascii=False), flush=True)
        np.save(f'{OUT}/q2_shared_{task}_{kind}.npy', p)
    json.dump(res, open(f'{OUT}/q2_shared_{task}.json', 'w', encoding='utf-8'), indent=1,
    ensure_ascii=False)
```

q2_decode.py

```
# 本程序及代码是在人工智能工具辅助下完成的：DeepSeek，模型 DeepSeek-V4，
# 开发机构 杭州深度求索人工智能基础技术研究有限公司，版本发布日期 2026 年 4 月 24 日。
"""问题二 第三段：能区分左右三角形刺激响应的特征表示。

平均波幅上左右无显著差，所以在单试次上找。四组特征逐一评比，统一用分层五折交叉验证，
再做标签置换检验给出零分布，避免只报一个看着高于 0.5 的 AUC 就下结论。
"""
import numpy as np, json, sys
import matplotlib; matplotlib.use('Agg'); import matplotlib.pyplot as plt
from scipy.signal import butter, sosfiltfilt, hilbert
from sklearn.model_selection import StratifiedKFold
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.metrics import roc_auc_score
from common import *
from preprocess import *

PRE, POST = int(0.5 * FS), int(1.2 * FS)
TT = (np.arange(PRE + POST) - PRE) / FS
BASE = slice(PRE - int(0.2 * FS), PRE)
BANDS = [('delta', 1, 4), ('theta', 4, 8), ('alpha', 8, 13), ('beta', 13, 20)]


def load_epochs(key):
    d = load(key); tr = trials(d)
    cues = np.array([k['cue'] for k in tr]); dirs = np.array([k['dir'] for k in tr])
    clean, info = clean_all(d)
    bad = info['bad']['Fz'] | info['bad']['F3'] | info['bad']['F4']
    E, ok = [], None
    for name in CH:
        ep, keep, o = make_epochs(clean[name], cues, PRE, POST, BASE, bad)
        E.append(ep); ok = o if ok is None else (ok & o)
    return np.stack(E, 1), dirs[keep], ok            # (n_trial, 3, n_time)


def f_wave(X, t0=0.0, t1=0.80, step=8):
    """时域波形：每通道在窗内降采样后拼起来。"""
    m = (TT >= t0) & (TT <= t1)
    return X[:, :, m][:, :, ::step].reshape(len(X), -1)


def f_lateral(X, t0=0.0, t1=0.80, step=8):
    """偏侧化波形：只留 F3 减 F4 这一路。"""
    m = (TT >= t0) & (TT <= t1)
    return (X[:, 1, m] - X[:, 2, m])[:, ::step]


def f_band(X, t0=0.0, t1=0.80):
```

```
    """分频带能量：每通道每频带一个对数能量，外加 F3 与 F4 的能量差。"""
    m = (TT >= t0) & (TT <= t1)
    out = []
    for lo, hi in [(b[1], b[2]) for b in BANDS]:
        sos = butter(3, [lo / (FS / 2), hi / (FS / 2)], 'band', output='sos')
        p = np.log(np.mean(sosfiltfilt(sos, X, axis=2)[:, :, m] ** 2, axis=2) + 1e-9)
        out.append(p); out.append((p[:, 1] - p[:, 2])[:, None])
    return np.concatenate(out, axis=1)


def f_cov(X, t0=0.0, t1=0.80):
    """协方差的上三角：三通道之间的瞬时耦合结构。"""
    m = (TT >= t0) & (TT <= t1)
    out = []
    for x in X[:, :, m]:
        c = np.cov(x)
        out.append(np.r_[np.log(np.diag(c) + 1e-9), c[0, 1], c[0, 2], c[1, 2]])
    return np.array(out)


FEATS = {'时域波形': f_wave, '偏侧化波形': f_lateral, '频带能量': f_band, '通道协方差': f_cov}


def cv_auc(F, y, seed=0, nfold=5):
    skf = StratifiedKFold(nfold, shuffle=True, random_state=seed)
    pred = np.zeros(len(y), float)
    for tri, tei in skf.split(F, y):
        clf = make_pipeline(StandardScaler(),
                            LogisticRegression(C=0.05, max_iter=3000, class_weight='balanced'))
        clf.fit(F[tri], y[tri]); pred[tei] = clf.predict_proba(F[tei])[:, 1]
    return roc_auc_score(y, pred), (np.round(pred) == y).mean()


def perm_p(F, y, obs, n=300, seed=0):
    rng = np.random.default_rng(seed); cnt = 0; null = []
    for i in range(n):
        yp = rng.permutation(y)
        a, _ = cv_auc(F, yp, seed=i % 5)
        null.append(a); cnt += a >= obs
    return (cnt + 1) / (n + 1), float(np.mean(null)), float(np.std(null))


if __name__ == '__main__':
    nperm = int(sys.argv[1]) if len(sys.argv) > 1 else 200
    EP = {}
    for key in FILES:
        X, dirs, ok = load_epochs(key)
        EP[key] = (X[ok], (dirs[ok] > 0).astype(int))
        print(key, 'trials', ok.sum(), 'right', int((dirs[ok] > 0).sum()), flush=True)
    # 每份数据单独评，再按项目合并两被试
    groups = {k: EP[k] for k in FILES}
    for task, ks in [('Task1', ['A1', 'B1']), ('Task2', ['A2', 'B2'])]:
        groups[task] = (np.concatenate([EP[k][0] for k in ks]),
                        np.concatenate([EP[k][1] for k in ks]))
    res = {}
    for g, (X, y) in groups.items():
        res[g] = {'n': int(len(y)), 'n_right': int(y.sum())}
        for fname, fn in FEATS.items():
            F = fn(X)
            auc, acc = cv_auc(F, y)
            p, mu, sd = perm_p(F, y, auc, n=nperm)
            res[g][fname] = dict(dim=int(F.shape[1]), auc=round(float(auc), 3),
                                 acc=round(float(acc), 3), p=round(float(p), 4),
                                 null_mean=round(mu, 3), null_sd=round(sd, 3))
            print(f'{g:6s} {fname:6s} dim={F.shape[1]:4d} AUC={auc:.3f} acc={acc:.3f} '
                  f'p={p:.4f} null={mu:.3f}+-{sd:.3f}', flush=True)
```

```
    # 时间窗扫描：用最好的那组特征看判别力落在哪一段
    best = max(FEATS, key=lambda f: np.mean([res[t][f]['auc'] for t in ('Task1', 'Task2')]))
    wins = [(a, a + 0.20) for a in np.arange(0.0, 0.81, 0.10)]
    scan = {}
    fig, ax = plt.subplots(figsize=(8, 4))
    for task in ('Task1', 'Task2'):
        X, y = groups[task]
        a = [cv_auc(FEATS[best](X, t0=w0, t1=w1), y)[0] for w0, w1 in wins]
        scan[task] = [round(float(x), 3) for x in a]
        ax.plot([w[0] + 0.10 for w in wins], a, 'o-', label=task)
        print(task, 'window scan', scan[task], flush=True)
    ax.axhline(0.5, color='k', lw=.6); ax.set_xlabel('window center / s'); ax.set_ylabel('AUC')
    ax.legend(); ax.set_title(f'left-right decoding, feature = {best}')
    fig.tight_layout(); fig.savefig(f'{FIGS}/q2_decode_scan.png', dpi=120)
    res['best_feature'] = best; res['window_scan'] = scan
    res['windows'] = [[round(w0, 2), round(w1, 2)] for w0, w1 in wins]
    # 检出限：给定单试次噪声与试次数，双样本 t 检验在 80% 把握度下能查出的最小左右差
    det = {}
    for g, (X, y) in groups.items():
        w = (TT >= 0.3) & (TT <= 0.5)
        lat = (X[:, 1] - X[:, 2])[:, w].mean(1)
        n1, n2 = int((y == 0).sum()), int((y == 1).sum())
        sd = float(np.std(lat, ddof=1))
        mdd = 2.80 * sd * np.sqrt(1 / n1 + 1 / n2)      # 1.96+0.84 双侧 0.05，把握度 0.8
        det[g] = dict(n_left=n1, n_right=n2, single_trial_sd=round(sd, 2),
                      min_detectable_uV=round(float(mdd), 2),
                      observed_uV=round(float(lat[y == 1].mean() - lat[y == 0].mean()), 2))
        print(f'{g:6s} F3-F4 300-500ms  sd={sd:.2f}  可查出的最小差={mdd:.2f} uV  '
              f'实测差={det[g]["observed_uV"]:.2f} uV', flush=True)
    res['detectability'] = det
    json.dump(res, open(f'{OUT}/q2_decode.json', 'w'), indent=1, ensure_ascii=False)
    print('best feature:', best)
```

q2_decode_compare.py

```
# 本程序及代码是在人工智能工具辅助下完成的：DeepSeek，模型 DeepSeek-V4，
# 开发机构 杭州深度求索人工智能基础技术研究有限公司，版本发布日期 2026 年 4 月 24 日。
"""问题二 多分类器对比：四组特征各配四种分类器，看左右三角能否在单试次上区分。

特征与 q2_decode 相同。分类器：逻辑回归、收缩 LDA、RBF 核 SVM、随机森林。
两项目各自合并两被试，分层五折，换 10 个划分种子取 AUC 的均值与标准差，同时记单次交叉验证用时。
"""
import numpy as np, json, time
from sklearn.model_selection import StratifiedKFold
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import roc_auc_score
from common import OUT, FILES
from q2_decode import load_epochs, FEATS

CLFS = {
    '逻辑回归': lambda: make_pipeline(StandardScaler(), LogisticRegression(C=0.05, max_iter=3000,
    class_weight='balanced')),
    '收缩 LDA': lambda: make_pipeline(StandardScaler(), LinearDiscriminantAnalysis(solver='lsqr',
    shrinkage='auto')),
    # SVM 用决策函数值算 AUC；开 probability 的 Platt 校准在小样本上会把排序整体翻转
    'RBF-SVM': lambda: make_pipeline(StandardScaler(), SVC(C=1.0, gamma='scale', class_weight='
    balanced')),
    '随机森林': lambda: RandomForestClassifier(n_estimators=300, min_samples_leaf=3, class_weight='
    balanced', random_state=0, n_jobs=-1),
}
```

```
def cv_auc(F, y, make, seed):
    pred = np.zeros(len(y))
    for tr, te in StratifiedKFold(5, shuffle=True, random_state=seed).split(F, y):
        clf = make(); clf.fit(F[tr], y[tr])
        pred[te] = clf.decision_function(F[te]) if hasattr(clf, 'decision_function') else clf.
    predict_proba(F[te])[:, 1]
    return roc_auc_score(y, pred)


if __name__ == '__main__':
    EP = {}
    for key in FILES:
        X, dirs, ok = load_epochs(key); EP[key] = (X[ok], (dirs[ok] > 0).astype(int))
    out = {}
    for task, ks in (('Task1', ('A1', 'B1')), ('Task2', ('A2', 'B2'))):
        X = np.concatenate([EP[k][0] for k in ks]); y = np.concatenate([EP[k][1] for k in ks])
        out[task] = {}
        for fname, fn in FEATS.items():
            F = fn(X); out[task][fname] = {}
            for cname, make in CLFS.items():
                t0 = time.time(); a = [cv_auc(F, y, make, s) for s in range(10)]
                out[task][fname][cname] = dict(auc=round(float(np.mean(a)), 3), sd=round(float(np.
    std(a)), 3),
                                               sec=round((time.time() - t0) / 10, 2))
                print(task, fname, cname, out[task][fname][cname], flush=True)
    json.dump(out, open(f'{OUT}/q2_decode_compare.json', 'w', encoding='utf-8'), indent=1,
    ensure_ascii=False)
```

q2_svm_perm.py

```
# 本程序及代码是在人工智能工具辅助下完成的：DeepSeek，模型 DeepSeek-V4，
# 开发机构 杭州深度求索人工智能基础技术研究有限公司，版本发布日期 2026 年 4 月 24 日。
"""复核 RBF-SVM 配偏侧化波形的 AUC 是否高于随机：用决策函数值算 AUC，再做 500 次标签置换。"""
import numpy as np, json
from sklearn.model_selection import StratifiedKFold
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import roc_auc_score
from common import OUT, FILES
from q2_decode import load_epochs, f_lateral


def auc(F, y, seed):
    s = np.zeros(len(y))
    for tr, te in StratifiedKFold(5, shuffle=True, random_state=seed).split(F, y):
        clf = make_pipeline(StandardScaler(), SVC(C=1.0, gamma='scale', class_weight='balanced'))
        clf.fit(F[tr], y[tr]); s[te] = clf.decision_function(F[te])
    return roc_auc_score(y, s)


if __name__ == '__main__':
    EP = {k: load_epochs(k) for k in FILES}
    out = {}
    for task, ks in (('Task1', ('A1', 'B1')), ('Task2', ('A2', 'B2'))):
        X = np.concatenate([EP[k][0][EP[k][2]] for k in ks])
        y = np.concatenate([(EP[k][1][EP[k][2]] > 0).astype(int) for k in ks])
        F = f_lateral(X)
        obs = float(np.mean([auc(F, y, s) for s in range(10)]))
        rng = np.random.default_rng(0)
        null = np.array([auc(F, rng.permutation(y), i % 5) for i in range(500)])
        p = (1 + (null >= obs).sum()) / (1 + len(null))
        # 双侧：偏离随机水平 0.5 至少与实测一样远的比例
        p2 = (1 + (np.abs(null - 0.5) >= abs(obs - 0.5)).sum()) / (1 + len(null))
```

```
        out[task] = dict(auc=round(obs, 3), null_mean=round(float(null.mean()), 3), null_sd=round(
    float(null.std()), 3),
                         null_95=round(float(np.percentile(null, 95)), 3), p=round(float(p), 4),
    p_two_sided=round(float(p2), 4))
        print(task, out[task], flush=True)
    json.dump(out, open(f'{OUT}/q2_svm_perm.json', 'w', encoding='utf-8'), indent=1, ensure_ascii=
    False)
```

q3_events.py

```
# 本程序及代码是在人工智能工具辅助下完成的：DeepSeek，模型 DeepSeek-V4，
# 开发机构 杭州深度求索人工智能基础技术研究有限公司，版本发布日期 2026 年 4 月 24 日。
"""问题三 第零步：把通道 9 的时间结构逐试次摊开，找项目一的应答时刻。

项目一没有 ±2 标记。如果目标在被点击时消失，那么 ±1 段的终点就是应答时刻。
先在项目二里验证 ±1 段终点与 ±2 标记的关系，成立的话就把同一规则搬到项目一。
"""
import numpy as np, json
from common import *


def segments(v):
    """所有非零段：(起点，终点，取值)，终点是最后一个非零采样的下一个。取值变化也算新段。"""
    out = []; n = len(v); i = 0
    while i < n:
        if v[i] != 0:
            j = i
            while j < n and v[j] == v[i]: j += 1
            out.append((i, j, float(v[i]))); i = j
        else:
            i += 1
    return out


if __name__ == '__main__':
    res = {}
    for key in FILES:
        d = load(key); cue_on, cue_v = onsets(d[7])
        seg = segments(d[8])
        rows = []
        for k, c in enumerate(cue_on):
            end = cue_on[k + 1] if k + 1 < len(cue_on) else len(d[8])
            ss = [s for s in seg if c < s[0] < end]
            rows.append(dict(cue=int(c), dir=int(cue_v[k]),
                             segs=[(round((a - c) / FS, 3), round((b - c) / FS, 3), v) for a, b, v
    in ss]))
        # 汇总：每试次 ±1 段的起点与时长，±2 段的起点
        on1, dur1, on2, gap12, n1 = [], [], [], [], []
        for r in rows:
            s1 = [s for s in r['segs'] if abs(s[2]) == 1]
            s2 = [s for s in r['segs'] if abs(s[2]) == 2]
            n1.append(len(s1))
            if s1:
                on1.append(s1[0][0]); dur1.append(s1[-1][1] - s1[0][0])
            if s2:
                on2.append(s2[0][0])
                if s1: gap12.append(s2[0][0] - s1[-1][1])
        f = lambda a: [round(float(np.min(a)), 3), round(float(np.median(a)), 3), round(float(np.
    max(a)), 3)] if len(a) else None
        s = dict(n_trial=len(rows), n_seg1_per_trial=sorted(set(n1)),
                 seg1_onset_min_med_max=f(on1), seg1_dur_min_med_max=f(dur1),
                 n_with_2=len(on2), seg2_onset_min_med_max=f(on2),
                 seg2_minus_seg1_end=f(gap12),
                 seg1_dur_sd=round(float(np.std(dur1)), 3) if dur1 else None)
        res[key] = s
        print(key, json.dumps(s, ensure_ascii=False))
```

```
        for r in rows[:4]: print('   ', r['dir'], r['segs'])
        # 项目一里 ±1 与提示不同号的那几次
        bad = [r for r in rows if any(abs(x[2]) == 1 and np.sign(x[2]) != r['dir'] for x in r['segs
    '])]
        if bad: print('   与提示不同号:', [(r['dir'], r['segs']) for r in bad])
        miss = [r for r in rows if not r['segs']]
        if miss: print('   无应答通道事件的试次数:', len(miss))
    json.dump(res, open(f'{OUT}/q3_events.json', 'w', encoding='utf-8'), indent=1, ensure_ascii=
    False)
```

q3_behavior.py

```
# 本程序及代码是在人工智能工具辅助下完成的：DeepSeek，模型 DeepSeek-V4，
# 开发机构 杭州深度求索人工智能基础技术研究有限公司，版本发布日期 2026 年 4 月 24 日。
"""问题三 第一段：行为层。逐试次取应答时刻、反应时、对错，再用证据累加模型拟合反应时分布。

通道 9 的 ±1 段是目标显示区间，它在点击的那一刻结束；项目二在同一采样点另记一个 ±2。
所以两项目的应答时刻都取 ±1 段的终点。对错的判据：项目一看 ±1 的符号是否与提示同向，
项目二看 ±2 的符号是否与 ±1 的符号同向，即点中的是不是与记忆形状相同的那个三角形。

累加模型取单边界的漂移扩散，首达时间服从带平移的 Wald 分布，三个参数是漂移率 nu、
边界 a、非决策时间 t0，噪声强度定为 1。拟合只用反应时，然后把同一组参数放进上下双边界
且起点居中的扩散过程，预测错误率 1/(1+exp(2 nu a))，与实测的错误次数对照，这是一处样本外检验。
"""
import numpy as np, json
from scipy.optimize import minimize
from scipy.stats import binom
from common import *


def segments(v):
    out = []; n = len(v); i = 0
    while i < n:
        if v[i] != 0:
            j = i
            while j < n and v[j] == v[i]: j += 1
            out.append((i, j, int(v[i]))); i = j
        else:
            i += 1
    return out


def trials_full(d):
    """每个试次：提示起点与方向，目标起点与目标侧，应答时刻，点击侧，对错，反应时。"""
    cue_on, cue_v = onsets(d[7]); seg = segments(d[8]); out = []
    for k, c in enumerate(cue_on):
        end = cue_on[k + 1] if k + 1 < len(cue_on) else len(d[8])
        s1 = [s for s in seg if c < s[0] < end and abs(s[2]) == 1]
        s2 = [s for s in seg if c < s[0] < end and abs(s[2]) == 2]
        rec = dict(cue=int(c), dir=int(cue_v[k]), tgt=None, resp=None, side=None,
                   click_side=None, correct=None, rt=None, miss=True)
        if s1:
            a, b, v = s1[0]
            rec.update(tgt=int(a), resp=int(b), side=int(np.sign(v)), rt=(b - a) / FS, miss=False)
            if s2:                                      # 项目二：目标侧与点击侧分开记
                rec['click_side'] = int(np.sign(s2[0][2]))
                rec['correct'] = bool(rec['click_side'] == rec['side'])
            else:                                       # 项目一：目标就在提示那一侧
                rec['click_side'] = rec['side']
                rec['correct'] = bool(rec['side'] == rec['dir'])
        out.append(rec)
    return out


def wald_nll(p, t):
    nu, a, t0 = p
```

```
    if nu <= 0 or a <= 0 or t0 < 0 or t0 >= t.min() - 1e-3: return 1e10
    s = t - t0
    ll = np.log(a) - 0.5 * np.log(2 * np.pi * s ** 3) - (a - nu * s) ** 2 / (2 * s)
    return -ll.sum()


def fit_wald(t):
    best = None
    for t0 in np.linspace(0.2, t.min() - 0.05, 8):
        s = t - t0; m, v = s.mean(), s.var()
        a0 = np.sqrt(m ** 3 / v); nu0 = a0 / m            # 矩估计作初值
        r = minimize(wald_nll, [nu0, a0, t0], args=(t,), method='Nelder-Mead',
                     options=dict(xatol=1e-5, fatol=1e-6, maxiter=4000))
        if best is None or r.fun < best.fun: best = r
    return best


def wald_cdf(x, nu, a, t0):
    from scipy.stats import norm
    s = np.maximum(x - t0, 1e-9)
    return norm.cdf((nu * s - a) / np.sqrt(s)) + np.exp(2 * nu * a) * norm.cdf(-(nu * s + a) / np.
    sqrt(s))


if __name__ == '__main__':
    res, RT = {}, {}
    for key in FILES:
        tr = trials_full(load(key))
        ok = [r for r in tr if not r['miss']]
        rt = np.array([r['rt'] for r in ok]); cor = np.array([r['correct'] for r in ok])
        RT[key] = rt
        fit = fit_wald(rt[cor]); nu, a, t0 = fit.x
        p_err = 1 / (1 + np.exp(2 * nu * a))
        n_err = int((~cor).sum()); n = len(ok)
        # 实测错误数在预测错误率下的双侧二项概率
        p_lo = binom.cdf(n_err, n, p_err); p_hi = 1 - binom.cdf(n_err - 1, n, p_err)
        # 拟合优度：五个分位点上的经验与模型累积概率之差
        qs = np.quantile(rt[cor], [.1, .3, .5, .7, .9])
        ks = float(np.max(np.abs(wald_cdf(qs, nu, a, t0) - np.array([.1, .3, .5, .7, .9]))))
        err_rt = [round(float(x), 3) for x in rt[~cor]]
        res[key] = dict(n=len(tr), n_resp=n, n_miss=len(tr) - n, n_error=n_err, error_rt=err_rt,
                        rt_median=round(float(np.median(rt)), 3), rt_mean=round(float(rt.mean()),
    3),
                        rt_sd=round(float(rt.std(ddof=1)), 3), rt_min=round(float(rt.min()), 3),
                        rt_max=round(float(rt.max()), 3),
                        nu=round(float(nu), 3), a=round(float(a), 3), t0=round(float(t0), 3),
                        decision_time_mean=round(float(a / nu), 3),
                        p_error_pred=float(f'{p_err:.3g}'), n_error_pred=round(float(n * p_err), 2)
    ,
                        binom_p_two_sided=float(f'{min(1, 2 * min(p_lo, p_hi)):.3g}'),
                        quantile_max_dev=round(ks, 3))
        print(key, json.dumps(res[key], ensure_ascii=False), flush=True)
    # 差异分解：项目二减项目一
    for s in 'AB':
        r1, r2 = res[f'{s}1'], res[f'{s}2']
        d = dict(d_rt_median=round(r2['rt_median'] - r1['rt_median'], 3),
                 d_t0=round(r2['t0'] - r1['t0'], 3),
                 d_decision=round(r2['decision_time_mean'] - r1['decision_time_mean'], 3),
                 nu_ratio=round(r2['nu'] / r1['nu'], 3), a_ratio=round(r2['a'] / r1['a'], 3))
        res[f'contrast_{s}'] = d
        print('被试', s, '项目二减项目一', d)
    # 单独拟合时三个参数互相替代，所以改做两个假设的正面对比，每位被试两项目联合拟合：
    #   H决策：两项目共用 t0，漂移率与边界各自取值，项目二多出的时间花在证据累加上；
    #   H串行：两项目共用漂移率与边界，t0 各自取值，项目二多出一段固定时长的模板比对。
    # 两个假设都是五个参数，直接比对数似然。再给一个三参数的全共享模型作底线。
    def joint(kind, t1, t2):
```

```
    def nll(p):
        if kind == 'decision':
            nu1, a1, nu2, a2, t0 = p; return wald_nll([nu1, a1, t0], t1) + wald_nll([nu2, a2,
t0], t2)
        if kind == 'serial':
            nu, a, t01, t02 = p[:4]; return wald_nll([nu, a, t01], t1) + wald_nll([nu, a, t02],
 t2)
        nu, a, t0 = p; return wald_nll([nu, a, t0], np.r_[t1, t2])
    f1, f2 = fit_wald(t1).x, fit_wald(t2).x
    tmin = min(t1.min(), t2.min())
    starts = {'decision': [[f1[0], f1[1], f2[0], f2[1], t0] for t0 in np.linspace(0.2, tmin -
0.05, 8)],
              'serial': [[(f1[0] + f2[0]) / 2, (f1[1] + f2[1]) / 2, f1[2], f2[2]],
                         [f1[0], f1[1], f1[2], f1[2] + np.median(t2) - np.median(t1)],
                         [f2[0], f2[1], f2[2] - np.median(t2) + np.median(t1), f2[2]]],
              'shared': [list(fit_wald(np.r_[t1, t2]).x)]}[kind]
    best = min((minimize(nll, s, method='Nelder-Mead', options=dict(maxiter=20000, xatol=1e-6,
fatol=1e-7))
                for s in starts), key=lambda r: r.fun)
    return best
for s in 'AB':
    t1 = RT[f'{s}1'][[r for r in range(len(RT[f'{s}1']))]]; t2 = RT[f'{s}2']
    cmp_ = {}
    for kind, k in (('decision', 5), ('serial', 4), ('shared', 3)):
        b = joint(kind, t1, t2)
        n = len(t1) + len(t2)
        cmp_[kind] = dict(nll=round(float(b.fun), 2), k=k, bic=round(float(2 * b.fun + k * np.
log(n)), 2),
                          params=[round(float(x), 3) for x in b.x])
    res[f'hypothesis_{s}'] = cmp_
    print('被试', s, {k: (v['nll'], v['bic']) for k, v in cmp_.items()},
          '决策减串行 BIC =', round(cmp_['decision']['bic'] - cmp_['serial']['bic'], 2), flush=
True)
# 错误：扩散过程给出的错误率几乎为零，实测的错误要由失误成分承担。失误时随机点一侧，
# 所以错误率等于失误率的一半。四份数据合并估计失误率，给 95% 精确区间。
from scipy.stats import beta
n_err = sum(res[k]['n_error'] for k in FILES); n_all = sum(res[k]['n_resp'] for k in FILES)
lam = 2 * n_err / n_all
lo, hi = beta.ppf(0.025, n_err, n_all - n_err + 1), beta.ppf(0.975, n_err + 1, n_all - n_err)
res['lapse'] = dict(n_error=n_err, n=n_all, lapse_rate=round(lam, 4),
                    error_rate_ci95=[round(float(lo), 4), round(float(hi), 4)],
                    error_rt=res['B1']['error_rt'], rt_median_B1=res['B1']['rt_median'])
print('失误', res['lapse'])
json.dump(res, open(f'{OUT}/q3_behavior.json', 'w', encoding='utf-8'), indent=1, ensure_ascii=
False)
```

q3_eeg.py

```
# 本程序及代码是在人工智能工具辅助下完成的：DeepSeek，模型 DeepSeek-V4，
# 开发机构 杭州深度求索人工智能基础技术研究有限公司，版本发布日期 2026 年 4 月 24 日。
"""问题三 第二段：脑电这一侧的三组观测量，全部截到应答前 100 ms。

一 频带：维持期 M 取提示后 0.6 到 2.0 s，目标还没出现；决策期 D 取目标出现到应答前 0.1 s。
   四个频带的包络在整段连续信号上算一次再按窗取均值，避免短窗的边缘效应。
   每个窗先除以自己四个频带之和，只比相对分布，因为四份记录的绝对量纲不一致。
二 应答锁时电位：以应答为零点取 -1.5 到 -0.1 s，基线取目标出现前 0.2 s。按反应时中位数把试次
   分成快慢两半，看应答前 0.1 s 的电位高度与此前 0.5 s 的斜率。证据累加到固定边界的说法预言：
   高度与快慢无关，斜率快组更陡。
三 脑与行为：维持期与决策期的相对能量与反应时的秩相关，反应时先在每份记录内标准化再合并两被试。
"""
import numpy as np, json
import matplotlib; matplotlib.use('Agg'); import matplotlib.pyplot as plt
from scipy.signal import butter, sosfiltfilt, hilbert
from scipy.stats import ttest_ind, spearmanr, linregress
from common import *
from preprocess import *
```

```
from q3_behavior import trials_full

BANDS = [('delta', 1, 4), ('theta', 4, 8), ('alpha', 8, 13), ('beta', 13, 20)]
PRE_R = int(1.5 * FS); END_R = int(0.1 * FS)
TT_R = (np.arange(PRE_R - END_R) - PRE_R) / FS            # -1.5 .. -0.1 s


def envelopes(x):
    out = {}
    for bn, lo, hi in BANDS:
        sos = butter(3, [lo / (FS / 2), hi / (FS / 2)], 'band', output='sos')
        out[bn] = np.abs(hilbert(sosfiltfilt(sos, x))) ** 2
    return out


def window_power(env, bad, a, b):
    if b - a < int(0.25 * FS) or bad[a:b].mean() > 0.2: return None
    p = np.array([env[bn][a:b].mean() for bn, _, _ in BANDS])
    return p / p.sum()


if __name__ == '__main__':
    res = {}; ROWS = []; ERP = {}
    for key in FILES:
        d = load(key); tr = trials_full(d); clean, info = clean_all(d)
        bad = info['bad']['Fz'] | info['bad']['F3'] | info['bad']['F4']
        ENV = {c: envelopes(clean[c]) for c in CH}
        rt = np.array([r['rt'] for r in tr]); rtz = (rt - rt.mean()) / rt.std()
        for i, r in enumerate(tr):
            if r['miss']: continue
            row = dict(key=key, task=key[1], subj=key[0], rt=r['rt'], rtz=float(rtz[i]), correct=r
    ['correct'])
            for wn, (a, b) in dict(M=(r['cue'] + int(0.6 * FS), r['cue'] + int(2.0 * FS)),
                                   D=(r['tgt'], r['resp'] - END_R)).items():
                ps = [window_power(ENV[c], bad, a, b) for c in CH]
                row[wn] = None if any(p is None for p in ps) else np.mean(ps, 0)
            ROWS.append(row)
        # 应答锁时：不做去趋势，去趋势会把累加的斜坡一起拿掉
        ep = {c: [] for c in CH}; keep = []
        for i, r in enumerate(tr):
            if r['miss'] or not r['correct']: continue
            a, b = r['resp'] - PRE_R, r['resp'] - END_R
            base = slice(r['tgt'] - int(0.2 * FS), r['tgt'])
            if a < 0 or bad[a:b].any() or bad[base].any(): continue
            for c in CH: ep[c].append(clean[c][a:b] - clean[c][base].mean())
            keep.append(r['rt'])
        keep = np.array(keep)
        X = {c: np.array(ep[c]) for c in CH}
        p2p = np.ptp(np.mean([X[c] for c in CH], 0), 1)
        med = np.median(p2p); mad = 1.4826 * np.median(np.abs(p2p - med))
        ok = p2p < min(150, med + 3 * mad)
        ERP[key] = dict(X={c: X[c][ok] for c in CH}, rt=keep[ok])
        print(key, '应答锁时可用试次', int(ok.sum()), '/', len(ok), flush=True)

    # ---------- 一 频带：项目二减项目一 ----------
    band = {}
    for wn in ('M', 'D'):
        for bi, (bn, _, _) in enumerate(BANDS):
            a = np.array([r[wn][bi] for r in ROWS if r['task'] == '1' and r[wn] is not None])
            b = np.array([r[wn][bi] for r in ROWS if r['task'] == '2' and r[wn] is not None])
            t, p = ttest_ind(b, a, equal_var=False)
            dz = (b.mean() - a.mean()) / np.sqrt((a.var(ddof=1) + b.var(ddof=1)) / 2)
            per = {}
            for s in 'AB':
                a_ = np.array([r[wn][bi] for r in ROWS if r['key'] == f'{s}1' and r[wn] is not None
    ])
```

```
            b_ = np.array([r[wn][bi] for r in ROWS if r['key'] == f'{s}2' and r[wn] is not None
])
            per[s] = round(float(b_.mean() - a_.mean()), 4)
        band[f'{wn}_{bn}'] = dict(task1=round(float(a.mean()), 4), task2=round(float(b.mean()),
 4),
                                  diff=round(float(b.mean() - a.mean()), 4), per_subject=per,
                                  n=[len(a), len(b)], t=round(float(t), 2), p=float(f'{p:.3g}')
,
                                  d=round(float(dz), 2))
        print(f'{wn} {bn}: 项目一 {a.mean():.4f} 项目二 {b.mean():.4f} 差 {b.mean()-a.mean():+.4
f} '
              f'A {per["A"]:+.4f} B {per["B"]:+.4f} t={t:+.2f} p={p:.3g} d={dz:+.2f}', flush=
True)
res['band'] = band

# ---------- 二 应答锁时电位 ----------
erp = {}
sl = (TT_R >= -0.6) & (TT_R <= -0.1); lv = (TT_R >= -0.2) & (TT_R <= -0.1)
for task in '12':
    out = {}
    for c in CH:
        Xs = np.concatenate([ERP[k]['X'][c] for k in FILES if k[1] == task])
        rts = np.concatenate([(ERP[k]['rt'] <= np.median(ERP[k]['rt'])) for k in FILES if k[1]
== task])
        level = Xs[:, lv].mean(1)
        slope = np.array([linregress(TT_R[sl], x[sl]).slope for x in Xs])
        fast, slow = rts, ~rts
        tl, pl = ttest_ind(level[fast], level[slow], equal_var=False)
        ts, ps = ttest_ind(slope[fast], slope[slow], equal_var=False)
        out[c] = dict(n=[int(fast.sum()), int(slow.sum())],
                      level_fast=round(float(level[fast].mean()), 2), level_slow=round(float(
level[slow].mean()), 2),
                      level_p=float(f'{pl:.3g}'),
                      slope_fast=round(float(slope[fast].mean()), 2), slope_slow=round(float(
slope[slow].mean()), 2),
                      slope_p=float(f'{ps:.3g}'),
                      level_all=round(float(level.mean()), 2),
                      level_all_t=round(float(level.mean() / (level.std(ddof=1) / np.sqrt(len(
level)))), 2),
                      wave_fast=[round(float(x), 3) for x in robust_mean(Xs[fast])],
                      wave_slow=[round(float(x), 3) for x in robust_mean(Xs[slow])])
        print(f'项目{task} {c}: 应答前电位 快 {out[c]["level_fast"]:+.2f} 慢 {out[c]["level_slow
"]:+.2f} '
              f'p={pl:.3g} | 斜率 快 {out[c]["slope_fast"]:+.2f} 慢 {out[c]["slope_slow"]:+.2f}
 uV/s p={ps:.3g} '
              f'| 全体 {out[c]["level_all"]:+.2f} t={out[c]["level_all_t"]:+.2f}', flush=True)
    erp[f'Task{task}'] = out
res['resp_locked'] = erp; res['t_resp'] = [round(float(x), 4) for x in TT_R]

# ---------- 三 脑与行为 ----------
bb = {}
for task in '12':
    for wn in ('M', 'D'):
        rr = [r for r in ROWS if r['task'] == task and r[wn] is not None and r['correct']]
        for bi, (bn, _, _) in enumerate(BANDS):
            rho, p = spearmanr([r[wn][bi] for r in rr], [r['rtz'] for r in rr])
            bb[f'Task{task}_{wn}_{bn}'] = [round(float(rho), 3), float(f'{p:.3g}'), len(rr)]
    print(f'项目{task} 与反应时的秩相关:',
          {k.split('_', 1)[1]: v[:2] for k, v in bb.items() if k.startswith(f'Task{task}')},
flush=True)
res['brain_behavior'] = bb
json.dump(res, open(f'{OUT}/q3_eeg.json', 'w', encoding='utf-8'), indent=1, ensure_ascii=False)

fig, ax = plt.subplots(2, 3, figsize=(15, 7), sharex=True)
for ti, task in enumerate('12'):
    for ci, c in enumerate(CH):
```

```
            o = erp[f'Task{task}'][c]
            ax[ti, ci].plot(TT_R, o['wave_fast'], label='fast'); ax[ti, ci].plot(TT_R, o['wave_slow
    '], label='slow')
            ax[ti, ci].axvline(-0.1, color='k', lw=.5); ax[ti, ci].set_title(f'Task{task} {c}')
            ax[ti, ci].legend(fontsize=7)
    fig.tight_layout(); fig.savefig(f'{FIGS}/q3_resp_locked.png', dpi=110)
    print('done')
```

q3_ocular.py

```
# 本程序及代码是在人工智能工具辅助下完成的：DeepSeek，模型 DeepSeek-V4，
# 开发机构 杭州深度求索人工智能基础技术研究有限公司，版本发布日期 2026 年 4 月 24 日。
"""问题三 排查：项目二维持期的 delta 升高与总能量升高，是不是眼动带进来的。

水平眼动在 F3 与 F4 上反号，所以看 F3 减 F4 这一路；垂直眼动与额中线的慢电位在三个电极上同号，
看三者平均这一路。再看绝对能量：每个频带取对数，项目二减项目一，两被试分开给。
眼动的慢漂移还有一个特征：能量集中在 1 到 2 Hz 且与时间窗内的线性趋势强相关，这里也量一下
窗内线性趋势解释的方差比例。清洗后的信号缓存到 out/clean_*.npz，供后面复用。
"""
import numpy as np, json, os
from scipy.stats import ttest_ind
from common import *
from preprocess import clean_all
from q3_behavior import trials_full
from q3_eeg import envelopes, BANDS


def cleaned(key):
    f = f'{OUT}/clean_{key}.npz'
    if os.path.exists(f):
        z = np.load(f); return {c: z[c] for c in CH}, z['bad']
    d = load(key); c_, info = clean_all(d)
    bad = info['bad']['Fz'] | info['bad']['F3'] | info['bad']['F4']
    np.savez(f, bad=bad, **c_); return c_, bad


if __name__ == '__main__':
    res = {}; W = {}
    for key in FILES:
        tr = trials_full(load(key)); cl, bad = cleaned(key)
        sig = {'Fz': cl['Fz'], 'F3': cl['F3'], 'F4': cl['F4'],
               'lat': cl['F3'] - cl['F4'], 'cm': (cl['Fz'] + cl['F3'] + cl['F4']) / 3}
        ENV = {s: envelopes(x) for s, x in sig.items()}
        rows = []
        for r in tr:
            a, b = r['cue'] + int(0.6 * FS), r['cue'] + int(2.0 * FS)
            if bad[a:b].any(): continue
            row = {s: np.log([ENV[s][bn][a:b].mean() for bn, _, _ in BANDS]) for s in sig}
            t = np.arange(b - a)
            for s in ('lat', 'cm'):
                x = sig[s][a:b]; p = np.polyfit(t, x, 1)
                row[f'{s}_trend_r2'] = 1 - np.var(x - np.polyval(p, t)) / np.var(x)
                row[f'{s}_rms'] = float(np.std(x))
            rows.append(row)
        W[key] = rows
    for s_ in 'AB':
        out = {}
        for s in ('Fz', 'F3', 'F4', 'lat', 'cm'):
            a = np.array([r[s] for r in W[f'{s_}1']]); b = np.array([r[s] for r in W[f'{s_}2']])
            out[s] = {bn: [round(float(b[:, i].mean() - a[:, i].mean()), 3),
                           float(f'{ttest_ind(b[:, i], a[:, i], equal_var=False)[1]:.2g}')]
                      for i, (bn, _, _) in enumerate(BANDS)}
        for s in ('lat', 'cm'):
            for m in ('trend_r2', 'rms'):
                a = np.array([r[f'{s}_{m}'] for r in W[f'{s_}1']]); b = np.array([r[f'{s}_{m}'] for
     r in W[f'{s_}2']])
```

```
                out[f'{s}_{m}'] = [round(float(a.mean()), 3), round(float(b.mean()), 3)]
        res[s_] = out
        print('被试', s_, '项目二减项目一 的对数能量差 [差, p]')
        for s in ('Fz', 'F3', 'F4', 'lat', 'cm'):
            print('   ', s, out[s])
        print('    窗内线性趋势解释的方差 [项目一, 项目二]: 偏侧', out['lat_trend_r2'], '共模', out['
    cm_trend_r2'])
        print('    窗内均方根 uV [项目一, 项目二]: 偏侧', out['lat_rms'], '共模', out['cm_rms'],
    flush=True)
    json.dump(res, open(f'{OUT}/q3_ocular.json', 'w', encoding='utf-8'), indent=1, ensure_ascii=
    False)
```

q3_model.py

```
# 本程序及代码是在人工智能工具辅助下完成的：DeepSeek，模型 DeepSeek-V4，
# 开发机构 杭州深度求索人工智能基础技术研究有限公司，版本发布日期 2026 年 4 月 24 日。
"""问题三 第三段：认知的宏观模型。

前额叶群沿用问题二拟合好的 QIF 精确平均场，头皮映射也沿用问题二的增益与导联场，所以模型输出
直接是微伏。它同时收到三路输入：
  丘脑 LGN 一路：以 alpha 频率谐振的随机振子，代表丘脑皮层的 alpha 节律，幅度 g_T；
  海马一路：theta 相位上骑着 gamma 爆发，theta 幅度有慢包络，幅度 g_H；
  背景一路：宽带的 Ornstein-Uhlenbeck 噪声，幅度 sigma_B。
观测量取三个电极的平均，过与实测相同的 1 Hz 高通与 20 Hz 低通，频带能量也用同一套包络算法。

theta 的频率不拟合，取行为拟合给出的串行比对时长的倒数：项目二多出的那段比对被看作海马的
一个 theta 周期。项目差异只允许落在两处：海马耦合 g_H 与丘脑门控 kappa。四个嵌套模型对比：
都不变、只变海马、只变丘脑、两者都变。

拟合目标八个数：项目一维持期的相对能量分布四个，项目二减项目一的对数绝对能量四个。两者都与
输出增益无关。不拟合绝对幅度，因为单试次的背景脑电大半来自这一群以外的皮层，问题二按平均诱发
响应标定的增益管不到它。检验两处：跨被试预测；项目差频谱在 theta 段的峰是否落在行为给出的频率上。
"""
import numpy as np, json, os, sys
import matplotlib; matplotlib.use('Agg'); import matplotlib.pyplot as plt
from scipy.optimize import least_squares
from scipy.signal import butter, sosfiltfilt, hilbert, iirpeak, lfilter, welch
from common import *
from q2_dynamics import _one, drive_of, LF
from q3_eeg import BANDS
from q3_behavior import trials_full
from q3_ocular import cleaned

DT = 0.1e-3; FSIM = int(1 / DT)                 # 与问题二同一步长
DS = int(round(1 / (250 * DT)))                 # 抽到 250 Hz 的步数
TBURN = 0.5; NWIN = 60
SOS_HP = butter(2, 1.0 / (FS / 2), 'high', output='sos')
SOS_LP = butter(4, 20 / (FS / 2), 'low', output='sos')
SOS_B = [butter(3, [lo / (FS / 2), hi / (FS / 2)], 'band', output='sos') for _, lo, hi in BANDS]
F_GAMMA = 90.0
LF_CM = float(LF.sum(1).mean())                 # 两侧源到三电极平均的导联系数


def unit(x):
    return (x - x.mean()) / (x.std() + 1e-12)


# 海马与背景输入的形状。网格 w_slow 取 0 0.3 0.6 0.85，tau_b 取 20 与 5 ms，在只变海马的模型上
# 选 chi2 最小的一组；选定之后四个嵌套模型都在同一组形状下重拟合，比较才公平。
SHAPE = dict(tau_env=0.3, D_phi=2.0, w_slow=0.3, tau_b=0.005)


def make_inputs(seed, nt, f_theta, tau_env=0.3, D_phi=2.0, w_slow=0.0, tau_b=0.02):
    """三路输入的形状，都归一到单位标准差，幅度留给拟合。固定种子即公共随机数。
    海马一路 = 慢包络 x [(1-w) * theta 门控的 gamma 爆发 + w]，w 是不随 theta 起伏的慢调制所占的份
    额。"""
```

```
    rng = np.random.default_rng(seed); t = np.arange(nt) * DT
    b = lfilter([DT / tau_b], [1, -(1 - DT / tau_b)], rng.standard_normal(nt))
    bb, aa = iirpeak(10.0, 2.0, fs=FSIM); al = lfilter(bb, aa, rng.standard_normal(nt))
    phi = 2 * np.pi * f_theta * t + np.cumsum(np.sqrt(2 * D_phi * DT) * rng.standard_normal(nt))
    env = np.abs(lfilter([DT / tau_env], [1, -(1 - DT / tau_env)], rng.standard_normal(nt)))
    th = (1 + np.cos(phi)) / 2
    H = env / env.mean() * ((1 - w_slow) * th * (1 + 0.5 * th * np.cos(2 * np.pi * F_GAMMA * t)) +
    w_slow)
    return unit(b), unit(al), H / H.std(), phi


def measure(y, starts, lens):
    """与数据同一把尺子：连续包络，逐窗取绝对能量、相对能量、对数能量与均方根。"""
    env = np.array([np.abs(hilbert(sosfiltfilt(s, y))) ** 2 for s in SOS_B])
    P = np.array([env[:, a:a + n].mean(1) for a, n in zip(starts, lens)])
    rms = np.array([y[a:a + n].std() for a, n in zip(starts, lens)])
    return P, rms


class Model:
    def __init__(self, Q2, gain, f_theta, win, seeds=(11, 12), evoked=None):
        self.Q2, self.gain, self.win = Q2, gain, win
        self.nt = int((TBURN + NWIN * win) / DT)
        self.inp = [make_inputs(s, self.nt, f_theta, **SHAPE) for s in seeds]
        self.ev = np.zeros(self.nt)
        if evoked is not None:                  # 每个窗开头放一次目标诱发的前馈驱动
            for k in range(NWIN):
                s = int((TBURN + k * win) / DT); n = min(len(evoked), self.nt - s)
                self.ev[s:s + n] += evoked[:n]
        self.starts = [int((TBURN + k * win) * FS) for k in range(NWIN)]
        self.lens = [int(win * FS)] * NWIN

    def run(self, sB, gT, gH):
        tau_e, tau_i, Je, Ji, eta, tau_m, wi = self.Q2; Ps, Rs = [], []
        for b, al, H, _ in self.inp:
            I = np.ascontiguousarray(sB * b + gT * al + gH * H + self.ev)
            r, v, se, si = _one(tau_e, tau_i, Je, Ji, eta, tau_m, I, self.nt, DT, 1.0)
            src = (se - wi * si)[::DS]
            y = np.interp(np.arange(0, self.nt * DT, 1 / FS), np.arange(len(src)) * DT * DS, src)
            y = self.gain * LF_CM * sosfiltfilt(SOS_LP, sosfiltfilt(SOS_HP, y))
            P, rms = measure(y, self.starts, self.lens); Ps.append(P); Rs.append(rms)
        return np.concatenate(Ps), np.concatenate(Rs)

    def predict(self, q, kind):
        sB, gT, gH1, dH, kap = q
        if kind in ('none', 'T'): dH = 0.0
        if kind in ('none', 'H'): kap = 1.0
        P1, R1 = self.run(sB, gT, gH1); P2, R2 = self.run(sB, kap * gT, gH1 + dH)
        return summarize(P1, R1, P2, R2)[0]


def summarize(P1, R1, P2, R2):
    """八个目标：项目一相对分布 4，项目二减项目一的对数能量差 4；同时给出标准误。"""
    rel1 = P1 / P1.sum(1, keepdims=True); L1, L2 = np.log(P1), np.log(P2)
    Y = np.r_[rel1.mean(0), L2.mean(0) - L1.mean(0)]
    se = lambda x: x.std(0, ddof=1) / np.sqrt(len(x))
    SE = np.r_[se(rel1), np.sqrt(se(L1) ** 2 + se(L2) ** 2)]
    return Y, SE


PARAM = ['sigma_B', 'g_T', 'g_H1', 'dg_H', 'kappa']
LO = [0.0, 0.0, 0.0, 0.0, 0.05]; HI = [150.0, 150.0, 150.0, 300.0, 3.0]
FREE = {'none': [0, 1, 2], 'H': [0, 1, 2, 3], 'T': [0, 1, 2, 4], 'HT': [0, 1, 2, 3, 4]}


def fit(model, Y, SE, kind, starts):
```

```
    free = FREE[kind]; base = np.array([5.0, 5.0, 2.0, 0.0, 1.0])

    def full(x):
        q = base.copy(); q[free] = x; return q

    def res(x):
        v = (model.predict(full(x), kind) - Y) / SE
        return np.where(np.isfinite(v), v, 1e3)
    best = None
    for s in starts:
        x0 = np.clip(np.array(s, float)[free], np.array(LO)[free] + 1e-6, np.array(HI)[free])
        r = least_squares(res, x0, bounds=(np.array(LO)[free], np.array(HI)[free]), max_nfev=250,
                          diff_step=1e-2, x_scale='jac')
        if best is None or r.cost < best.cost: best = r
    return full(best.x), 2 * best.cost, len(free)


def data_windows(keys, win):
    """逐试次的窗：M 为提示后 0.6 到 2.0 s，D 为目标出现到应答前 0.1 s，S 为提示后 0.5 到 2.2 s 供谱估计。"""
    out = {}
    for key in keys:
        tr = trials_full(load(key)); cl, bad = cleaned(key)
        y = (cl['Fz'] + cl['F3'] + cl['F4']) / 3
        st, ln = [], []
        for r in tr:
            if r['miss'] or not r['correct']: continue
            a, b = {'M': (r['cue'] + int(0.6 * FS), r['cue'] + int(2.0 * FS)),
                    'D': (r['tgt'], r['resp'] - int(0.1 * FS)),
                    'S': (r['cue'] + int(0.5 * FS), r['cue'] + int(2.2 * FS))}[win]
            if bad[a:b].any(): continue
            st.append(a); ln.append(b - a)
        out[key] = (y, st, ln)
    return out


def data_targets(keys1, keys2, win):
    W1, W2 = data_windows(keys1, win), data_windows(keys2, win)
    P1 = np.concatenate([measure(*W1[k])[0] for k in keys1]); R1 = np.concatenate([measure(*W1[k])
    [1] for k in keys1])
    P2 = np.concatenate([measure(*W2[k])[0] for k in keys2]); R2 = np.concatenate([measure(*W2[k])
    [1] for k in keys2])
    return summarize(P1, R1, P2, R2)


def diff_spectrum(keys1, keys2, nboot=500, seed=0):
    """项目二减项目一的对数功率谱，以及 3 到 8 Hz 内局部峰频率的自助法区间。"""
    def psds(keys):
        W = data_windows(keys, 'S'); out = []
        for k in keys:
            y, st, ln = W[k]
            for a, n in zip(st, ln):
                f, p = welch(y[a:a + n], FS, nperseg=n, nfft=1024, window='hann', detrend='linear')
                out.append(p)
        return f, np.array(out)
    f, A = psds(keys1); _, B = psds(keys2)
    m = (f >= 3) & (f <= 8)
    d = lambda a, b: np.log(b.mean(0)) - np.log(a.mean(0))
    D = d(A, B); rng = np.random.default_rng(seed); pk = []
    for _ in range(nboot):
        a = A[rng.integers(0, len(A), len(A))]; b = B[rng.integers(0, len(B), len(B))]
        dd = d(a, b)[m]
        # 去掉 1/f 的倾斜后再找峰：在 3 到 8 Hz 内对对数频率做线性去趋势
        x = np.log(f[m]); dd = dd - np.polyval(np.polyfit(x, dd, 1), x)
        pk.append(f[m][np.argmax(dd)])
    x = np.log(f[m]); dm = D[m] - np.polyval(np.polyfit(x, D[m], 1), x)
```

```
    return f, D, float(f[m][np.argmax(dm)]), np.percentile(pk, [2.5, 97.5]), dm, f[m]


if __name__ == '__main__':
    beh = json.load(open(f'{OUT}/q3_behavior.json', encoding='utf-8'))
    ser = [beh[f'hypothesis_{s}']['serial']['params'] for s in 'AB']
    d_serial = float(np.mean([p[3] - p[2] for p in ser])); f_theta = 1 / d_serial
    print(f'串行比对时长 {d_serial:.3f} s, theta 频率取 {f_theta:.2f} Hz', flush=True)
    q2 = np.load(f'{OUT}/q2_fit_Task1_params.npy')
    gain = json.load(open(f'{OUT}/q2_fit_Task1.json', encoding='utf-8'))['gain']
    Q2 = (q2[3] * 1e-3, q2[4] * 1e-3, q2[5], q2[6], q2[7], q2[9] * 1e-3, q2[8])
    model = Model(Q2, gain, f_theta, win=1.4)
    if len(sys.argv) > 1 and sys.argv[1] == 'probe':
        for g in [(5, 5, 0), (50, 50, 0), (200, 200, 0), (50, 50, 50)]:
            P, R = model.run(*g); print(g, 'rms uV', round(float(R.mean()), 2),
                                        'rel', np.round((P / P.sum(1, keepdims=True)).mean(0), 3))
        sys.exit()

    Y, SE = data_targets(('A1', 'B1'), ('A2', 'B2'), 'M')
    print('数据 项目一相对', np.round(Y[:4], 3), '对数差', np.round(Y[4:8], 3), flush=True)
    starts = [[40, 40, 5, 60, 0.8], [20, 80, 1, 150, 0.5], [80, 20, 20, 40, 1.0], [40, 60, 0.5,
    300, 0.7]]
    res = dict(f_theta=round(f_theta, 3), d_serial=round(d_serial, 3), gain_q2=gain,
               data_M=np.round(Y, 4).tolist(), data_M_se=np.round(SE, 4).tolist(),
               targets=['rel1_delta', 'rel1_theta', 'rel1_alpha', 'rel1_beta',
                        'dlog_delta', 'dlog_theta', 'dlog_alpha', 'dlog_beta'])
    fits = {}
    for kind in ('none', 'H', 'T', 'HT'):
        q, chi2, k = fit(model, Y, SE, kind, starts)
        pred = model.predict(q, kind); bic = chi2 + k * np.log(len(Y))
        fits[kind] = q
        res[f'fit_{kind}'] = dict(params=dict(zip(PARAM, np.round(q, 3).tolist())), chi2=round(
    float(chi2), 2),
                                  k=k, bic=round(float(bic), 2), pred=np.round(pred, 4).tolist())
        print(f'{kind:4s} chi2={chi2:9.2f} BIC={bic:9.2f} 相对 {np.round(pred[:4],3)} 对数差 {np.
    round(pred[4:8],3)} '
              f'参数 {np.round(q,2)}', flush=True)
    best = min(('H', 'T', 'HT'), key=lambda k: res[f'fit_{k}']['bic'])
    res['best'] = best; print('最优', best)

    # 检验一: 跨被试
    YA, SEA = data_targets(('A1',), ('A2',), 'M'); YB, SEB = data_targets(('B1',), ('B2',), 'M')
    qA, _, _ = fit(model, YA, SEA, best, starts); pB = model.predict(qA, best)
    res['cross_subject'] = dict(pred_B=np.round(pB, 4).tolist(), data_B=np.round(YB, 4).tolist(),
                                data_B_se=np.round(SEB, 4).tolist(),
                                z=np.round((pB - YB) / SEB, 2).tolist())
    print('跨被试 预测B', np.round(pB, 3), '\n        实测B', np.round(YB, 3), flush=True)

    # 检验二: 项目差频谱在 theta 段的峰
    ds = {}
    for s in ('AB', 'A', 'B'):
        f, D, pk, ci, dm, fm = diff_spectrum(tuple(x + '1' for x in s), tuple(x + '2' for x in s))
        ds[s] = dict(peak_hz=round(pk, 2), ci95=np.round(ci, 2).tolist())
        print(f'被试 {s}: 项目差谱去倾斜后 theta 段峰 {pk:.2f} Hz, 95% 区间 {np.round(ci,2)}', flush
    =True)
        if s == 'AB':
            ds['f'] = np.round(f[(f >= 1) & (f <= 20)], 3).tolist(); ds['dlog'] = np.round(D[(f >=
    1) & (f <= 20)], 4).tolist()
            ds['theta_f'] = np.round(fm, 3).tolist(); ds['theta_detrended'] = np.round(dm, 4).
    tolist()
    res['diff_spectrum'] = ds

    # 海马输入的 theta-gamma 相幅耦合, 调制指数的算法与 Daume 2024 一致
    b, al, H, phi = model.inp[0]
    amp = np.abs(hilbert(sosfiltfilt(butter(3, [70 / (FSIM / 2), 140 / (FSIM / 2)], 'band', output
    ='sos'), H)))
```

```
    ph = np.mod(phi, 2 * np.pi); bins = np.linspace(0, 2 * np.pi, 19)
    pa = np.array([amp[(ph >= bins[i]) & (ph < bins[i + 1])].mean() for i in range(18)]); pa /= pa.
    sum()
    res['hippo_MI'] = round(float((np.log(18) + (pa * np.log(pa)).sum()) / np.log(18)), 4)
    json.dump(res, open(f'{OUT}/q3_model.json', 'w', encoding='utf-8'), indent=1, ensure_ascii=
    False)
    print('MI', res['hippo_MI'], 'done')
```

q3_revise.py

```
# 本程序及代码是在人工智能工具辅助下完成的：DeepSeek，模型 DeepSeek-V4，
# 开发机构 杭州深度求索人工智能基础技术研究有限公司，版本发布日期 2026 年 4 月 24 日。
"""问题三修订计算。
一 剔除含眨眼的维持期窗口后复算项目差：眨眼峰前后 0.3 s 内有落进窗口的，整窗不用。
二 两个对照模型：纯慢输入，即海马一路去掉 theta 与 gamma 只留慢包络，w=1；
   只变背景，即项目二只把背景噪声幅度乘一个系数。与只变海马、只变丘脑一起比较。
三 只变海马模型的优化器对比：多起点信赖域、Nelder-Mead、差分进化。
"""
import numpy as np, json, time, sys
from scipy.optimize import least_squares, minimize, differential_evolution
import q3_model as M
from common import *
from preprocess import fix_saturation, highpass, detect_blinks
from q3_behavior import trials_full
from q3_ocular import cleaned

part = sys.argv[1] if len(sys.argv) > 1 else 'all'
OUTF = f'{OUT}/q3_revise.json'
try: res = json.load(open(OUTF, encoding='utf-8'))
except Exception: res = {}

q2 = np.load(f'{OUT}/q2_fit_Task1_params.npy'); gain = json.load(open(f'{OUT}/q2_fit_Task1.json',
    encoding='utf-8'))['gain']
Q2 = (q2[3] * 1e-3, q2[4] * 1e-3, q2[5], q2[6], q2[7], q2[9] * 1e-3, q2[8])
beh = json.load(open(f'{OUT}/q3_behavior.json', encoding='utf-8'))
f_theta = 1 / np.mean([p[3] - p[2] for p in (beh[f'hypothesis_{s}']['serial']['params'] for s in '
    AB')])


def blink_peaks(key):
    d = load(key)
    xs = [highpass(fix_saturation(d[i])[0]) for i in range(3)]
    pk, _ = detect_blinks(xs)
    return pk


def targets_noblink(keys1, keys2, pad=0.3):
    out = {}
    for tag, keys in (('1', keys1), ('2', keys2)):
        P, R = [], []
        for k in keys:
            y, st, ln = M.data_windows((k,), 'M')[k]
            pk = blink_peaks(k)
            keep_s, keep_l = [], []
            for a, n in zip(st, ln):
                if not np.any((pk >= a - int(pad * FS)) & (pk < a + n + int(pad * FS))):
                    keep_s.append(a); keep_l.append(n)
            p, r = M.measure(y, keep_s, keep_l); P.append(p); R.append(r)
            out[f'n_{k}'] = [len(keep_s), len(st)]
        out[tag] = (np.concatenate(P), np.concatenate(R))
    Y, SE = M.summarize(*out['1'], *out['2'])
    return Y, SE, {k: v for k, v in out.items() if k.startswith('n_')}


if part in ('blink', 'all'):
    Yall, SEall = M.data_targets(('A1', 'B1'), ('A2', 'B2'), 'M')
```

```
    Ynb, SEnb, counts = targets_noblink(('A1', 'B1'), ('A2', 'B2'))
    res['noblink'] = dict(all_dlog=np.round(Yall[4:], 3).tolist(), noblink_dlog=np.round(Ynb[4:],
    3).tolist(),
                          noblink_se=np.round(SEnb[4:], 3).tolist(), all_rel1=np.round(Yall[:4], 3)
    .tolist(),
                          noblink_rel1=np.round(Ynb[:4], 3).tolist(), counts=counts)
    print('全部窗口 对数差', np.round(Yall[4:], 3), '| 剔除眨眼窗口', np.round(Ynb[4:], 3), counts,
    flush=True)
    json.dump(res, open(OUTF, 'w', encoding='utf-8'), indent=1, ensure_ascii=False)

if part in ('control', 'all'):
    Y, SE = M.data_targets(('A1', 'B1'), ('A2', 'B2'), 'M')
    starts = [[2.3, 1.65, 0.1, 2.3, 1.0], [2, 3, 0.5, 5, 1.0], [0.5, 1, 0.1, 1, 1.0]]
    ctrl = {}
    # 纯慢输入：海马一路只留慢包络
    M.SHAPE.update(w_slow=1.0); mS = M.Model(Q2, gain, f_theta, win=1.4)
    q, chi2, k = M.fit(mS, Y, SE, 'H', starts); p = mS.predict(q, 'H')
    ctrl['slow_only'] = dict(chi2=round(float(chi2), 1), k=k, bic=round(float(chi2 + k * np.log(8))
    , 1),
                             pred=np.round(p, 3).tolist(), params=np.round(q, 3).tolist())
    print('纯慢输入', ctrl['slow_only'], flush=True)
    # 只变背景：项目二背景幅度乘 beta，海马与丘脑不变
    M.SHAPE.update(w_slow=0.3); mB = M.Model(Q2, gain, f_theta, win=1.4)

    def predB(x):
        sB, gT, gH, beta = x
        P1, R1 = mB.run(sB, gT, gH); P2, R2 = mB.run(beta * sB, gT, gH)
        return M.summarize(P1, R1, P2, R2)[0]
    best = None
    for s in ([2.3, 1.65, 0.1, 1.5], [2, 3, 0.5, 3.0], [1, 1, 0.1, 0.5]):
        r = least_squares(lambda x: np.nan_to_num((predB(x) - Y) / SE, nan=1e3), s,
                          bounds=([0, 0, 0, 0.05], [150, 150, 150, 20]), max_nfev=250, diff_step=1e
    -2, x_scale='jac')
        if best is None or r.cost < best.cost: best = r
    p = predB(best.x)
    ctrl['background_only'] = dict(chi2=round(float(2 * best.cost), 1), k=4, bic=round(float(2 *
    best.cost + 4 * np.log(8)), 1),
                                   pred=np.round(p, 3).tolist(), params=np.round(best.x, 3).tolist
    ())
    print('只变背景', ctrl['background_only'], flush=True)
    res['controls'] = ctrl
    json.dump(res, open(OUTF, 'w', encoding='utf-8'), indent=1, ensure_ascii=False)

if part in ('opt', 'all'):
    M.SHAPE.update(w_slow=0.3); mH = M.Model(Q2, gain, f_theta, win=1.4)
    Y, SE = M.data_targets(('A1', 'B1'), ('A2', 'B2'), 'M')
    LO = np.array([0, 0, 0, 0.0]); HI = np.array([20, 20, 20, 20.0])

    def rH(x):
        return np.nan_to_num((mH.predict(np.r_[x, 1.0], 'H') - Y) / SE, nan=1e3)

    def cH(x):
        x = np.clip(x, LO, HI); r = rH(x); return float(r @ r)
    rng = np.random.default_rng(7); opt = {}
    runs = {'多起点信赖域': 3, 'Nelder-Mead': 3, '差分进化': 2}
    for name, nrun in runs.items():
        cs, ts = [], []
        for k in range(nrun):
            t0 = time.time()
            if name == '多起点信赖域':
                best = None
                for _ in range(3):
                    r = least_squares(rH, LO + (HI - LO) * rng.random(4) * 0.3, bounds=(LO, HI),
    max_nfev=120, diff_step=1e-2, x_scale='jac')
                    if best is None or r.cost < best.cost: best = r
                c = 2 * best.cost
```

```
            elif name == 'Nelder-Mead':
                r = minimize(cH, LO + (HI - LO) * rng.random(4) * 0.3, method='Nelder-Mead',
    options=dict(maxfev=400)); c = cH(r.x)
            else:
                r = differential_evolution(cH, list(zip(LO, HI)), popsize=6, maxiter=12, seed=int(
    rng.integers(1 << 30)), polish=False); c = cH(r.x)
            cs.append(c); ts.append(time.time() - t0)
            print(name, k, round(c, 1), round(ts[-1], 1), flush=True)
        opt[name] = dict(chi2=[round(x, 1) for x in cs], sec=[round(x, 1) for x in ts])
    res['optimizers_H'] = opt
    json.dump(res, open(OUTF, 'w', encoding='utf-8'), indent=1, ensure_ascii=False)
```

q_sens.py

```
# 本程序及代码是在人工智能工具辅助下完成的：DeepSeek，模型 DeepSeek-V4，
# 开发机构 杭州深度求索人工智能基础技术研究有限公司，版本发布日期 2026 年 4 月 24 日。
"""灵敏度分析。
一 问题二项目一：每个参数单独上下调 10%，看三通道 R2 的变化；再对全部参数同时随机扰动 5%，看源项形状与
    原解的相关。
二 问题三：海马输入的 theta 频率、慢成分份额 w、背景噪声时间常数 tau_b 各取几档，重拟合只变海马的模型，
    看 chi2 与预测的对数差。
"""
import numpy as np, json, sys
import q2_dynamics as Q
from common import OUT, CH

part = sys.argv[1] if len(sys.argv) > 1 else 'all'
res = {}

if part in ('q2', 'all'):
    pooled = json.load(open(f'{OUT}/q1_pooled.json', encoding='utf-8'))
    t_out = np.array(pooled['t']); P = pooled['pooled']['Task1']
    m = (t_out >= -0.1) & (t_out <= 0.65)          # 与正文一致，R2 在诱发段上算
    Y = np.array([P[c]['all'] for c in CH])[:, m]
    SE = np.array([P[c]['all_se'] for c in CH])[:, m] + 0.25
    q = np.load(f'{OUT}/q2_fit_Task1_params.npy'); g0 = json.load(open(f'{OUT}/q2_fit_Task1.json',
    encoding='utf-8'))['gain']

    def r2(qq, g=g0):
        yh, (r, v, se, si, tm_) = Q.scalp_common(qq, t_out, g)
        return [float(1 - ((yh[i, m] - Y[i]) ** 2).sum() / ((Y[i] - Y[i].mean()) ** 2).sum()) for i
     in range(3)], se - qq[8] * si

    base, src0 = r2(q); mm = Q.tm > -0.3
    one = {}
    for k, name in enumerate(Q.NAMES):
        row = {}
        for f in (0.9, 1.1):
            qq = q.copy(); qq[k] *= f; rr, s = r2(qq)
            row[str(f)] = [round(x, 3) for x in rr] + [round(float(np.corrcoef(s[mm], src0[mm])[0,
    1]), 3)]
        one[name] = row
        print(name, row, flush=True)
    rng = np.random.default_rng(0); cs = []
    for _ in range(40):
        qq = q * (1 + 0.05 * rng.standard_normal(len(q))); _, s = r2(qq)
        cs.append(float(np.corrcoef(s[mm], src0[mm])[0, 1]))
    res['q2'] = dict(base_r2=[round(x, 3) for x in base], one_at_a_time=one,
                     joint5_corr_median=round(float(np.median(cs)), 3), joint5_corr_min=round(float
    (np.min(cs)), 3))
    print('联合扰动 5%', res['q2']['joint5_corr_median'], res['q2']['joint5_corr_min'], flush=True)

if part in ('q3', 'all'):
    import q3_model as M
    q2 = np.load(f'{OUT}/q2_fit_Task1_params.npy'); gain = json.load(open(f'{OUT}/q2_fit_Task1.json
    ', encoding='utf-8'))['gain']
```

```
    Q2p = (q2[3] * 1e-3, q2[4] * 1e-3, q2[5], q2[6], q2[7], q2[9] * 1e-3, q2[8])
    Y, SE = M.data_targets(('A1', 'B1'), ('A2', 'B2'), 'M')
    best = json.load(open(f'{OUT}/q3_model.json', encoding='utf-8'))['fit_H']['params']
    start = [[best['sigma_B'], best['g_T'], best['g_H1'], best['dg_H'], 1.0], [2, 3, 0.5, 5, 1]]
    grid = []
    for ft in (4.44, 5.5):
        for w in (0.0, 0.3, 0.6):
            grid.append(dict(f_theta=ft, w_slow=w, tau_b=0.005))
    grid += [dict(f_theta=4.44, w_slow=0.3, tau_b=0.02)]
    out = []
    for gcfg in grid:
        M.SHAPE.update(w_slow=gcfg['w_slow'], tau_b=gcfg['tau_b'])
        mod = M.Model(Q2p, gain, gcfg['f_theta'], win=1.4)
        qq, chi2, k = M.fit(mod, Y, SE, 'H', start); p = mod.predict(qq, 'H')
        row = dict(gcfg, chi2=round(float(chi2), 1), dlog=[round(float(x), 3) for x in p[4:]], dg_H
    =round(float(qq[3]), 3))
        out.append(row); print(row, flush=True)
    res['q3'] = out

prev = {}
try: prev = json.load(open(f'{OUT}/q_sens.json', encoding='utf-8'))
except Exception: pass
prev.update(res)
json.dump(prev, open(f'{OUT}/q_sens.json', 'w', encoding='utf-8'), indent=1, ensure_ascii=False)
```